

Fine Tuning 예제 링크

<https://colab.research.google.com/drive/1MacxGADL7YsWlCcxhnH9pxFu36jYl0bC?usp=sharing>

임베딩(Embedding), 잠재공간(Latent Space)

2025년 10월

표현(Representation ~ Feature)



Individual decimal places

	Thousands	Hundreds	Tens	Units
1	M	C	X	I
2	MM	CC		
3	MMM	CCC	XXX	III
4		CD	XL	IV
5		D	L	V
6		DC	LX	VI
7		DCC		
8		DCC		
9		CM	XC	IX

MMMDCCCLXXXIX – MCDXLVII = ???

3889 – 1447 = ???

정보 처리 분야의 많은 일은

정확한 표현을 필요로 한다.

매우 어려울 수도 있다.

매우 어려워질 수도 있다.

very easy

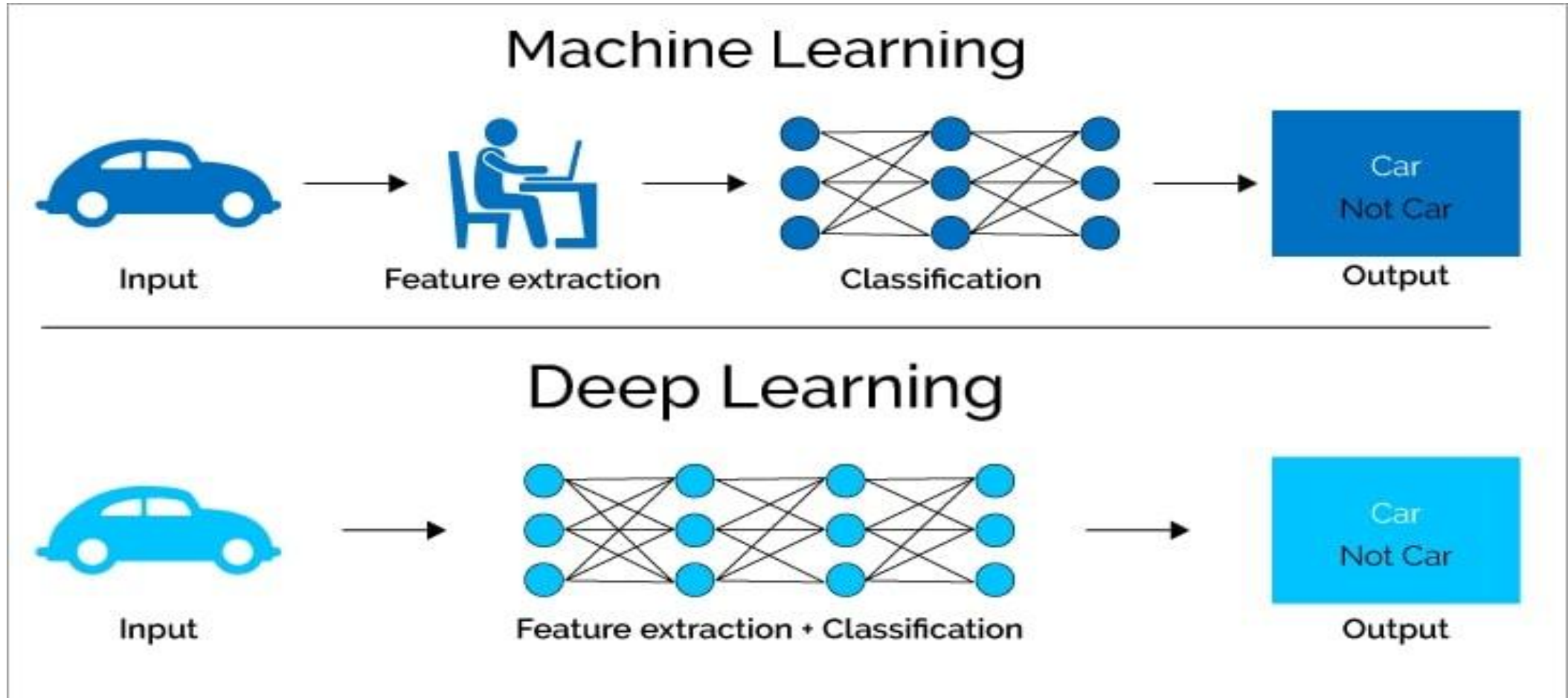
or very difficult depending on how the information is

represented.)

로마 숫자 vs 아라비아 숫자

출처 : https://en.wikipedia.org/wiki/Roman_numerals

숫자 표현(feature) 찾기 : 머신러닝 vs 딥러닝



발췌 : <https://levity.ai/blog/difference-machine-learning-deep-learning>

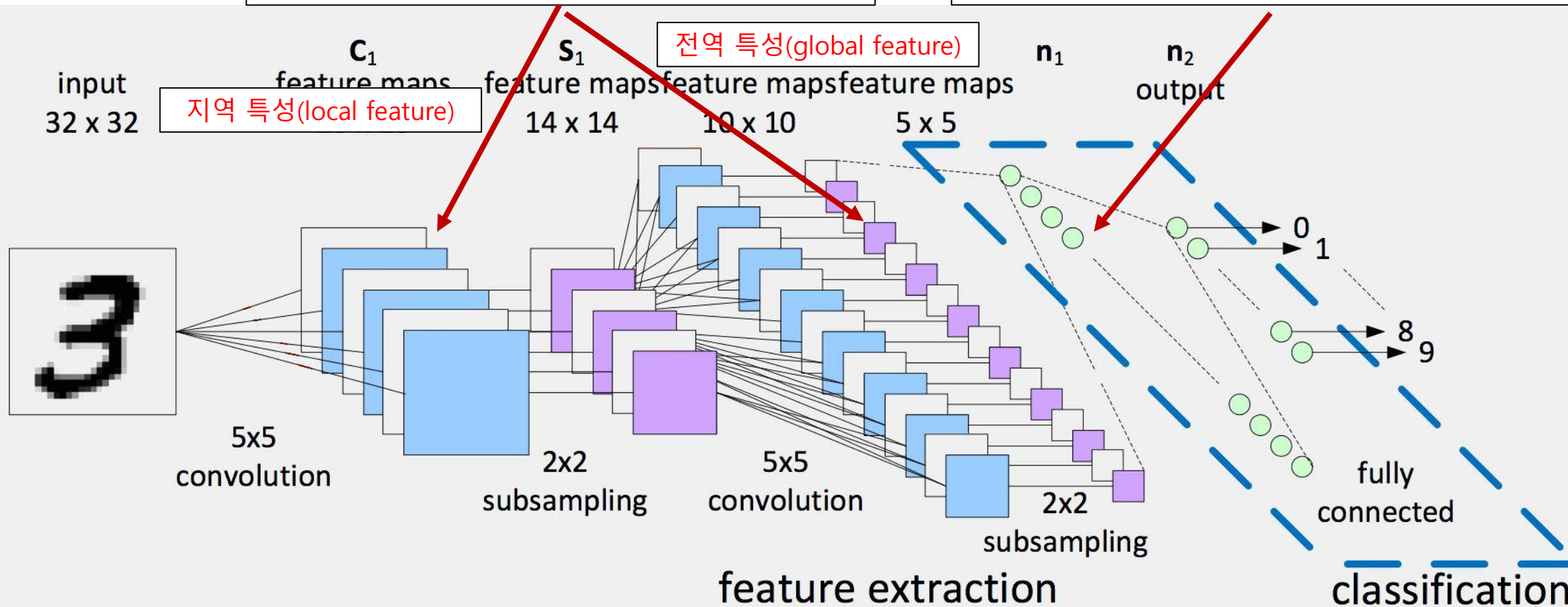
딥러닝 : 판단하기 쉬운 표현(feature) 찾기

이미지 잠재 공간(벡터 ~ 숫자) 생성

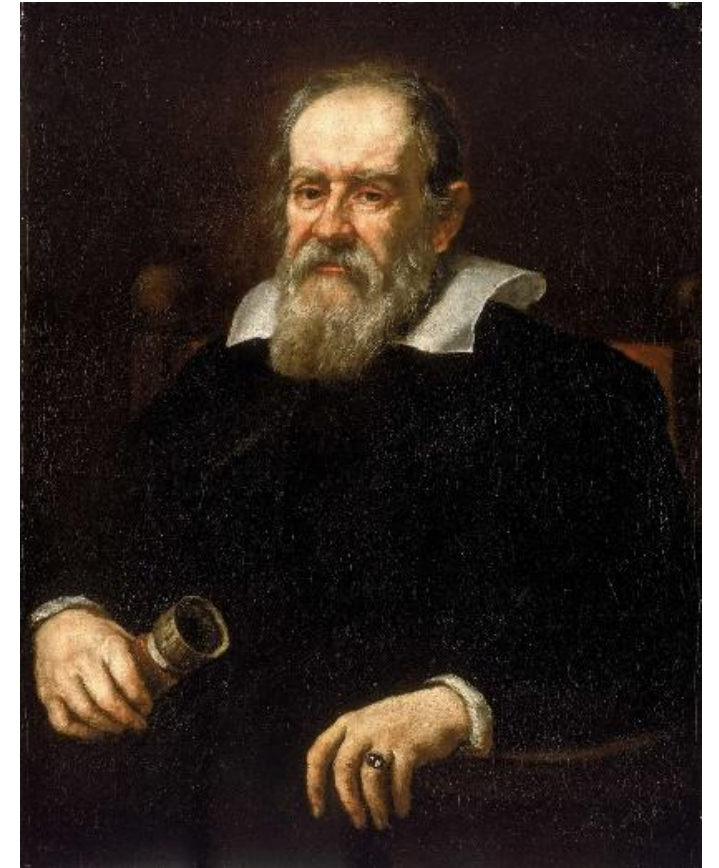


특성(feature) 찾기

판별(classification)



과학적 접근의 기초 → 수치화



셀 수 있는 모든 것을 세고, 잴 수 있는 모든 것을 재어라.

그리고 잴 수 없는 것은 잴 수 있게 만들어라

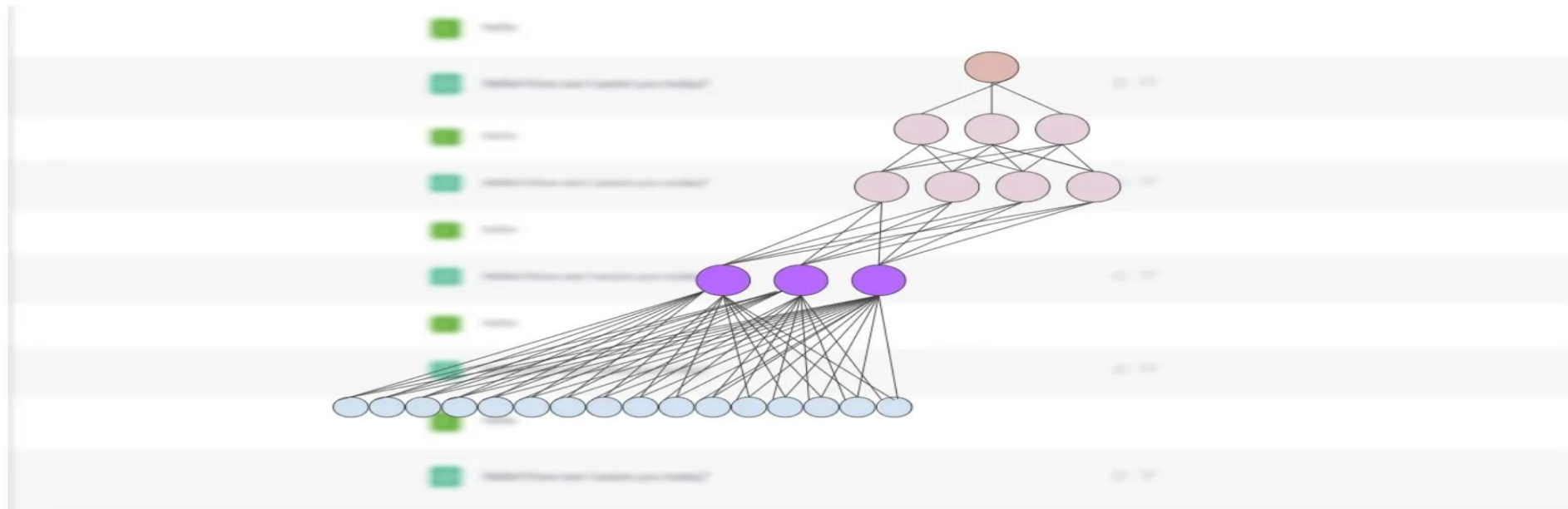
– 갈릴레오

ChatGPT의 숨겨진 무기 – 임베딩(Embedding)



Embeddings: ChatGPT's Secret Weapon

Embeddings, and how they help ChatGPT predict the next word



(image by author)

- 컴퓨터가 다루는(다룰 수 있는) 것은 숫자
- 인공지능은 숫자를 예측한다.

목차

- 잠재공간 (Latent Space)
- 언어의 잠재공간
- 그림의 잠재공간
- 소리의 잠재공간
- 정리

잠재 공간(latent space) : 말, 그림, 소리가 수치화 된 공간

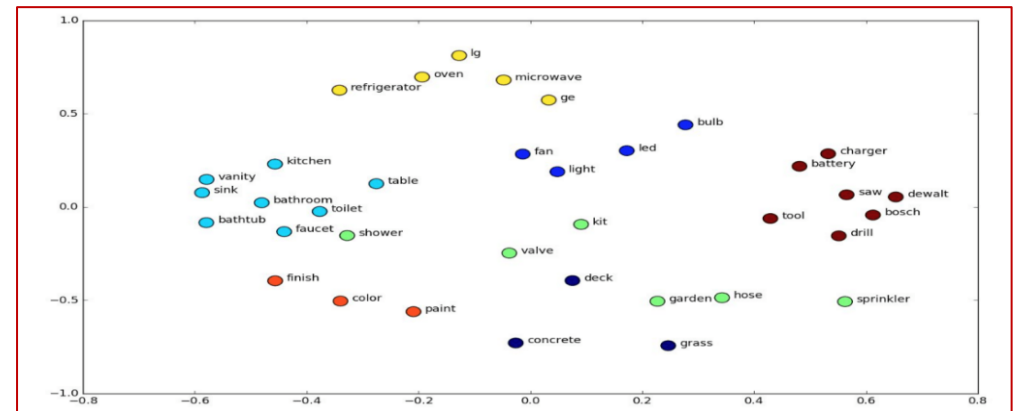
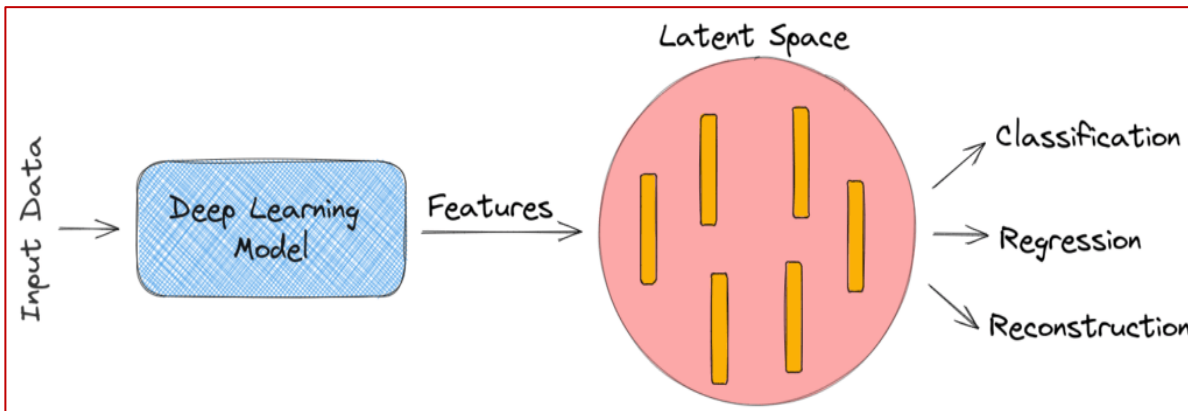
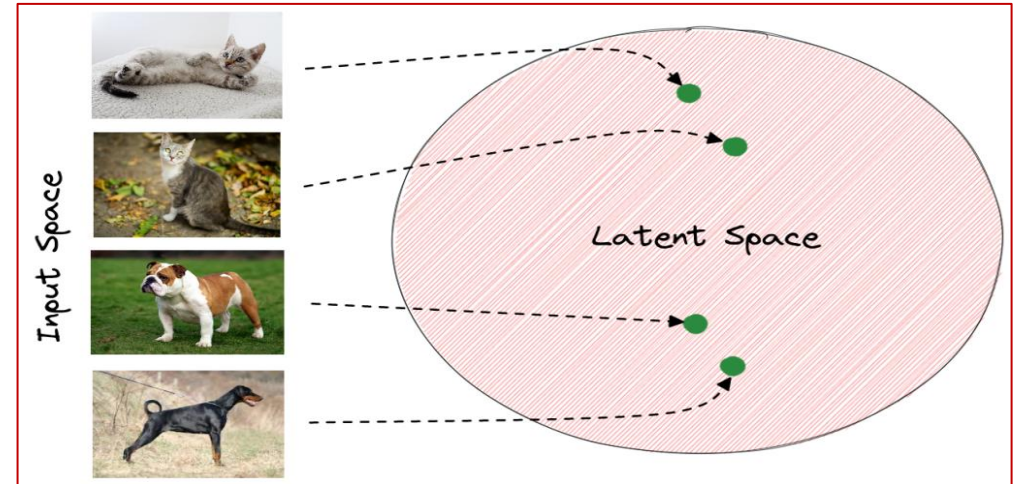
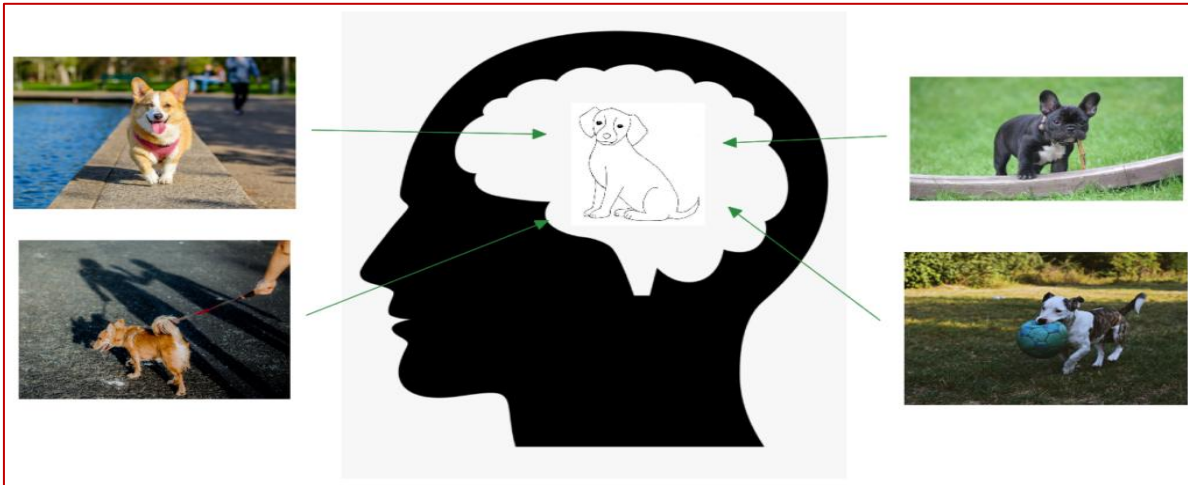
Formally, a **latent space** is defined as an abstract multi-dimensional space that encodes a meaningful internal representation of externally observed events. **Samples that are similar in the external world are positioned close to each other in the latent space.**

To better understand the concept, let's think about how humans perceive the world. **We are able to understand a broad range of topics by encoding each observed event in a compressed representation in our brain.**

- 관찰되는 이벤트의 **의미 있는 내적 표현**이 부호화된 추상적 다차원 공간
- 비슷한 샘플이 가까운 자리에 위치함
- 이벤트를 인간이 인식하는 방식에 비유될 수 있음
- 외부 입력이 **함축된 형태로 뇌에 저장되는 것과 유사함**

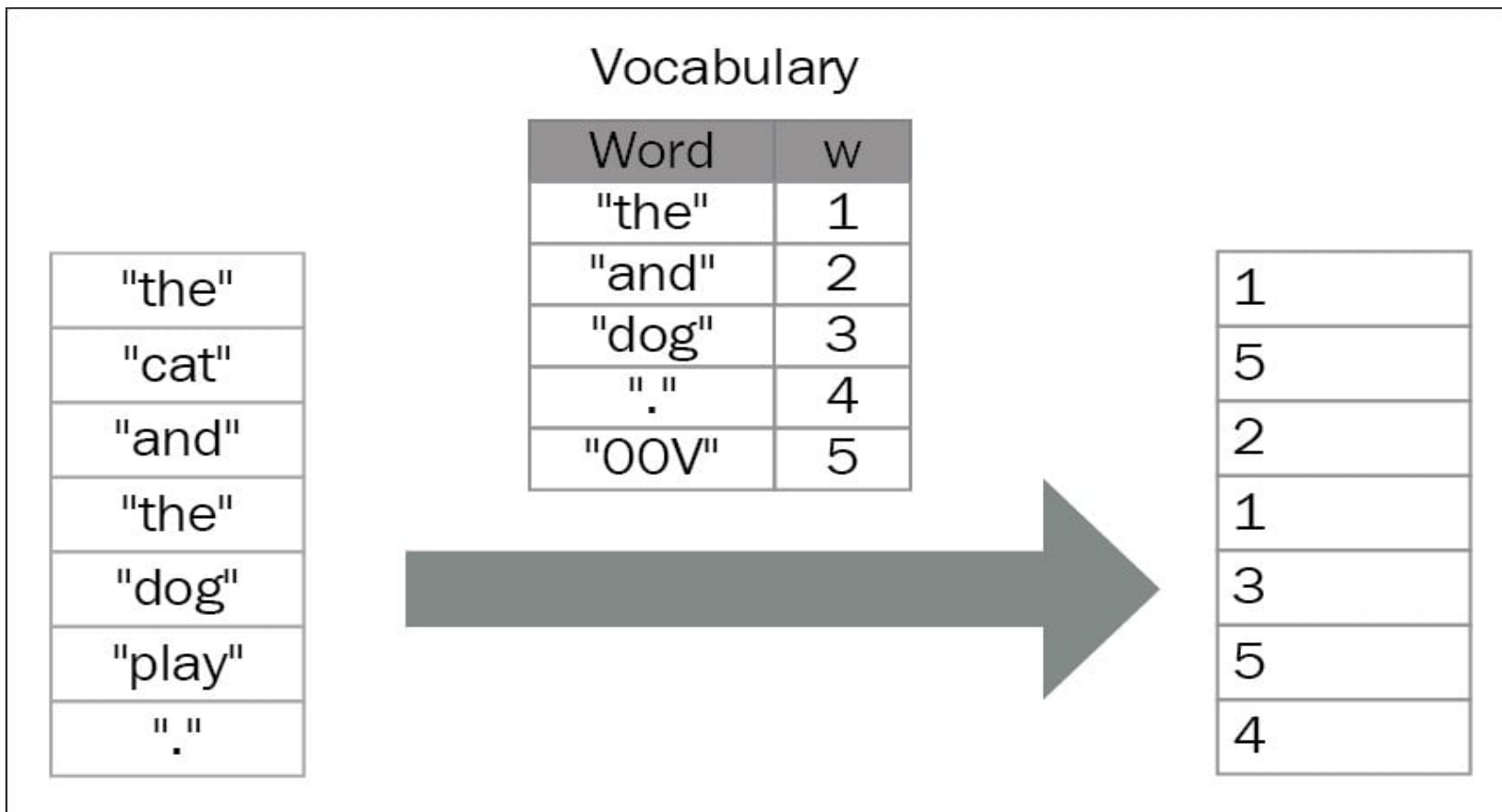
잠재 공간(latent space) : 말, 그림, 소리가 수치화 된 공간

- 딥러닝 모델은 내부적으로 잠재 공간을 생성한다.
- 생성된 잠재 공간에서는 분류, 회귀, 생성이 쉬워진다.



언어의 공간

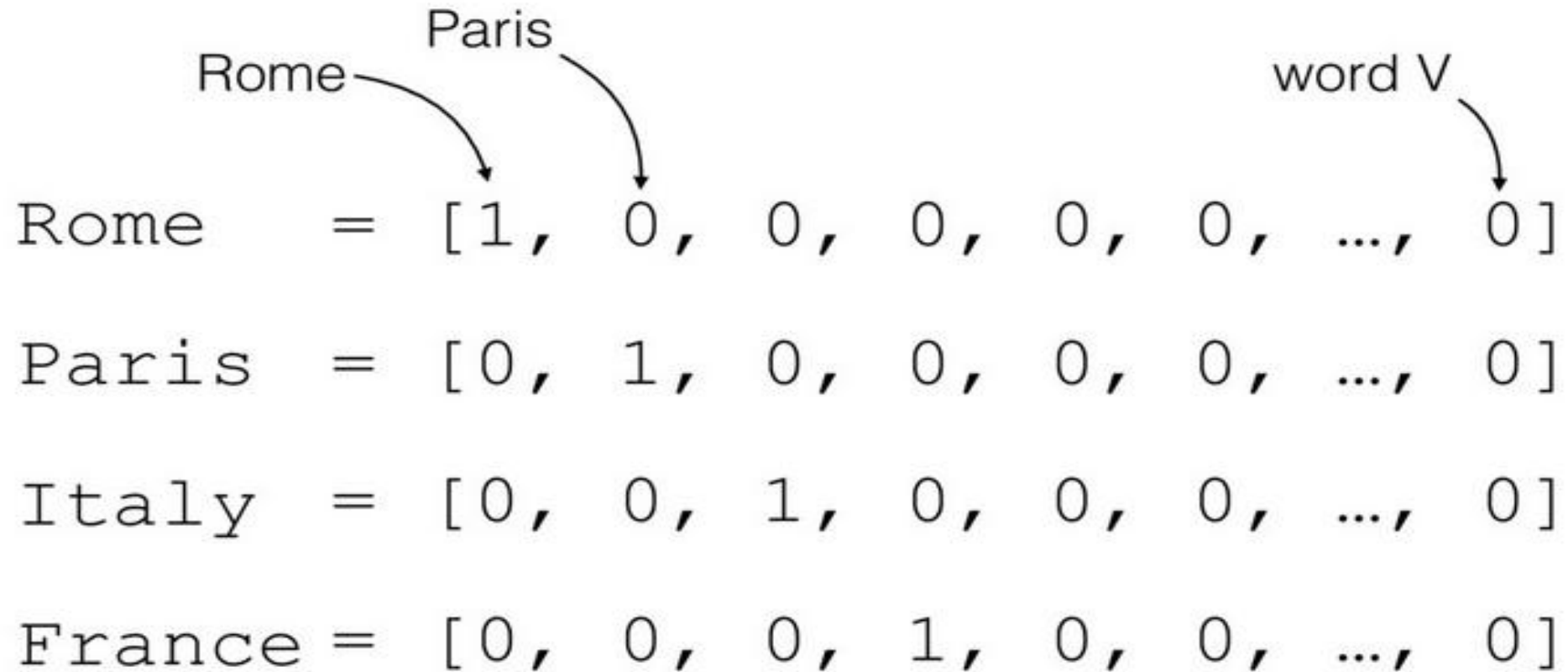
말(단어)를 수치화 할 수 있을까?



단어	코드
사과	1
치킨	2
바나나	3
양배추	4

출처 : <https://subscription.packtpub.com/book/data/9781786465825/3/ch03lvl1sec32/encoding-and-embedding>

- 언어의 의미를 담지 못한 수치화



출처 : <https://medium.com/intelligentmachines/word-embedding-and-one-hot-encoding-ad17b4bbe111>

- 단어 표현에 큰 벡터가 필요 (One-Hot encoding)
- 언어의 의미를 담지 못한 수치화

텍스트의 수치화 : 벡터화 - 지정된 특성을 담은 수치에 대응



	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	<u>0.93</u>	<u>0.95</u>	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
⋮						

출처 : <https://medium.com/intelligentmachines/word-embedding-and-one-hot-encoding-ad17b4bbe111>

- 단어의 특성을 결정하기 어려움
- 선택한 특성 수에 따라 벡터의 차원이 결정됨

- 빈도 기반 분석의 문제점
 - 단어 표현에 큰 벡터가 필요 (One-Hot encoding)
 - 언어의 의미를 담지 못한 수치화
- 언어(단어)의 의미를 수치에 담을 수 있을까?
 - 문장은 의미를 갖는다.
 - 같은 문장 내의 단어는 하나의 의미를 전달한다.
 - 문장 내의 단어를 가까운 위치에 존재하게 만들자.

→ Word2Vec



텍스트의 수치화 : 벡터화 - 2) 딥러닝 방식

중심 단어 주변 단어

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

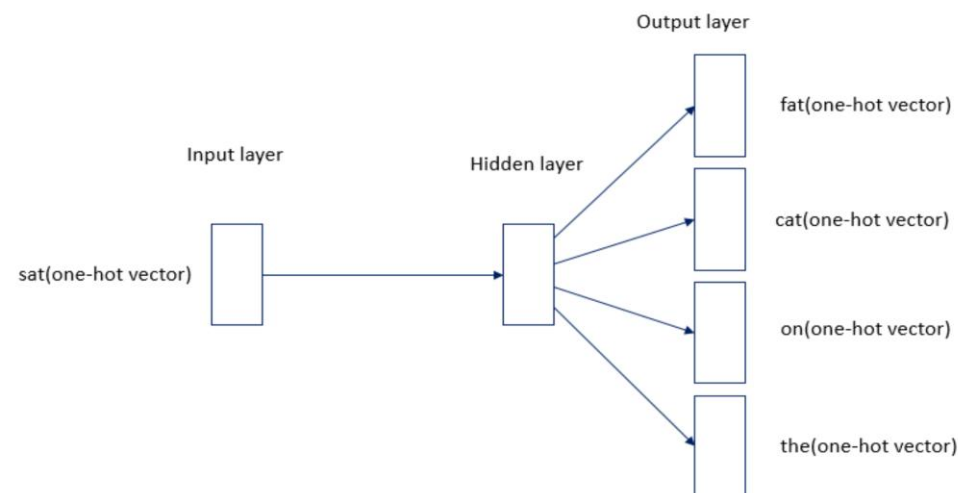
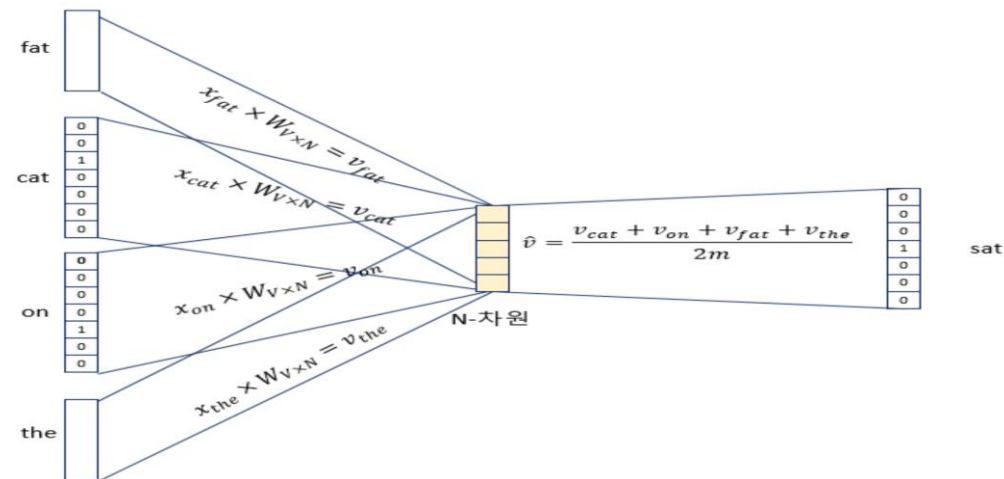
The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

중심 단어	주변 단어
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

(그림3) 출처 : <https://wikidocs.net/22660>



```
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
```

```
# 단어와 2차원 x축의 값, y축의 값을 입력받아 2차원 그래프를 그린다
def plot_2d_graph(vocabs, xs, ys):
    plt.figure(figsize=(8,6))
    plt.scatter(xs, ys, marker='o')
    for i, v in enumerate(vocabs):
        plt.annotate(v, xy=(xs[i], ys[i]))
```

```
sentences = [
    ['this', 'is', 'a', 'good', 'product'],
    ['it', 'is', 'a', 'excellent', 'product'],
    ['it', 'is', 'a', 'bad', 'product'],
    ['that', 'is', 'the', 'worst', 'product']
]
```

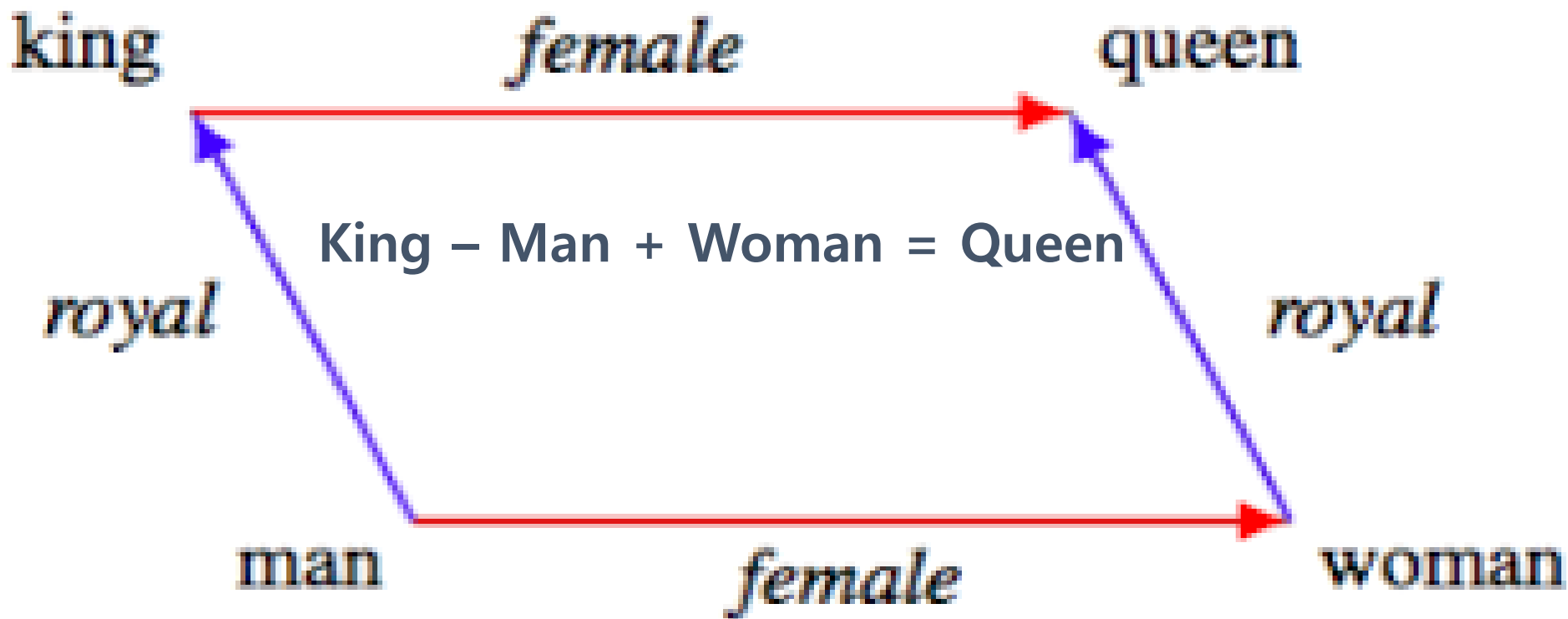
```
# 문장을 이용하여 단어와 벡터를 생성한다.
model = Word2Vec(sentences, size=300, window=3, min_count=1, workers=1)
```

```
# 단어벡터를 구한다.
word_vectors = model.wv

vocabs = word_vectors.vocab.keys()
word_vectors_list = [word_vectors[v] for v in vocabs]
```

```
# 단어간 유사도를 확인하다
print(word_vectors.similarity(w1='it', w2='this'))
```

-0.022223482



훈련에 사용된 문장에서 잠재 공간 상의 거리를 배운다
→ 잠재 공간에서는 단어간 계산이 가능하다

```
model.most_similar(positive=['king','woman'], negative=['man'],topn=10)
Out[12]:
[('queen', 0.7118192911148071),
 ('monarch', 0.6189674139022827),
 ('princess', 0.5902431607246399),
 ('crown_prince', 0.5499460697174072),
 ('prince', 0.5377321243286133),
 ('kings', 0.5236844420433044),
 ('Queen_Consort', 0.5235945582389832),
 ('queens', 0.5181134343147278),
 ('sultan', 0.5098593235015869),
 ('monarchy', 0.5087411999702454)]
```

훈련된 문장에서 거리를 배운다
→ 영역 별 거리도 만든다

Word2Vec 임베딩 테스트



```
test_words = ["espresso"]
out_words = model.most_similar(positive=test_words)
print( test_words[0], " most similar ===== ")
print_list(out_words)
```

```
espresso most similar =====
0 == ('cappuccino', 0.6888187527656555)
1 == ('mocha', 0.6686209440231323)
2 == ('coffee', 0.6616825461387634)
3 == ('latte', 0.6536751985549927)
4 == ('espressos', 0.6438628435134888)
5 == ('macchiato', 0.6428249478340149)
6 == ('brewed_coffee', 0.6234086751937866)
7 == ('iced_coffee', 0.6213865280151367)
8 == ('expresso', 0.6108267307281494)
9 == ('coffees', 0.6038862466812134)
```

```
test_words = ["samsung"]
out_words = model.most_similar(positive=test_words)
print( test_words[0], " most similar ===== ")
print_list(out_words)
```

```
samsung most similar =====
0 == ('htc', 0.7326198816299438)
1 == ('nokia', 0.7076619267463684)
2 == ('sony', 0.6818498373031616)
3 == ('motorola', 0.6689046025276184)
4 == ('ipad', 0.6625324487686157)
5 == ('iphone', 0.6582239866256714)
6 == ('macbook', 0.6297854781150818)
7 == ('verizon', 0.6176369786262512)
8 == ('ipod_touch', 0.6170758008956909)
9 == ('ps3', 0.607496440410614)
```

```
# Public Opinion
country = 'Japan'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
Japan pos/neg == 1.1084418
```

```
# Public Opinion
country = 'Korea'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
Korea pos/neg == 0.90629506
```

```
# Public Opinion
country = 'USA'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
USA pos/neg == 34.609974
```

```
model = Word2Vec(sentences, size=10, window=5, min_count=1, workers=1, iter=1000)

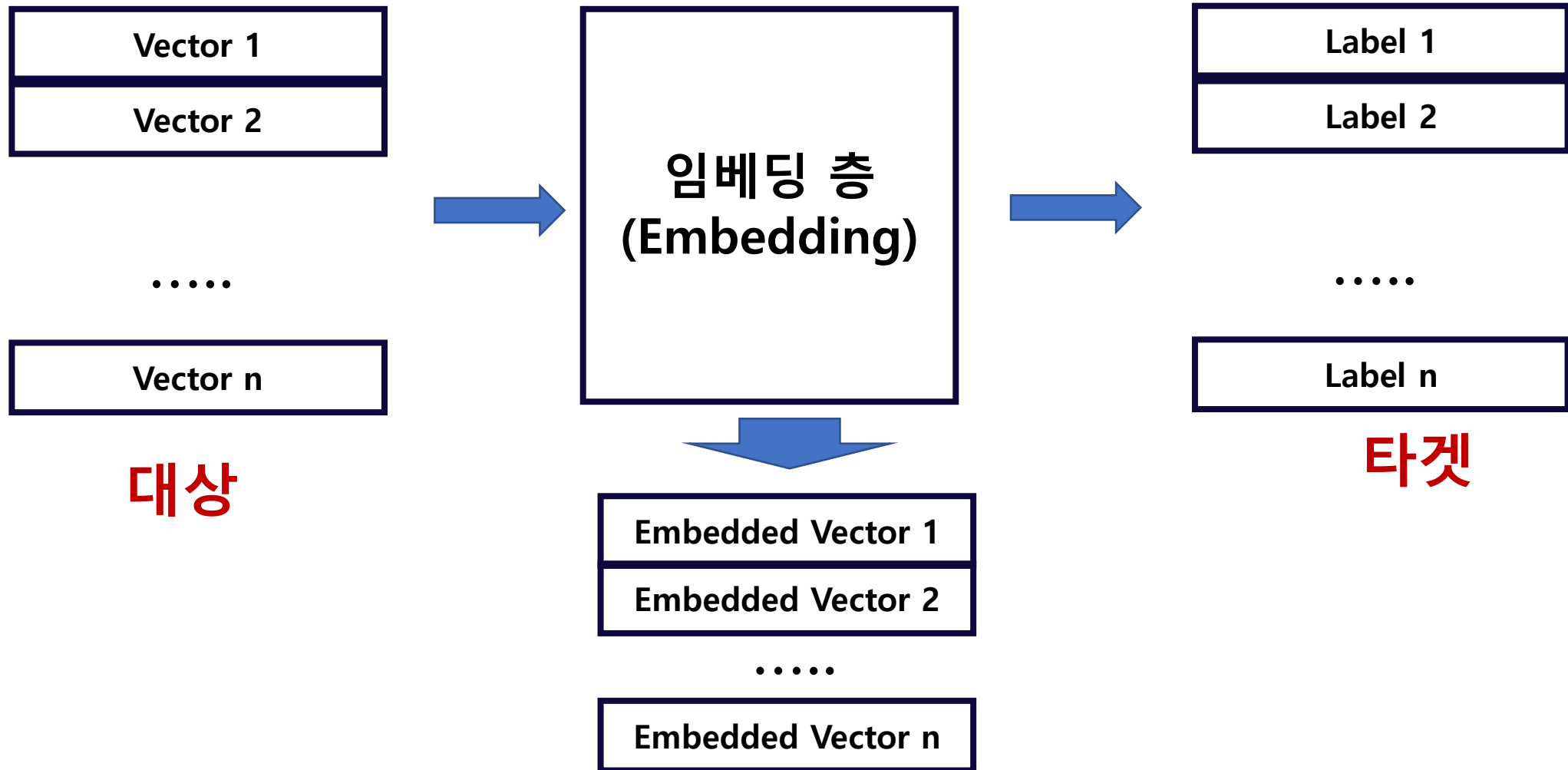
word_vectors = model.wv

vocabs = word_vectors.vocab.keys()
print("vocabs == ", vocabs)

word_vectors_list = [ word_vectors[v] for v in vocabs ]
# print("word_vectors_list == ", word_vectors_list)
```

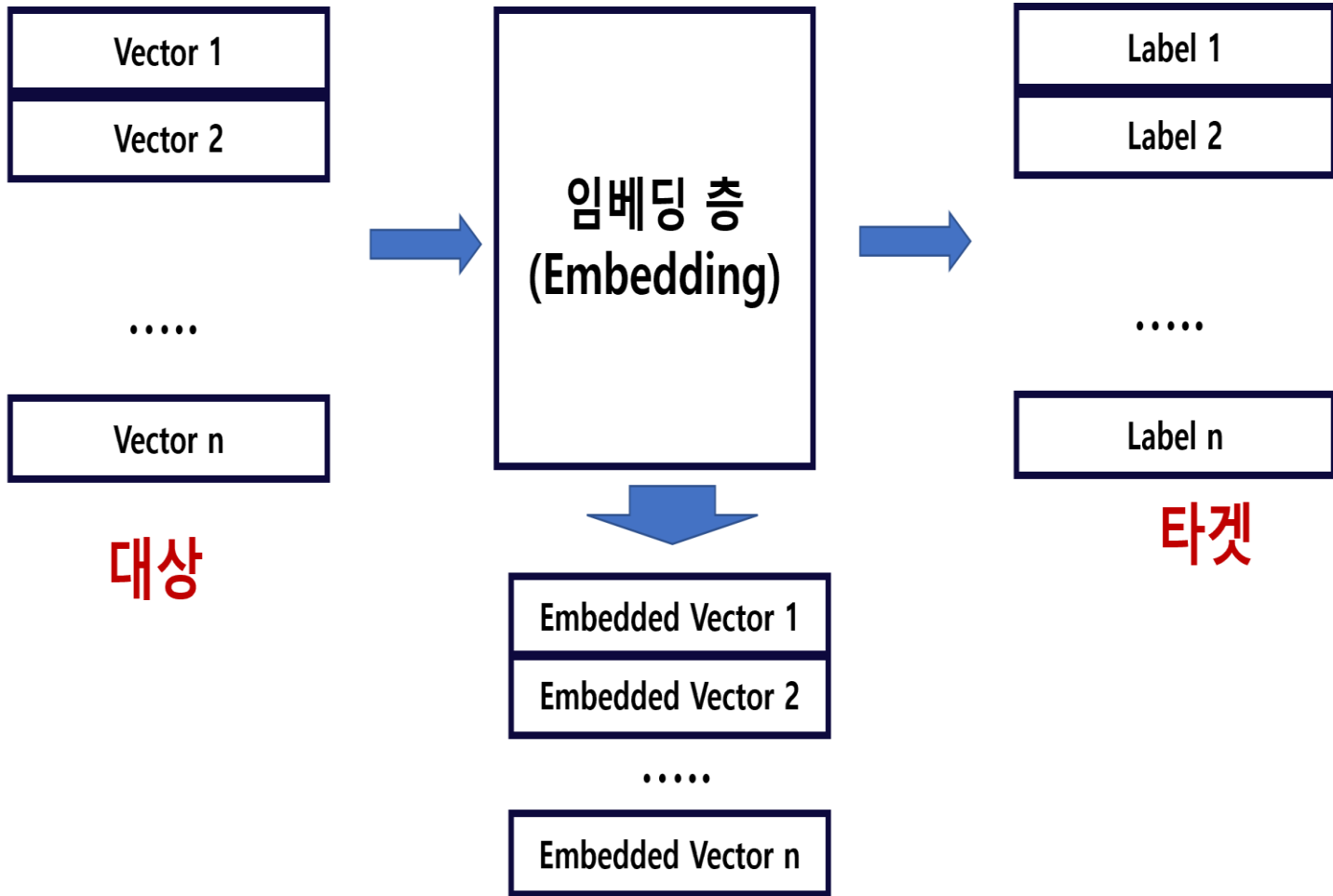
```
1 sentences = [
2     ['this', 'is', 'a', 'good', 'product'],
3     ['it', 'is', 'an', 'excellent', 'product'],
4     ['it', 'is', 'a', 'bad', 'product'],
5     ['that', 'is', 'the', 'worst', 'product'],
6     ['that', 'is', 'the', 'good', 'product'],
7     ['this', 'is', 'the', 'best', 'product'],
8     ['this', 'is', 'a', 'bad', 'product'],
9 ]
```

수치적 표현 → 훈련 방향에 따라 만들어진다.



주어진 타겟에 맞춰 표현을 정렬한다.

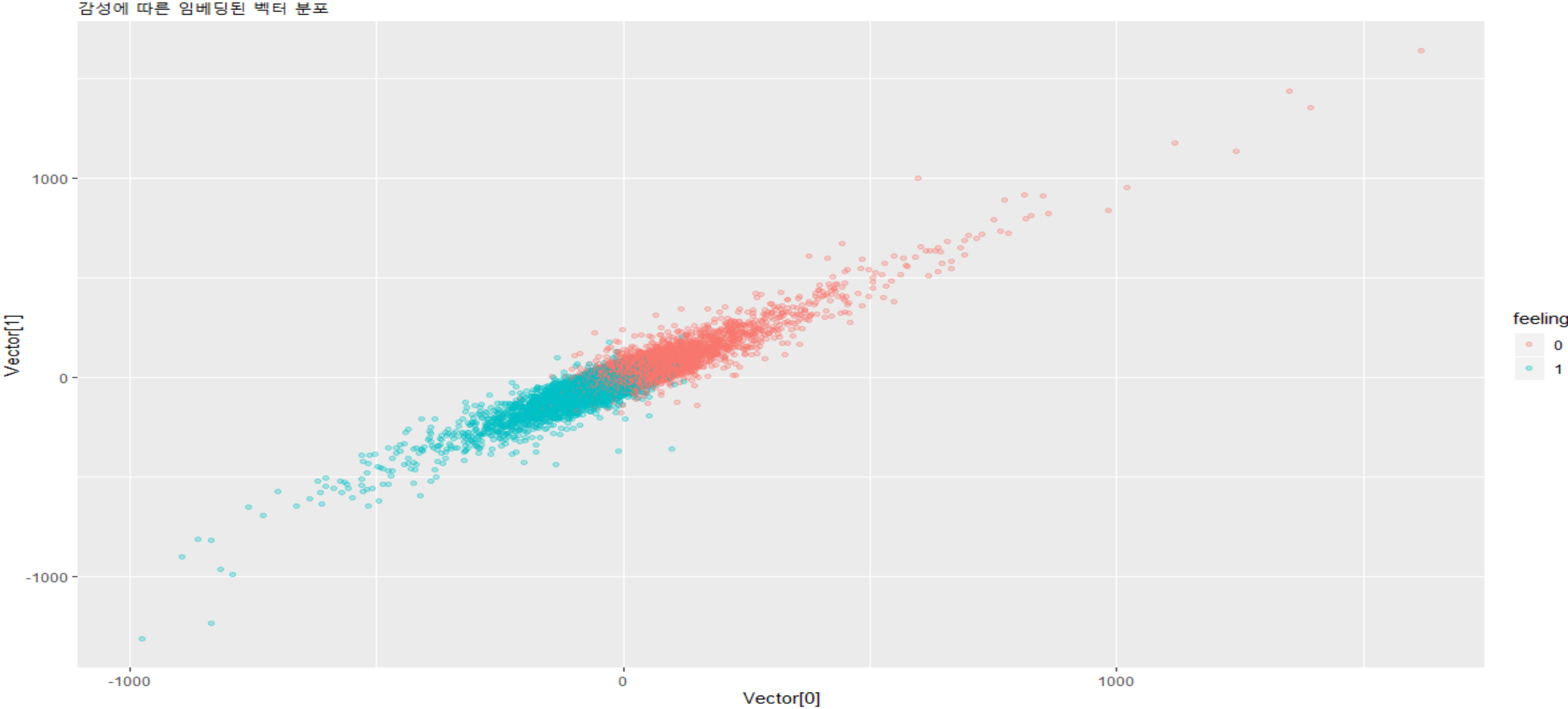
훈련에 의한 임베딩(Embedding)



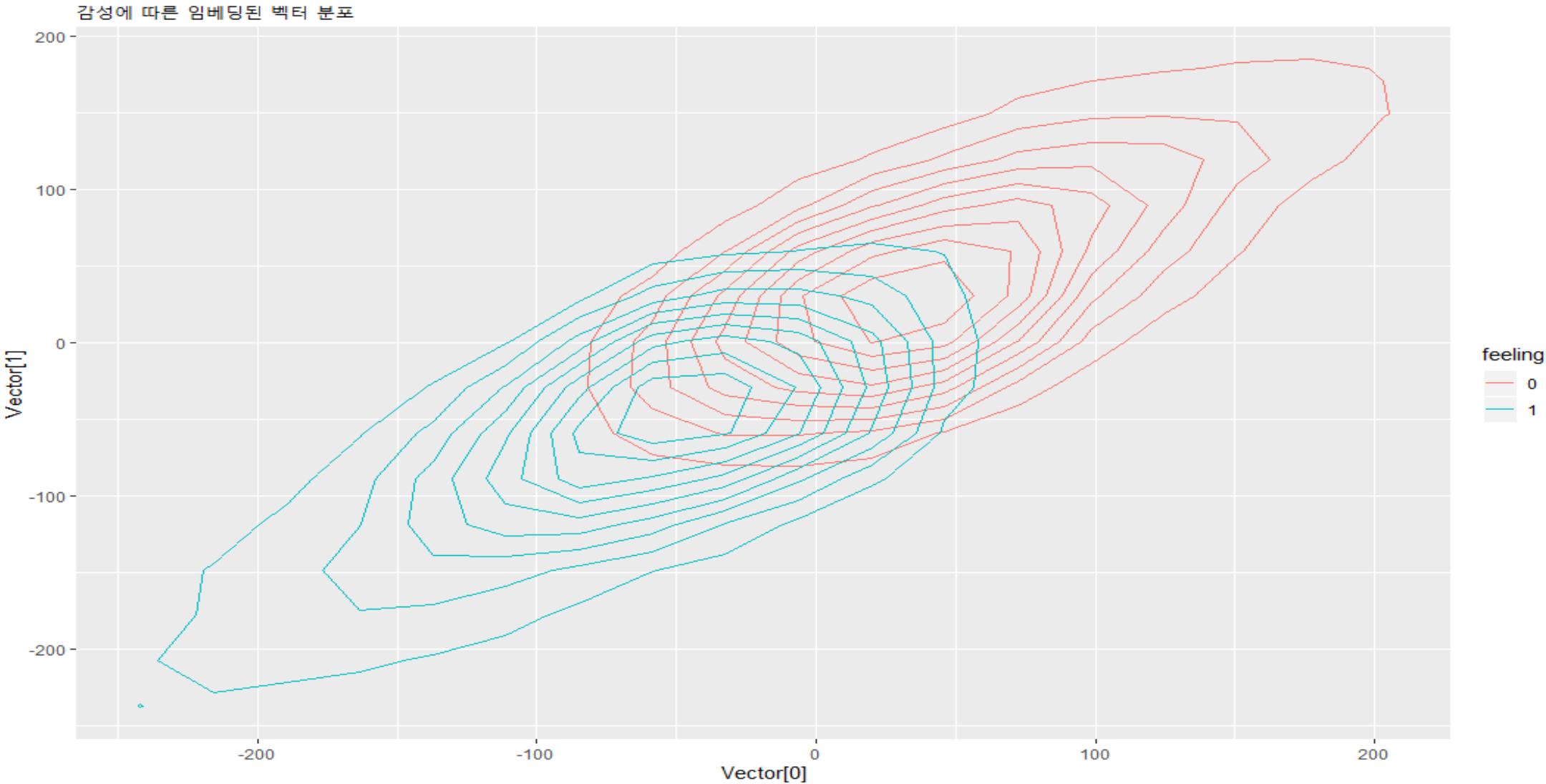
주어진 타겟에 맞춰 표현을 정렬한다.

- 훈련 목표에 따라 달라지는 수치화
 - ✓ 6개의 문장
 - ① 아주 좋아
 - ② 정말 좋아
 - ③ 좋아
 - ④ 나빠
 - ⑤ 정말 나빠
 - ⑥ 아주 나빠
 - ✓ 훈련 목표
 - 1) 긍정 vs 부정
 - 2) 긍정 점수 : 0 ~ 5
 - ✓ 훈련 목표에 따른 단어의 중요도는?
 - '정말', '아주'의 중요도는?

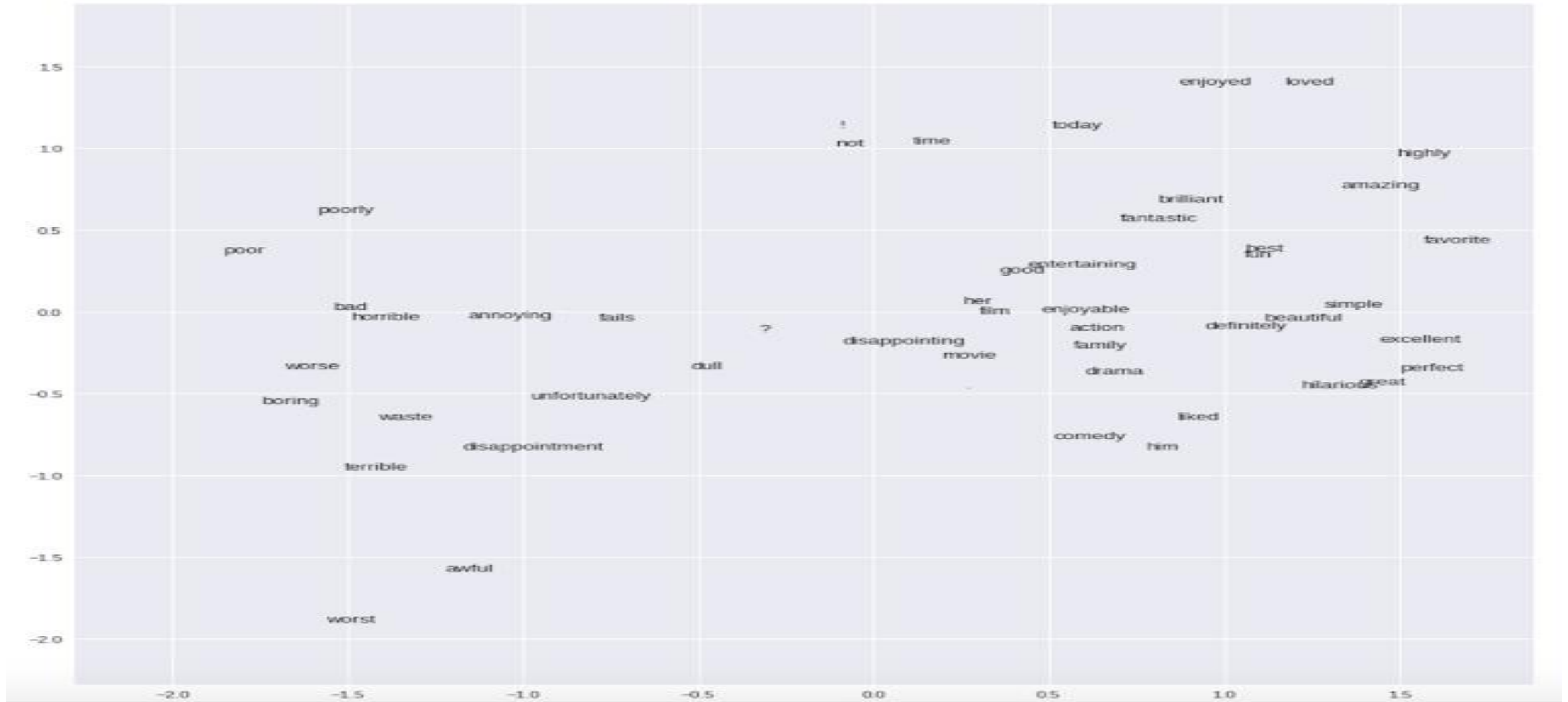
감성에 따른 임베딩 벡터 2차원 분포



감성에 따른 임베딩 벡터 2차원 분포



감성에 따른 임베딩 벡터 2차원 분포



출처 <https://www.juliabloggers.com/intro-to-sparse-data-and-embeddings/>

실습 : <https://colab.research.google.com/drive/1X8dDX8FuLgzzMAIrJuzJhXy6x6ligK7Z?usp=sharing>

- 축소된 차원 → 계산이 빠르다
- 단어의 감성 값 자동 수치화
 - ✓ '서울'의 느낌은?
 - ✓ 시대별 '강남'의 이미지는?
 - ✓ 지도도 변화
 - 수동 vs 자동
 - 여론 조사 자동화 가능할까??

I • SEÒUL • U

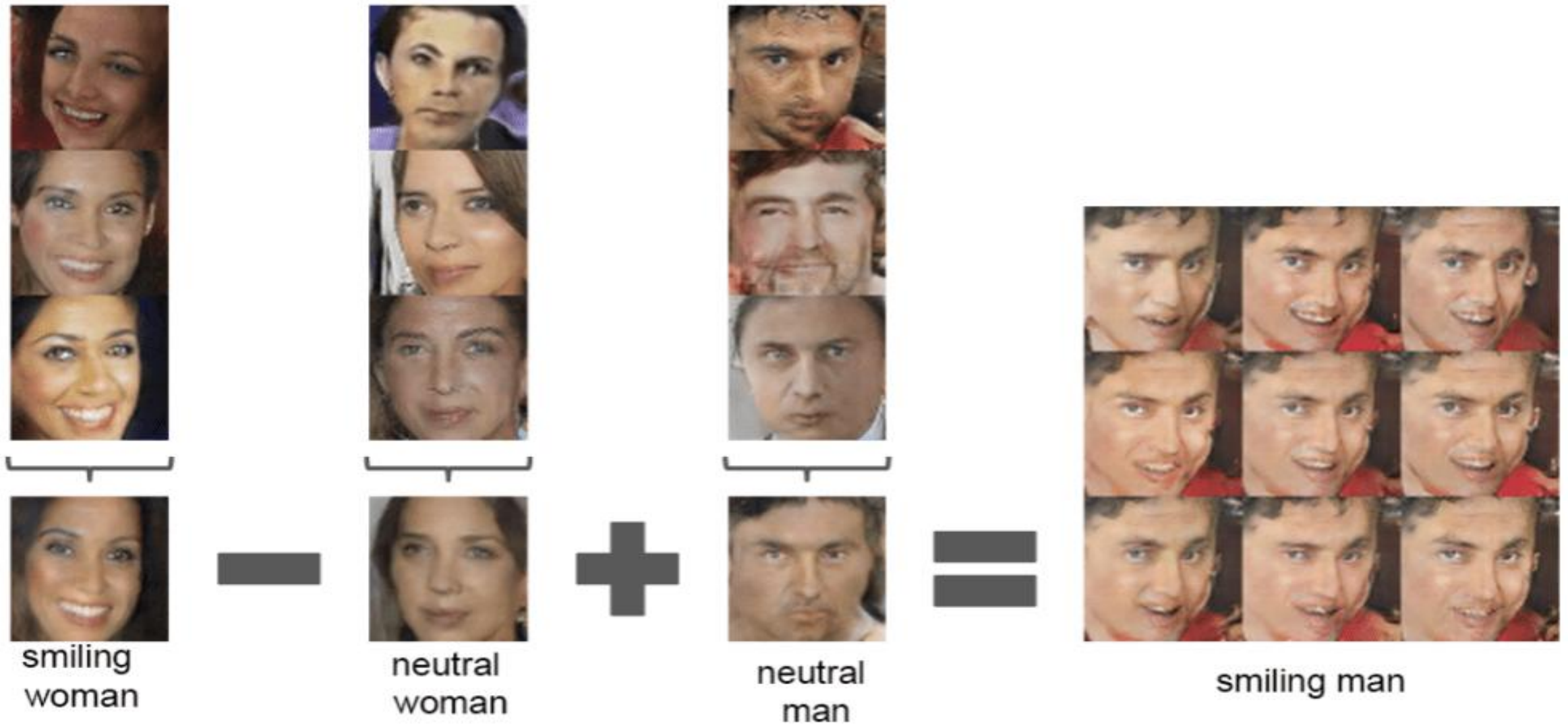


YAHOO!
VS
Google

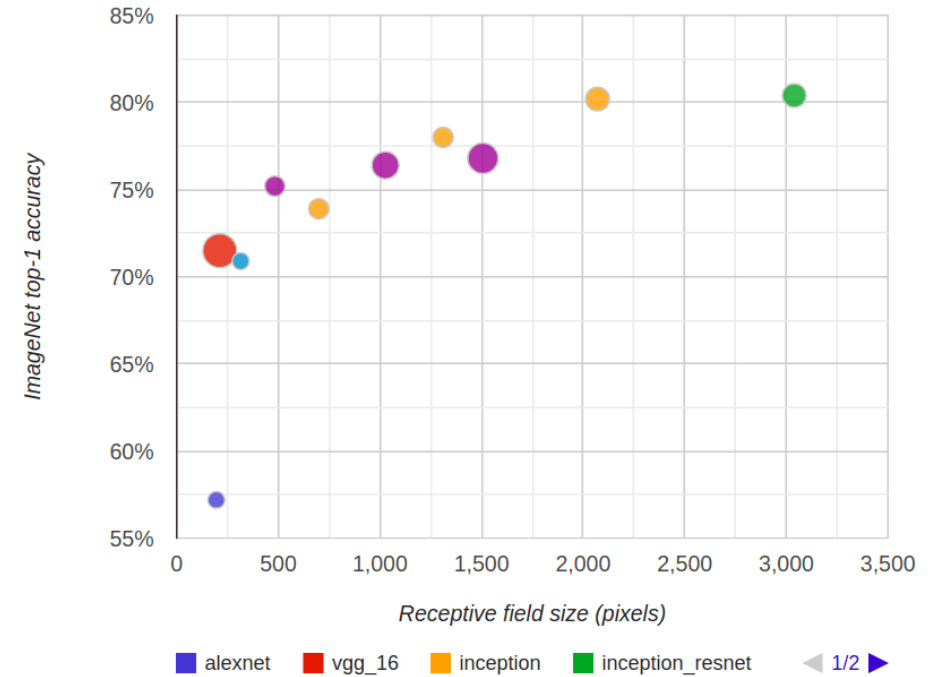
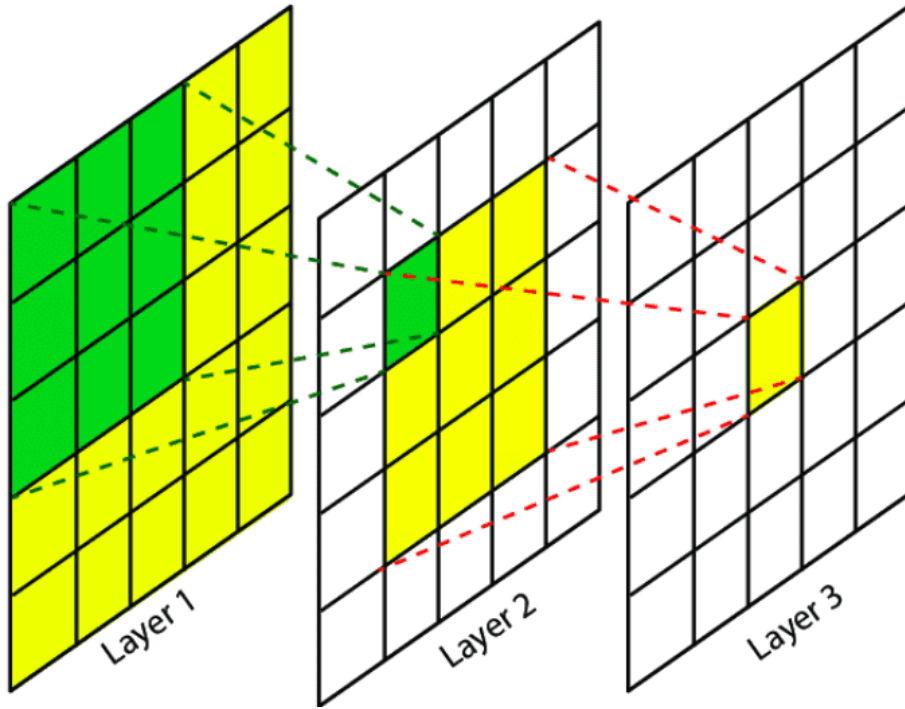
그림의 잠재공간

이미지(사진)를 수치화 할 수 있을까?

이미지(사진)이 수치화(벡터화)가 되면

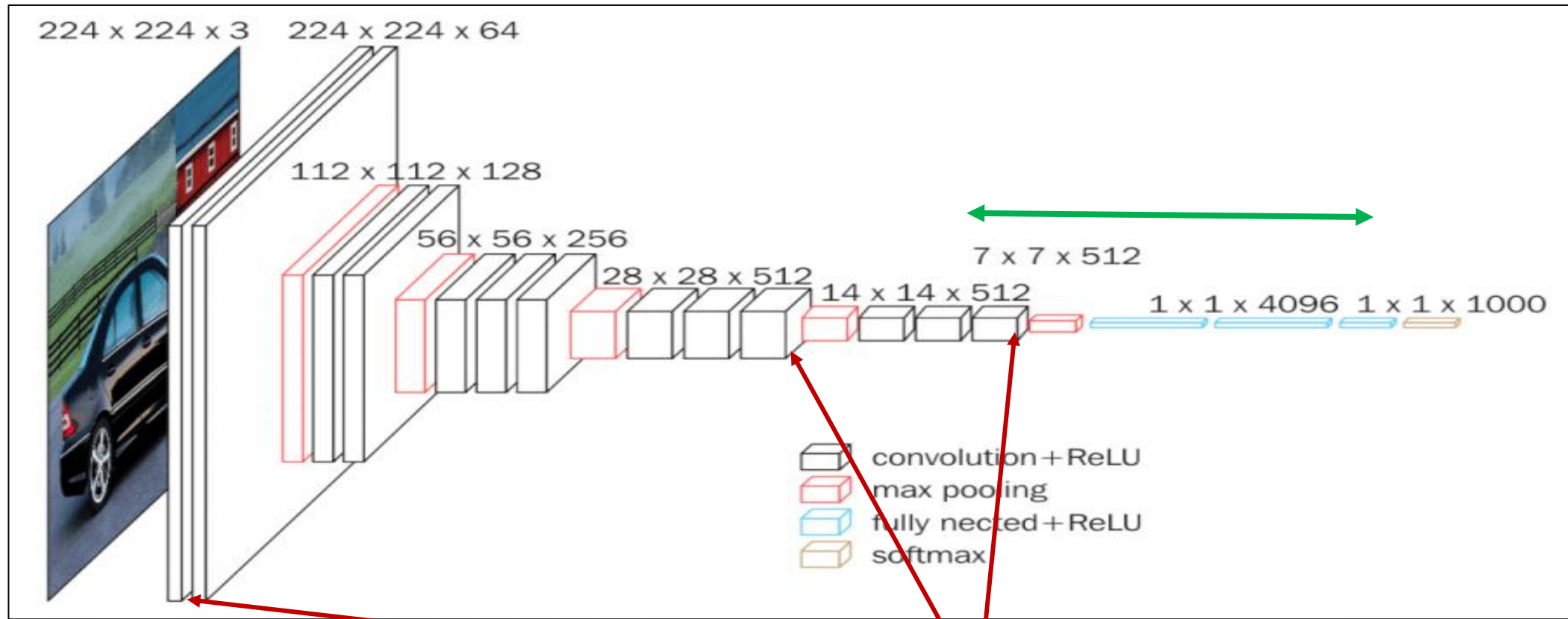


수용 영역 (Receptive Field)



- 하나의 뉴론이 접근할 수 있는 정보
- 수용 영역의 크기에 따라 취득하는 특징에 차이 (Local feature vs Global feature)
- CNN에서는 분류 정확도와 수용 영역의 크기는 로그 관계

CNN 연결의 특징 : 수용 영역 → 각 레이어에서 보는 것은?



이미지의 스타일(style)

이미지의 내용(content)

각 층의 뉴런이 수용하는 패턴은?

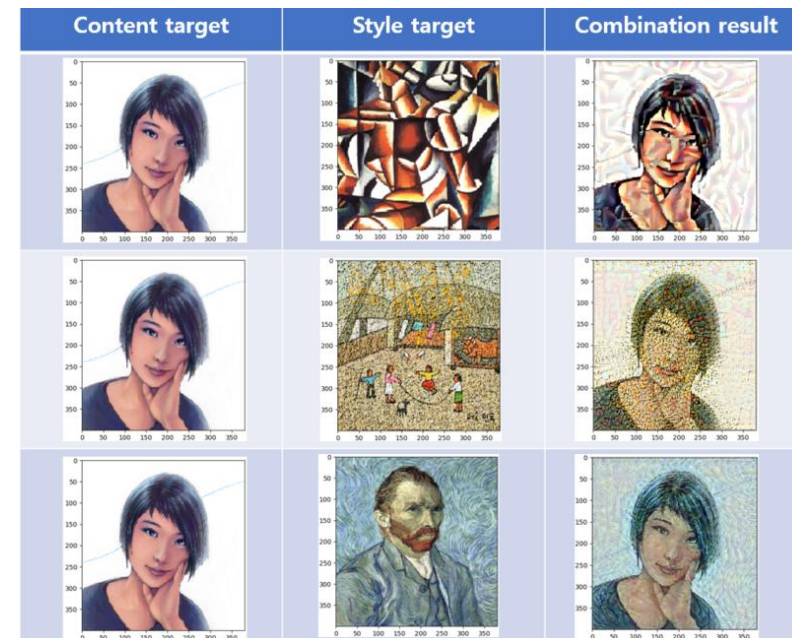
- 나무를 보는가? 숲을 보는가?
- 판별(Classification)을 위한 정렬

수용 영역의 차이에 활용 : 스타일 트랜스퍼

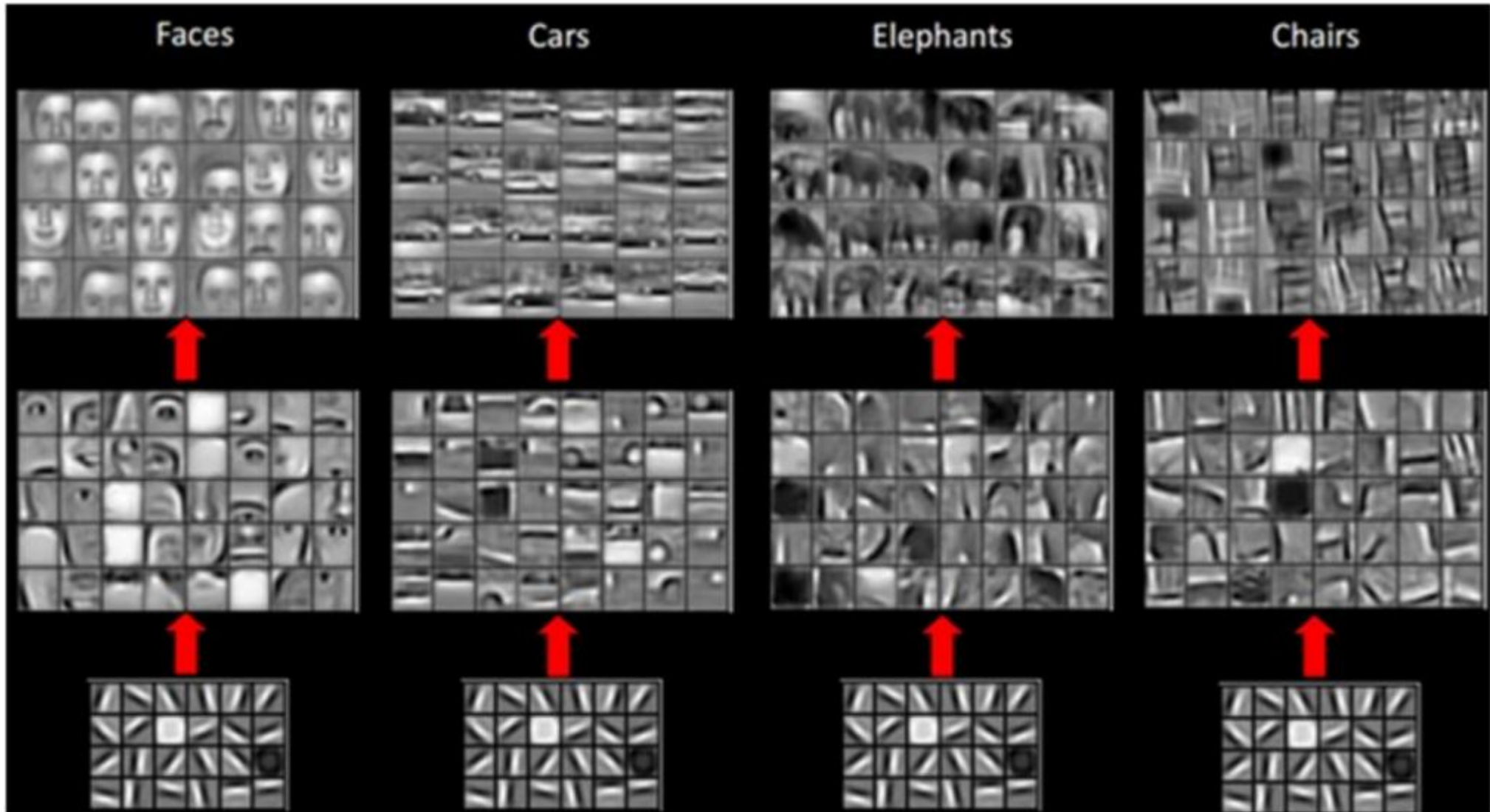
- 2015년 여름 리온 게티스(Leon Gatys)
- 타겟 이미지 콘텐츠를 보존하면서 참조 이미지 스타일 적용
- 목표를 손실함수로 정의하고 이를 최소화 → weigh가 아닌 이미지 변환
 - $\text{Loss} = \text{distance}(\text{style}(\text{참조이미지}) - \text{style}(\text{생성이미지})) +$
 - $\text{distance}(\text{content}(\text{원래이미지}) - \text{content}(\text{생성이미지}))$
- 하위층 : 스타일(style), 상위층 : 콘텐츠(contents)
 - 1개의 상위층과 다수의 하위층 사용



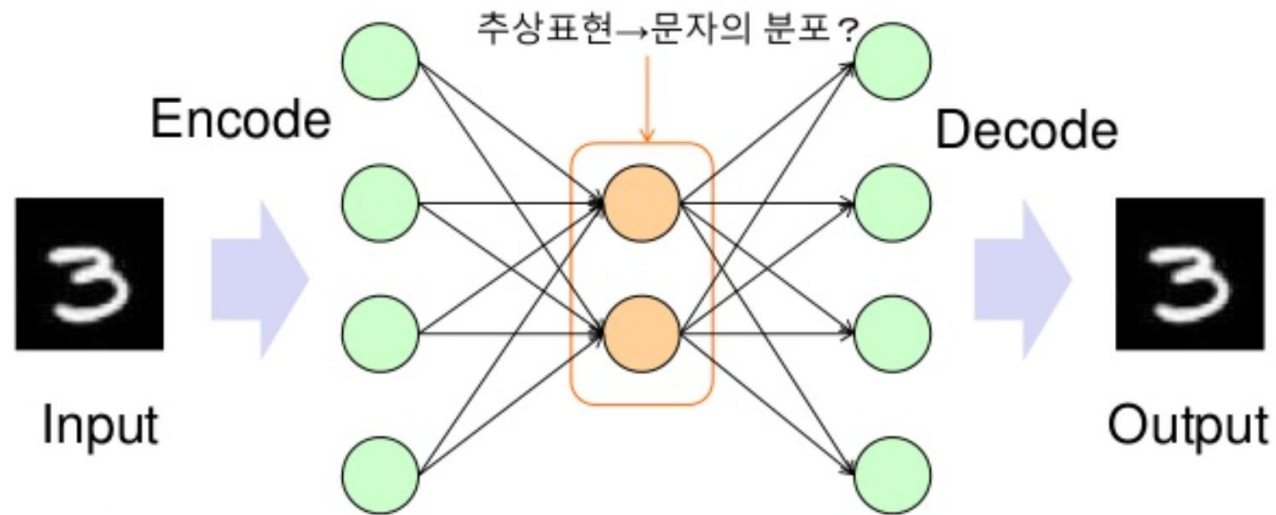
이미지 출처 <https://m.blog.naver.com/jyoun/221608828909>



수용 영역 (Receptive Field)



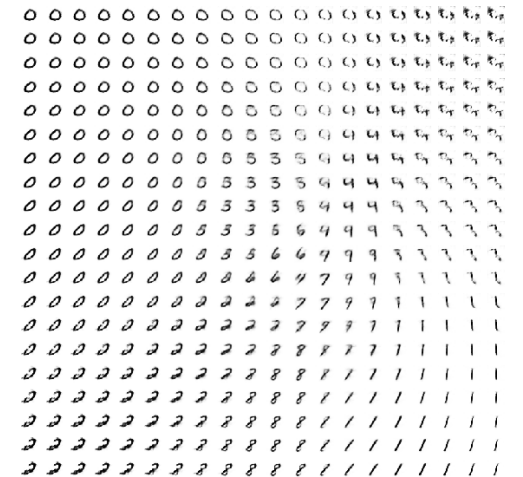
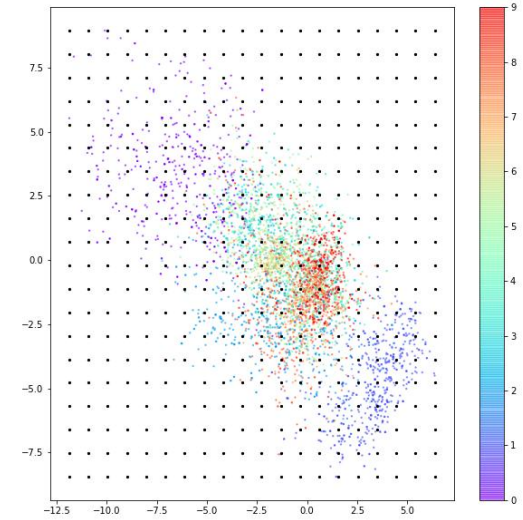
Output을 Input과 동일하게 하여 Hidden Layer를 만든다
→ 차원 축소에 사용할 수 있다



- AutoEncoder에 의한 이미지의 압축·재구성
- 중간층에서 이미지의 추상표현이 획득

Auto-Encoder에 의한 잠재공간 생성

- 잠재 공간에서의 위치
 - 같은 모양의 숫자(그림)은 비슷한 위치
 - 위치의 변화에 따라 점차적인 그림 변화
- 숫자에 따라 분포 영역 크기가 차이가 큼
 - 분포의 모양이 균일하지 않음
 - 잠재 공간에서 점 사이가 채워지지 않음



Variational AutoEncoder (VAE)

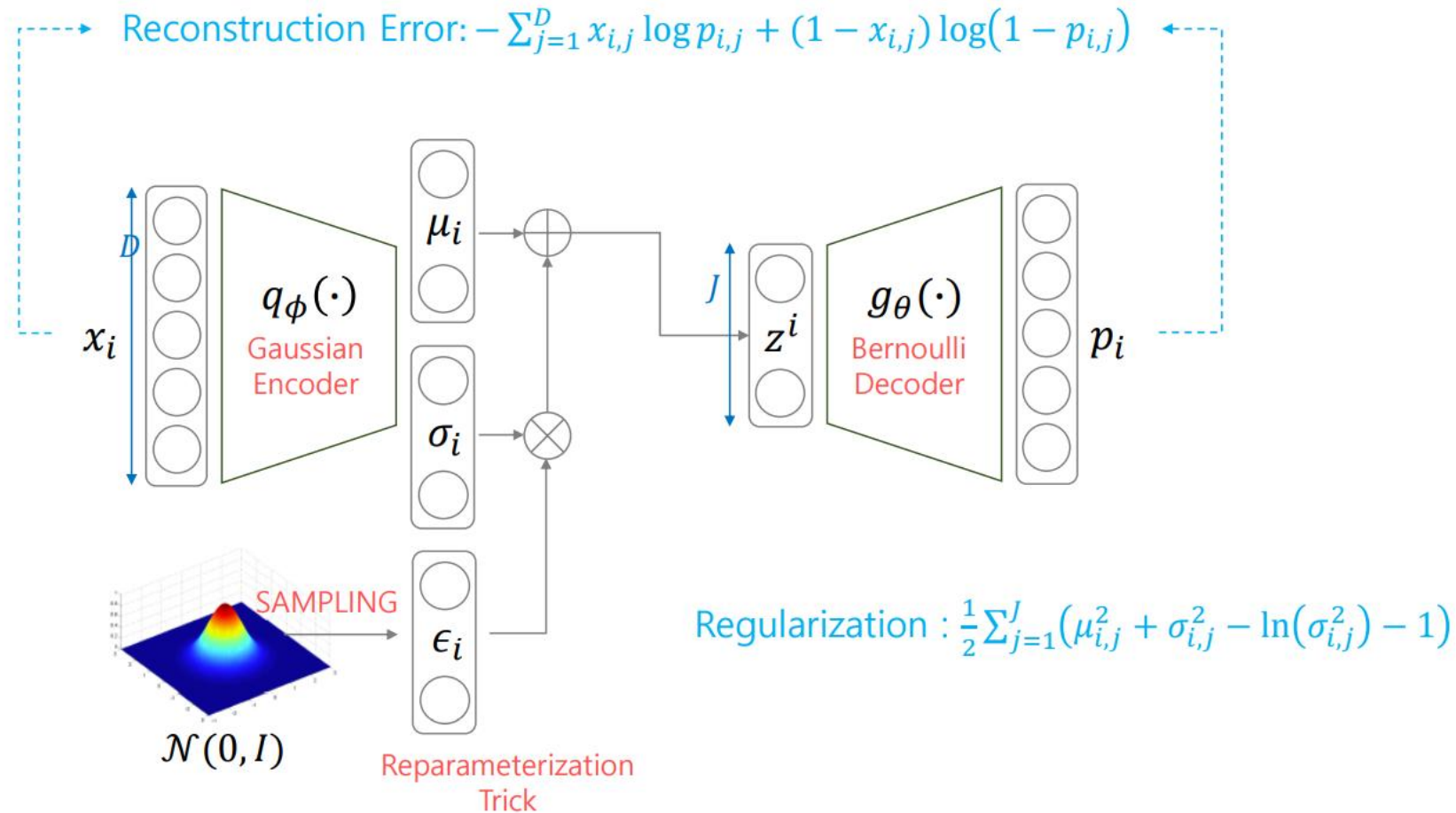


STRUCTURE

Default : Gaussian Encoder + Bernoulli Decoder

VAE

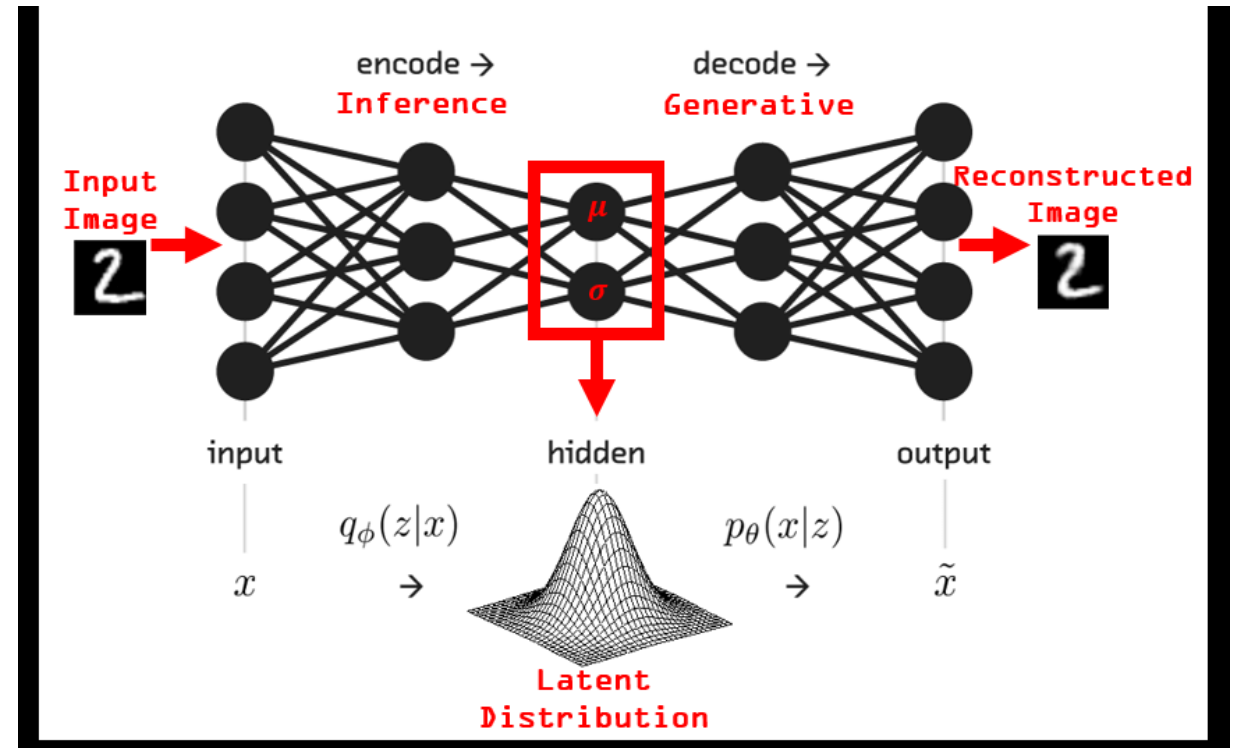
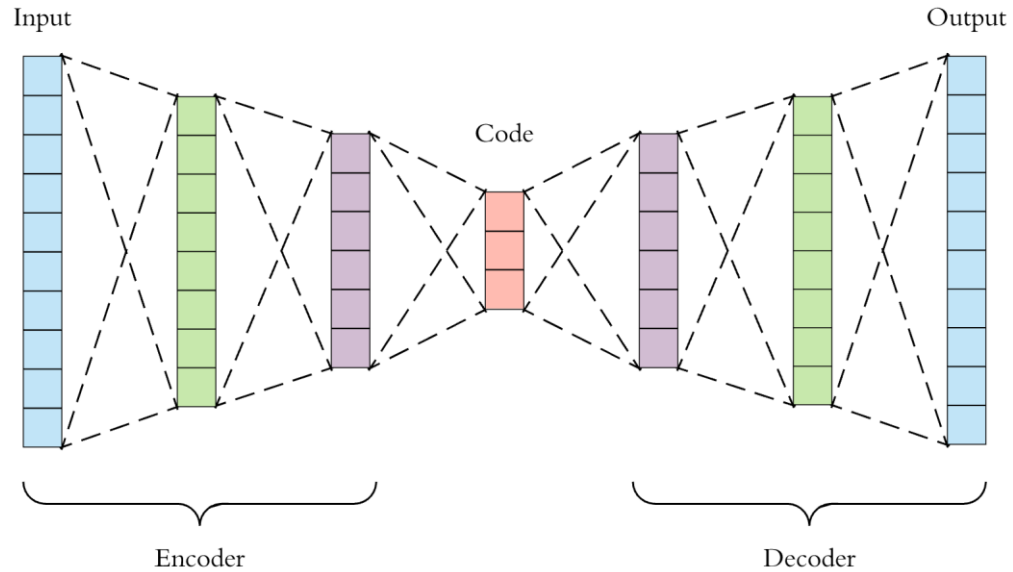
18 / 49



[참고] AutoEncoder vs Variational AutoEncoder



Point vs Distribution



Variational AutoEncoder (VAE)



RESULT

MNIST

VAE

22 / 49

Reproduce



Input image



$J = |z| = 2$



$J = |z| = 5$



$J = |z| = 20$

Variational AutoEncoder (VAE)



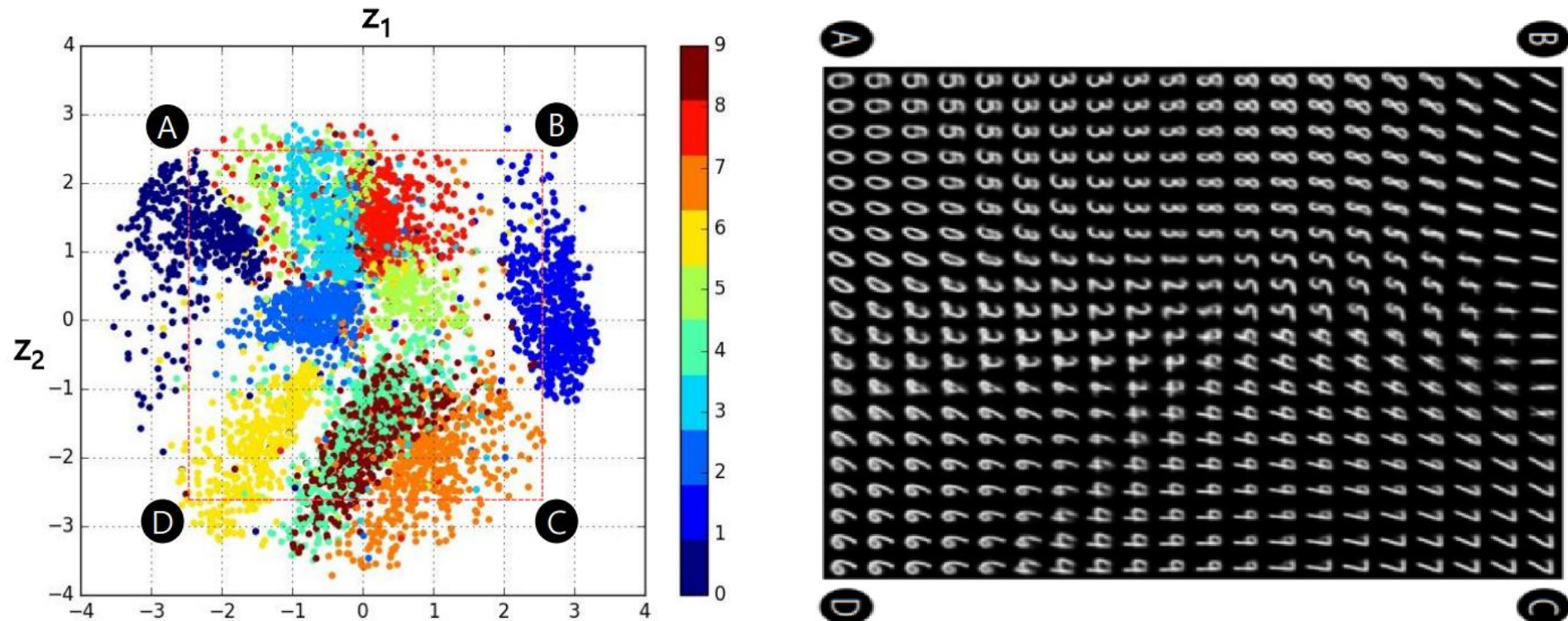
RESULT

MNIST

VAE

25 / 49

Learned Manifold

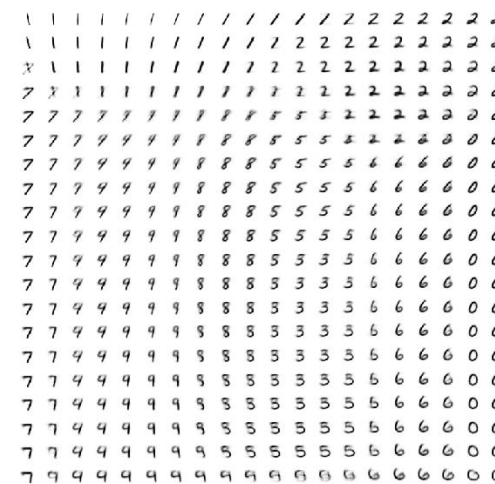
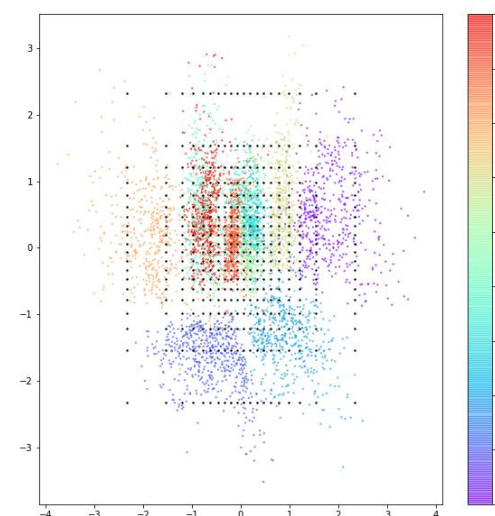


학습이 잘 되었을 수록 2D공간에서 같은 숫자들을 생성하는 z들은 뭉쳐있고,
다른 숫자들은 생성하는 z들은 떨어져 있어야 한다.

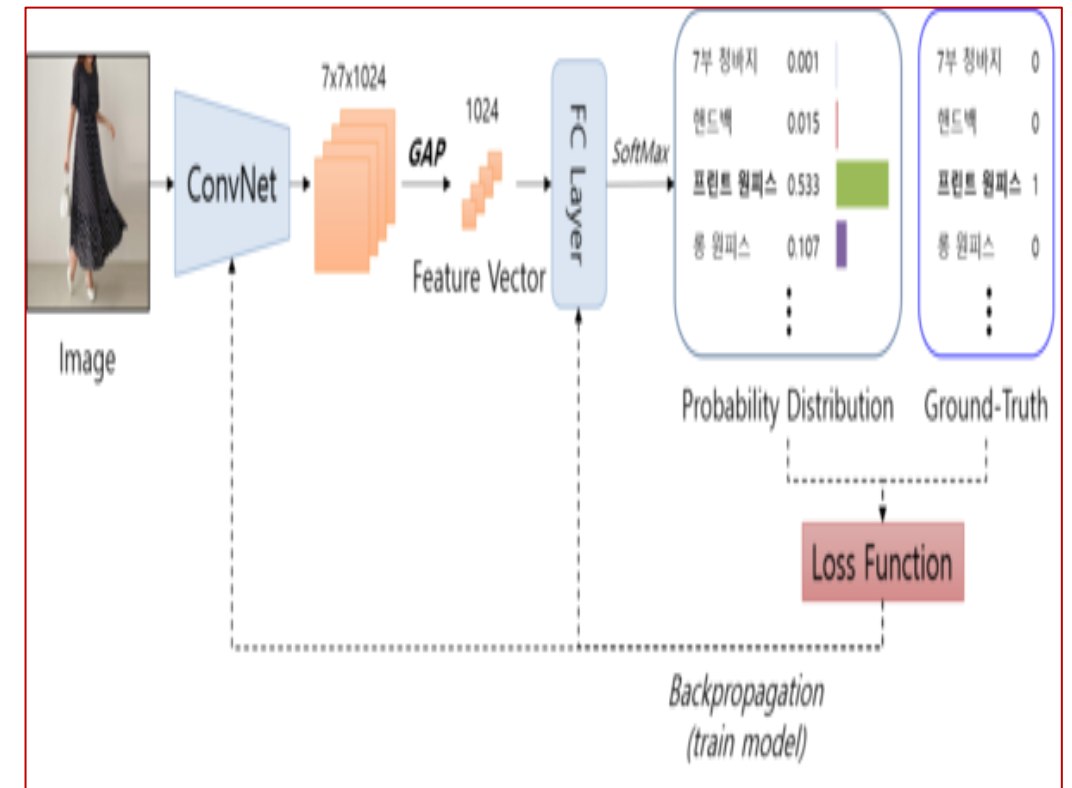
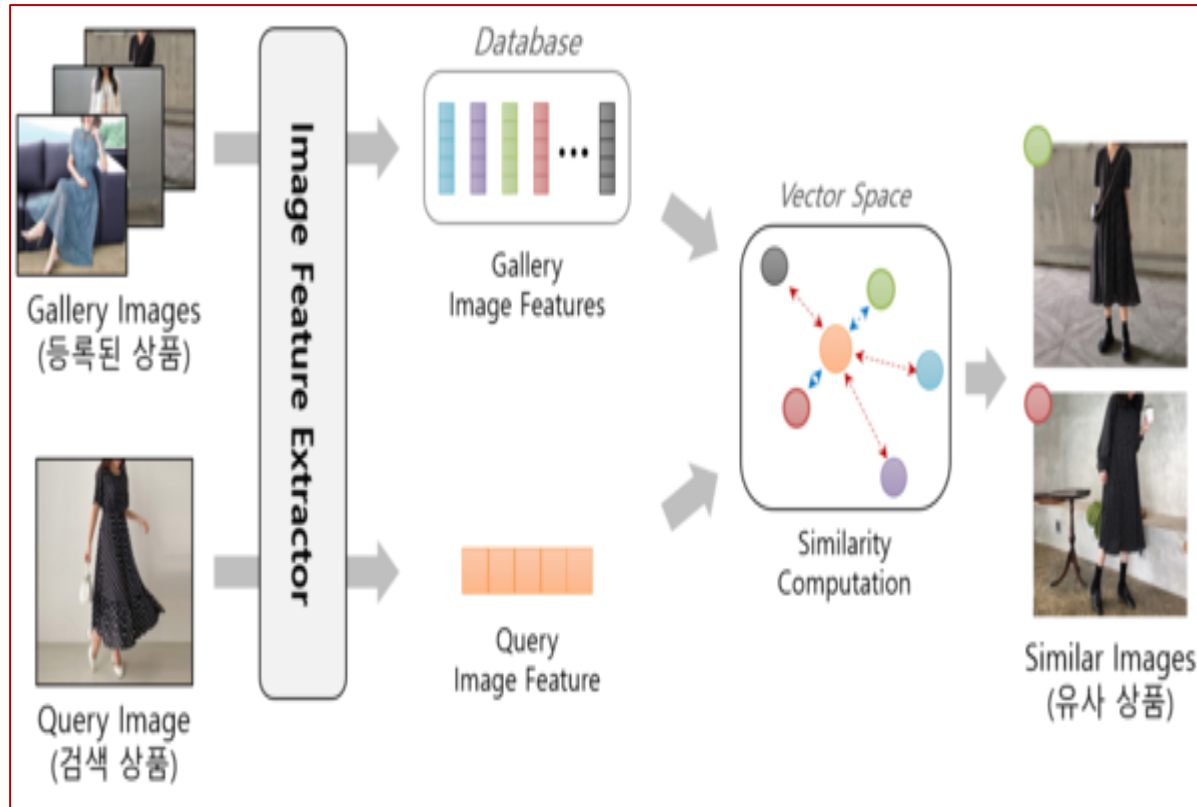
<https://github.com/hwalsuklee/tensorflow-mnist-VAE>

VAE 실습 : <https://colab.research.google.com/drive/1N1jkCiV1A1yDmFIMyKMRQkYBr--x80FF?usp=sharing>

잠재 공간에서의 위치는 절대적인가?



이미지 잠재공간의 활용



출처 : <https://www.comworld.co.kr/news/articleView.html?idxno=50015>

- 이미지의 잠재 공간 : 유사한 이미지가 가까이 존재

이미지 잠재 공간의 활용 - 짹통 탐지



이름·디자인·맛... 다 똑같은 中 '짹통 불닭볶음면' 이젠 세계 라면
시장까지 공격 중!
입력 2025.04.02. 15:08

韓 '불닭볶음면', 작년 해외 매출액 1조3000억 원 넘어
中 기업 9곳 이상이 짹통 '불닭' 만들어... 美·아프리카·동남아로 수출



삼양식품에서 만든 핵불닭볶음면./삼양



해외에서 판매 중인 중국산 짹통 핵불닭볶음면./seokyoungduk 인스타그램

짹통 불닭볶음면은 우리나라 불닭볶음면과 구별이 안 갈 정도로 비슷해요. 흰 닭이 불닭
볶음면을 먹고 매운 듯 입에서 불이 나오는 디자인마저 일치합니다. 이렇게 다른 제품을
마음대로 똑같이 베끼는 것은 '지식재산권(知識財産權·Intellectual Property Right)
'을 침해하는 일이에요. 지식재산권은 사람이나 기업이 만들어낸 상품의 상표나 디자인
을 다른 사람이나 기업이 사용하지 못하도록 할 수 있는 권리랍니다.



북한에서도 '짹통 불닭볶음면'을 만들고 있어요!

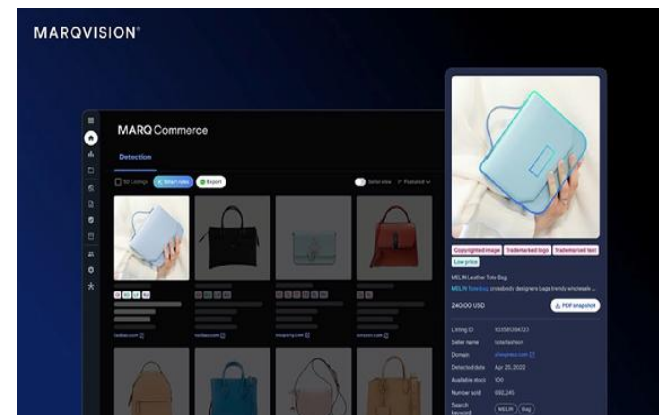
중국뿐 아니라 북한에서도 우리나라 불닭볶음면을 흉내 낸 제품을 만들고 있어요. 북한의
'경흥'이라는 회사는 '매운 닭고기맛 볶음국수'라는 제품을 파는데요. 검은 바탕에 닭 캐릭터
터가 불을 뿜고 있는 디자인이 우리나라 불닭볶음면과 매우 비슷해요. 불닭볶음면이 북한
까지 스며든 건 그만큼 사람들이 한국 라면을 몰래 사 먹기 때문입니다. 이렇게 세계적으로
우리나라 제품을 본뜬 제품을 만드는 곳이 늘어나자, 2021년에 삼양을 비롯한 우리나라 식
품 회사 4곳이 중국 회사를 고소하기도 했어요. 이에 배상금을 약 3700만 원 지불하라는
판결이 나왔죠. 법적으로 피해를 인정받긴 했지만, 사실상 큰 효과가 없는 겁니다. 지금은
우리나라 라면이 세계적으로 인기지만, 다른 나라에서 경쟁력을 갖추면 언제든 뒤집힐 수
도 있는 거예요.

이미지 잠재 공간의 활용 - 짝퉁 탐지



현대 디지털 경제에서는 지적재산(IP) 보호가 점점 더 중요한 과제가 되고 있다. 이커머스로 대표되는 온라인 상거래에서는 수많은 브랜드들이 경쟁을 치르고 있으며, 소위 '명품' 반열에 오른 브랜드들은 '짝퉁'이라는 시장추종자들의 추앙을 넘어서 '지식재산 도둑질'을 하루에도 수천건씩 당한다. 명품이 아니더라도, 조금만 알려진 브랜드와 콘텐츠들은 불법 복제와 무단 판매로 인해 막대한 경제적 피해를 입고 있다. 이런 문제를 해결하기 위해 등장한 마크비전(MarqVision)과 같은 AI기술을 활용한 글로벌 IP 보호 서비스가 등장하고 있다.

또한, 마크비전은 아시아 시장에서의 강점을 지니고 있다. 전 세계 불법 복제와 해적판의 90%가 아시아에서 발생하고 있음에도 불구하고, 아시아 시장을 집중적으로 다루는 글로벌 IP 보호 회사는 거의 없다. 마크비전은 이 부분에서 앞서고 있으며, 아시아 언어를 지원하는 자동 보고 시스템과 판매자 기반의 집행 전략을 통해 강력한 보호 솔루션을 제공한다. 마크비전의 자동화된 보고 시스템은 탐지된 불법 상품에 대한 상세한 문서화를 자동으로 생성하고, 이를 시장에 제출하는 과정을 자동화한다. 이 시스템은 또한 판매자에게 직접 알림을 보내어 신속한 조치를 취할 수 있도록 지원한다. 이러한 접근 방식은 법률 서비스 및 법률 회사와 협력하여 법적 전략을 수립하고, 수익 회복 프로그램을 통해 고객의 경제적 손실을 최소화하는 데 도움을 준다.



‘Furiosa: A Mad Max Saga’ Used AI to Mix Anya Taylor-Joy’s Face With Child Actor

🕒 MAY 29, 2024 👤 PESALA BANDARA

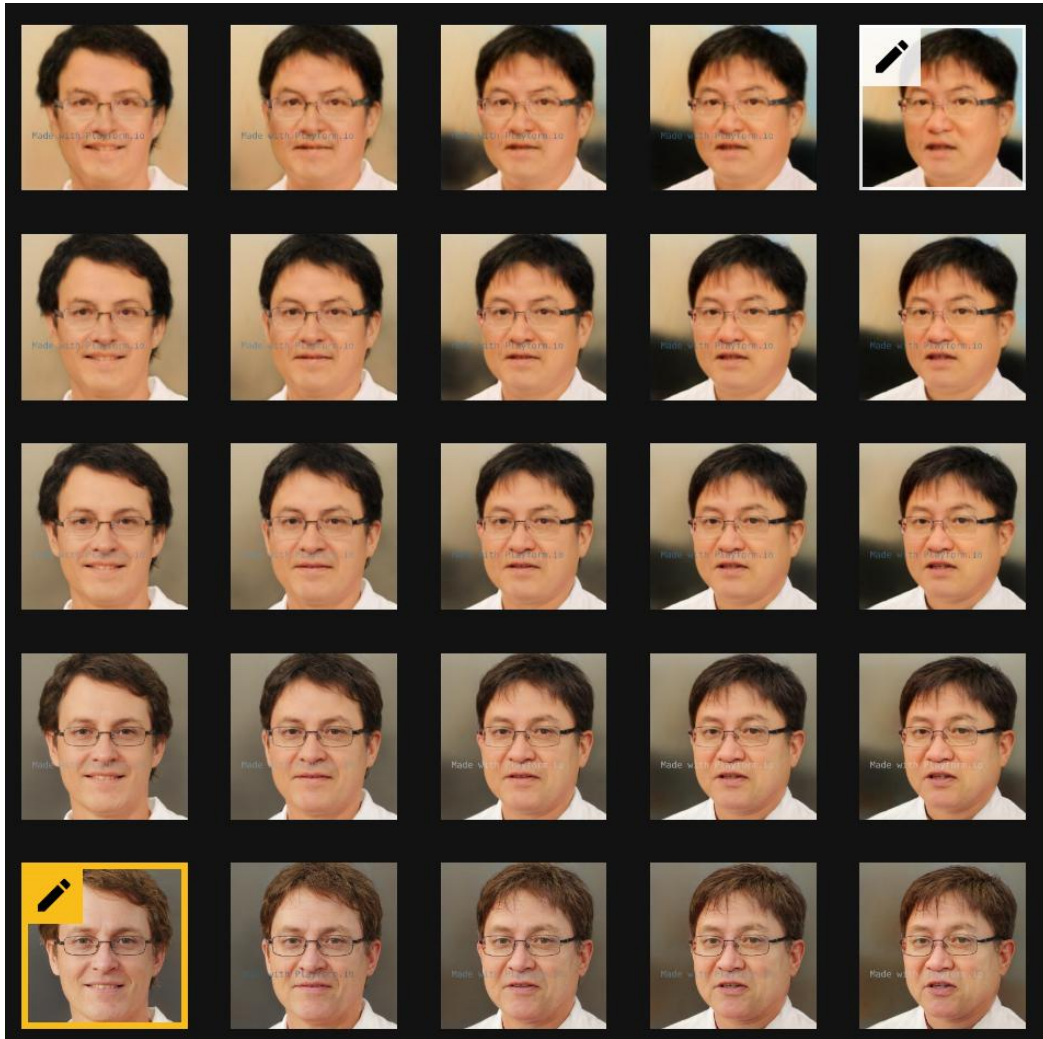


The filmmakers behind *Furiosa: A Mad Max Saga* [used AI](#) to make a child actor look like the lead actress Anya Taylor-Joy and blend their faces together.

출처 : <https://petapixel.com/2024/05/29/furiosa-a-mad-max-saga-used-ai-to-mix-anya-taylor-joys-face-with-child-actor/>



이미지 합성(Mix) - 얼굴 합성





"더 이쁘게" 네이버 스노우 AI 증명사진, 여권사진으로 쓸 수 있을까?

오승혁 · 2024. 9. 5. 00:02



뉴진스 해린의 실제 증명사진(오른쪽)과 스노우를 통해 생성한 증명사진(왼쪽)을 나란히 뒀다. /스노우

[더팩트 | 오승혁 기자] 기존 사진에 AI(인공지능) 기술을 적용해 매력적인 부분을 극대화하는 스노우의 증명사진 기술이 전 세계적으로 큰 관심을 끄는 가운데, 신분증 위조 및 도용 등의 범죄를 막으려면 사진 관련 정책을 변경해야 한다는 주장도 함께 나온다.

4일 업계에 따르면 네이버의 자회사 스노우는 자사의 스노우 앱에 투입되는 증명사진 생성 AI 기술을 고도화하면서 매출 및 시장 내 사용량 지속 증가하고 있다.

반면 과도한 보정이 적용된 사진은 증명사진의 역할을 하기 힘들어 주민등록증 및 여권 등의 신분증 발급에 사용할 수 없다는 규정에도 불구하고 스노우로 만든 증명사진도 발급이 가능한 점은 문제로 지적된다.

현행 주민등록법 시행령 제36조는 주민등록증 신규 또는 재발급 신청자가 관계 공무원에게 6개월 이내에 촬영한 사진 한 장을 제출해야 한다고 규정하고 있다. 주민등록증 발급 과정에서 인화 사진을 제출하게 해 포토샵 등의 편집 프로그램으로 사진을 변형시켜 제출하는 것을 막고자 생긴 규정이지만, 사진관의 보정이 일반화된 상황에서 이 규정 자체가 낡았다는 비판이 나온다.

한 법조계 관계자는 "증명사진이 신분증에서 제 역할을 하길 바란다면 정부 차원에서 행정 센터에서 웹캠과 포토프린터 등의 장비를 도입하고 관계 공무원을 교육해 신분증 발급을 신청하는 현장에서 사진을 촬영하는 방식도 검토해 볼 필요가 있다"고 제언했다.

VAE : Celeb. Face 훈련 – CelebA (CelebFaces Attributes)

- 데이터 : <https://www.kaggle.com/jessicali9530/celeba-dataset>
- 이미지 : 202599 개의 (128x128x3) 이미지
- 테스트 장비

Windows 버전

Windows 10 Pro

© 2019 Microsoft Corporation. All rights reserved.

시스템

프로세서: Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz 2.90 GHz

설치된 메모리(RAM): 32.0GB(31.8GB 사용 가능)

시스템 종류: 64비트 운영 체제, x64 기반 프로세서

펜 및 터치: 이 디스플레이에 사용할 수 있는 펜 또는 터치식 입력이 없습니다.

컴퓨터 이름, 도메인 및 작업 그룹 설정

컴퓨터 이름: DESKTOP-RU5NEBC 

전체 컴퓨터 이름: DESKTOP-RU5NEBC

컴퓨터 설명:

작업 그룹: WORKGROUP

Windows 정품 인증

Windows 정품 인증을 받았습니다. [Microsoft 소프트웨어 사용 조건 읽기](#)

제품 ID: 00330-50665-60992-AAOEM 

시스템 디스플레이 렌더링 소리 입력

디바이스

이름: NVIDIA GeForce 940MX

제조업체: NVIDIA

칩 유형: GeForce 940MX

DAC 유형: Integrated RAMDAC

디바이스 유형: 렌더링 전용 디스플레이 장치

전체 메모리 근사값: 18305 MB

디스플레이 메모리(VRAM): 2010 MB

드라이버

주 드라이버: nvdumdx.dll,nvdumdx.dll,nvdumdx.dll

버전: 26.21.14.4332

날짜: 6/5/2020 09:00:00

WHQL 서명됨: 예

Direct3D DDI: 12

기능 수준: 11_0,10_1,10_0,9_3,9_2,9_1

드라이버 모델: WDDM 2.6

DirectX 기능

DirectDraw 가속: 사용

Direct3D 가속: 사용

AGP 텍스처 가속: 사용

참고

- 아무 문제를 찾지 못했습니다.

도움말(H) 다음 페이지(N) 모든 정보 저장(S)... 끝내기(X)

VAE Face Analysis 훈련 – CelebA (CelebFaces Attributes)

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 128, 128, 3)	0	
encoder_conv_0 (Conv2D)	(None, 64, 64, 32)	896	encoder_input[0][0]
batch_normalization_57 (Batch Normalization)	(None, 64, 64, 32)	128	encoder_conv_0[0][0]
leaky_re_lu_92 (LeakyReLU)	(None, 64, 64, 32)	0	batch_normalization_57[0][0]
dropout_57 (Dropout)	(None, 64, 64, 32)	0	leaky_re_lu_92[0][0]
encoder_conv_1 (Conv2D)	(None, 32, 32, 64)	18496	dropout_57[0][0]
batch_normalization_58 (Batch Normalization)	(None, 32, 32, 64)	256	encoder_conv_1[0][0]
leaky_re_lu_93 (LeakyReLU)	(None, 32, 32, 64)	0	batch_normalization_58[0][0]
dropout_58 (Dropout)	(None, 32, 32, 64)	0	leaky_re_lu_93[0][0]
encoder_conv_2 (Conv2D)	(None, 16, 16, 64)	36928	dropout_58[0][0]
batch_normalization_59 (Batch Normalization)	(None, 16, 16, 64)	256	encoder_conv_2[0][0]
leaky_re_lu_94 (LeakyReLU)	(None, 16, 16, 64)	0	batch_normalization_59[0][0]
dropout_59 (Dropout)	(None, 16, 16, 64)	0	leaky_re_lu_94[0][0]
encoder_conv_3 (Conv2D)	(None, 8, 8, 64)	36928	dropout_59[0][0]
batch_normalization_60 (Batch Normalization)	(None, 8, 8, 64)	256	encoder_conv_3[0][0]
leaky_re_lu_95 (LeakyReLU)	(None, 8, 8, 64)	0	batch_normalization_60[0][0]
dropout_60 (Dropout)	(None, 8, 8, 64)	0	leaky_re_lu_95[0][0]
flatten_14 (Flatten)	(None, 4096)	0	dropout_60[0][0]
mu (Dense)	(None, 200)	819400	flatten_14[0][0]
log_var (Dense)	(None, 200)	819400	flatten_14[0][0]
encoder_output (Lambda)	(None, 200)	0	mu[0][0] log_var[0][0]
Total params: 1,732,944 Trainable params: 1,732,496 Non-trainable params: 448			

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 200)	0
dense_14 (Dense)	(None, 4096)	823296
reshape_14 (Reshape)	(None, 8, 8, 64)	0
decoder_conv_t_0 (Conv2DTranspose)	(None, 16, 16, 64)	36928
batch_normalization_61 (Batch Normalization)	(None, 16, 16, 64)	256
leaky_re_lu_96 (LeakyReLU)	(None, 16, 16, 64)	0
dropout_61 (Dropout)	(None, 16, 16, 64)	0
decoder_conv_t_1 (Conv2DTranspose)	(None, 32, 32, 64)	36928
batch_normalization_62 (Batch Normalization)	(None, 32, 32, 64)	256
leaky_re_lu_97 (LeakyReLU)	(None, 32, 32, 64)	0
dropout_62 (Dropout)	(None, 32, 32, 64)	0
decoder_conv_t_2 (Conv2DTranspose)	(None, 64, 64, 32)	18464
batch_normalization_63 (Batch Normalization)	(None, 64, 64, 32)	128
leaky_re_lu_98 (LeakyReLU)	(None, 64, 64, 32)	0
dropout_63 (Dropout)	(None, 64, 64, 32)	0
decoder_conv_t_3 (Conv2DTranspose)	(None, 128, 128, 3)	867
activation_14 (Activation)	(None, 128, 128, 3)	0
Total params: 917,123 Trainable params: 916,803 Non-trainable params: 320		

VAE에 의한 이미지 변환 (Mouth Open Vector) $Z_n = Z + \alpha \cdot \text{feature_vector}$

원본

-4

-3

-2

-1

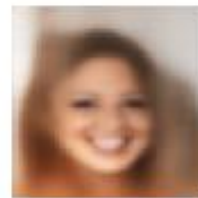
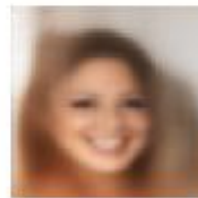
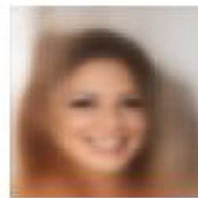
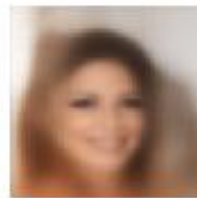
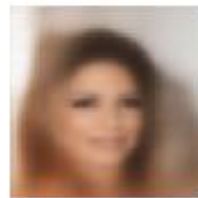
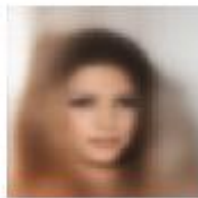
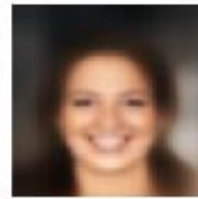
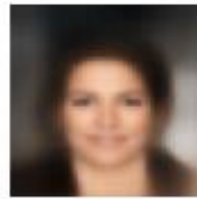
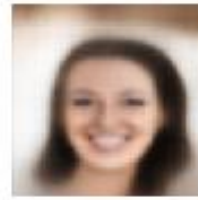
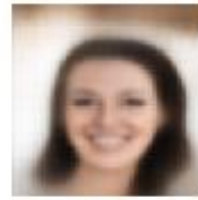
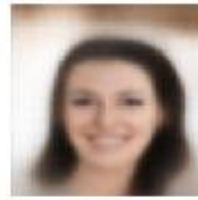
0

1

2

3

4



VAE에 의한 이미지 변환 (Smiling Vector)

$$Z_n = Z + \alpha \cdot \text{feature_vector}$$

원본

-4

-3

-2

-1

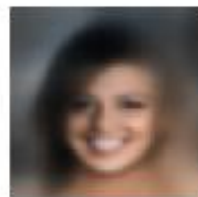
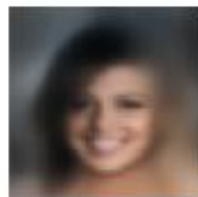
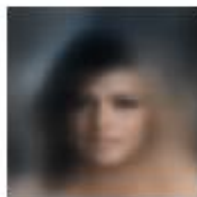
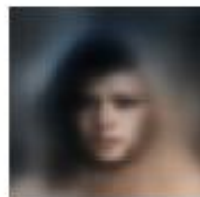
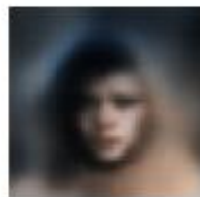
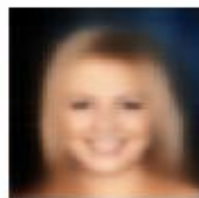
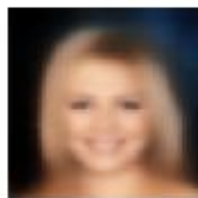
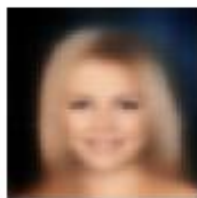
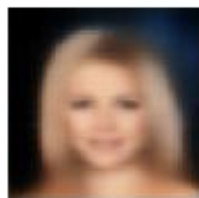
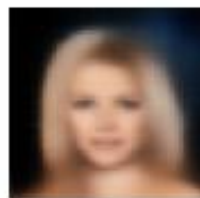
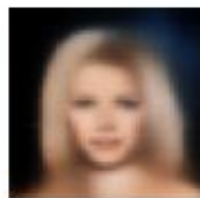
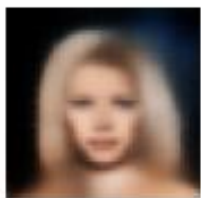
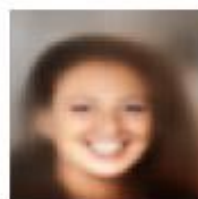
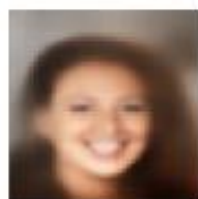
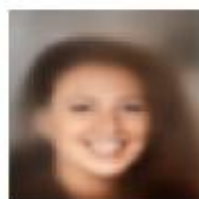
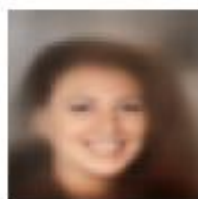
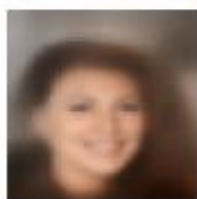
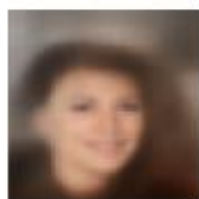
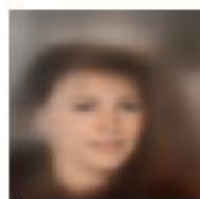
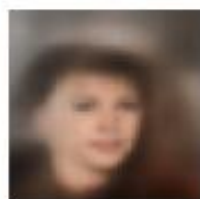
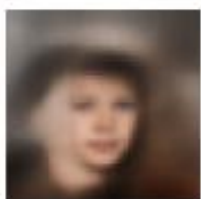
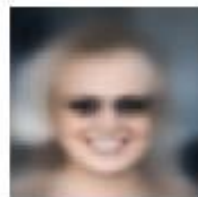
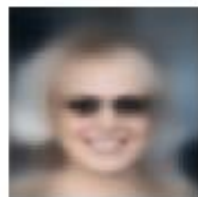
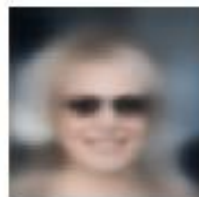
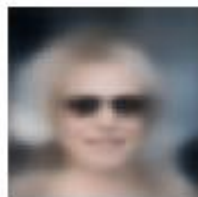
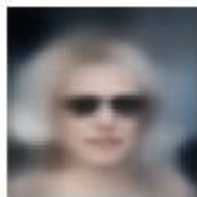
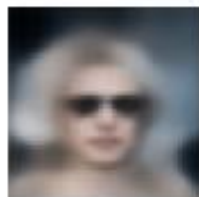
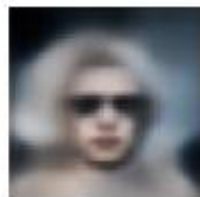
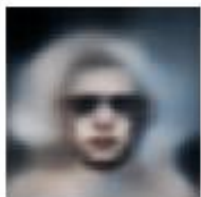
0

1

2

3

4



VAE에 의한 이미지 변환 (Eyeglasses Vector)

$$Z_n = Z + \alpha \cdot \text{feature_vector}$$

원본

-4

-3

-2

-1

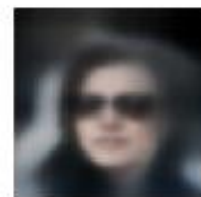
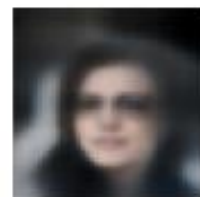
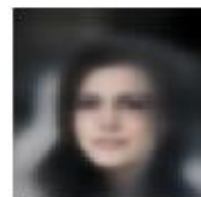
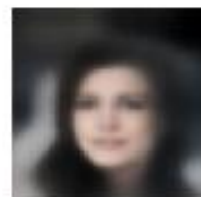
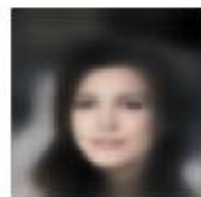
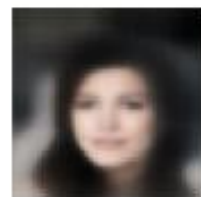
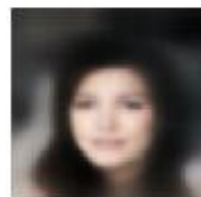
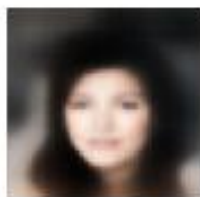
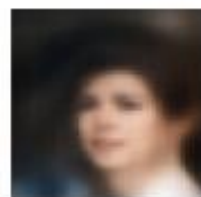
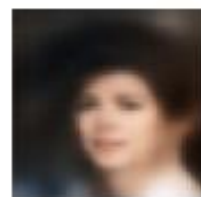
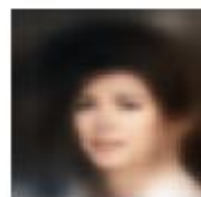
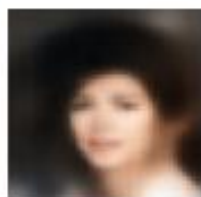
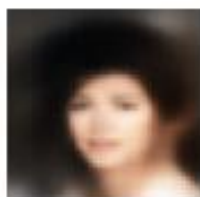
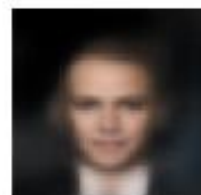
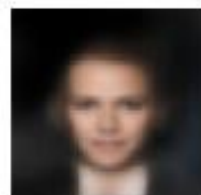
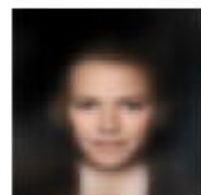
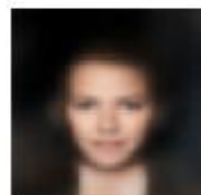
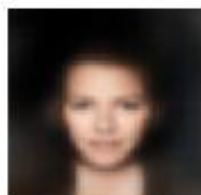
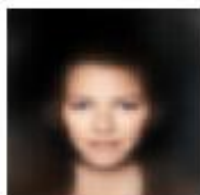
0

1

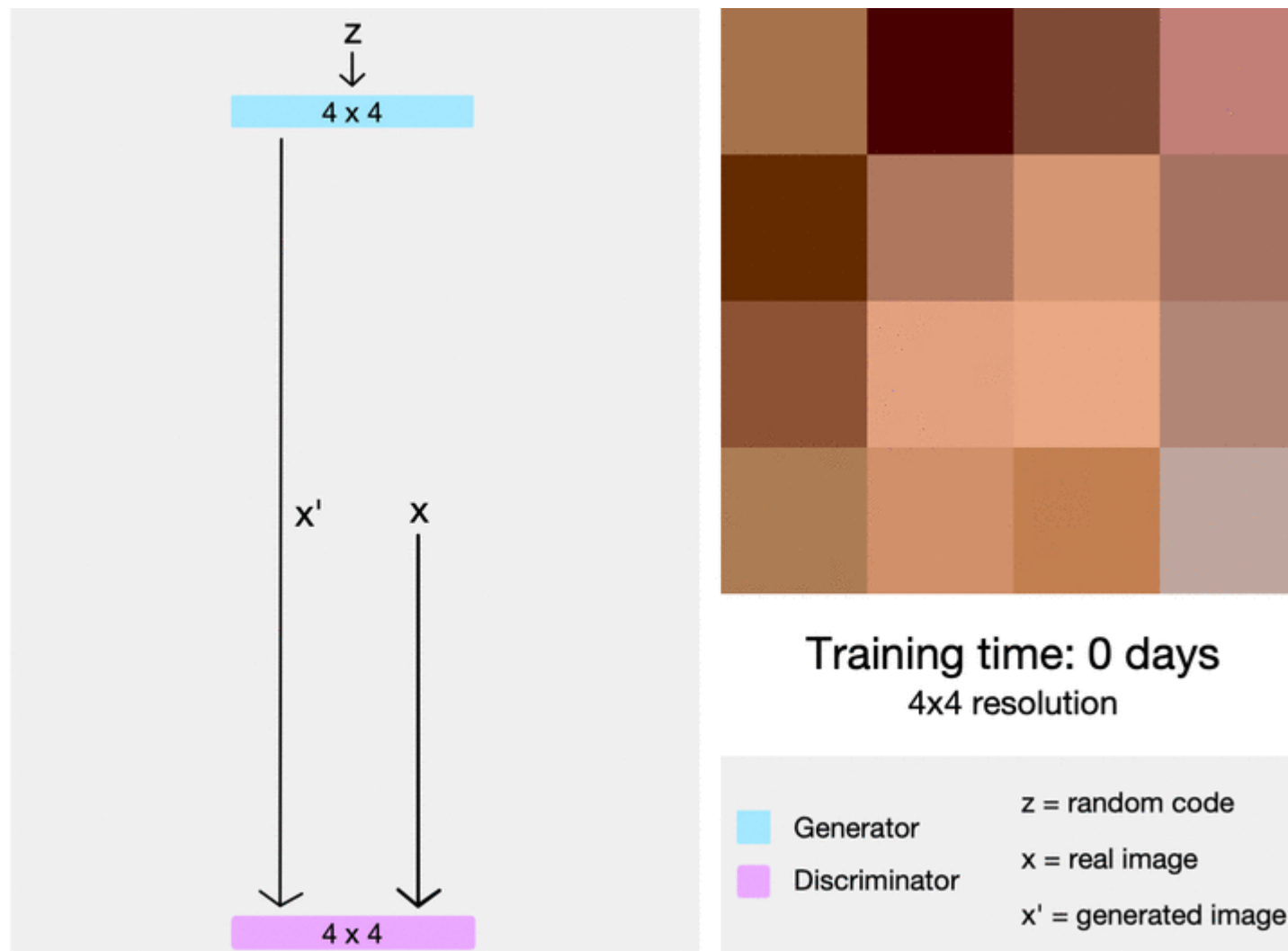
2

3

4



Progressive Growing GAN 대표 개념



GAN 특성 공간(임베딩) 이미지 생성



2022.07.17(일)

홈 > 뉴스 > 인물

대전 ▾

21°C



미세먼지

실종 38년만에 가족 품으로, 사장될뻔 한 이 기술 쓰였다

김지영 기자 | orghs12345@hellodd.com | 입력 2022.07.14 17:30 | 수정 2022.07.14 17:58 | 댓글 0



1. 기본 얼굴



2. 매력적임



3. 비열함



1. 유아기



2. 청소년기



3. 20대



4. 40대

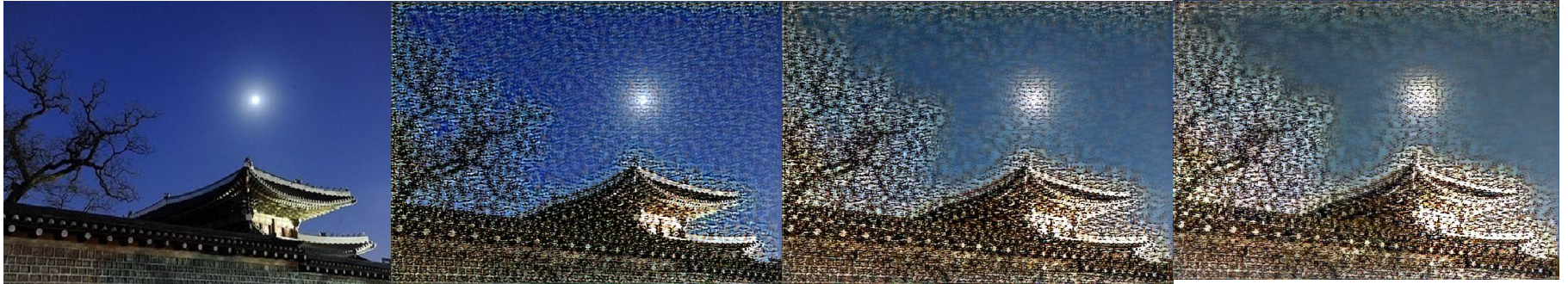
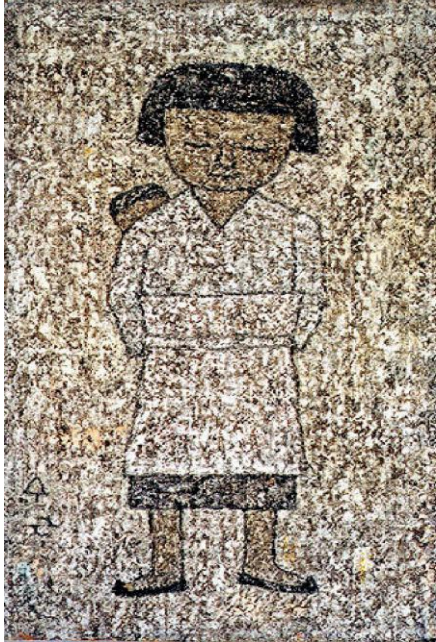


5. 60대



6. 80대

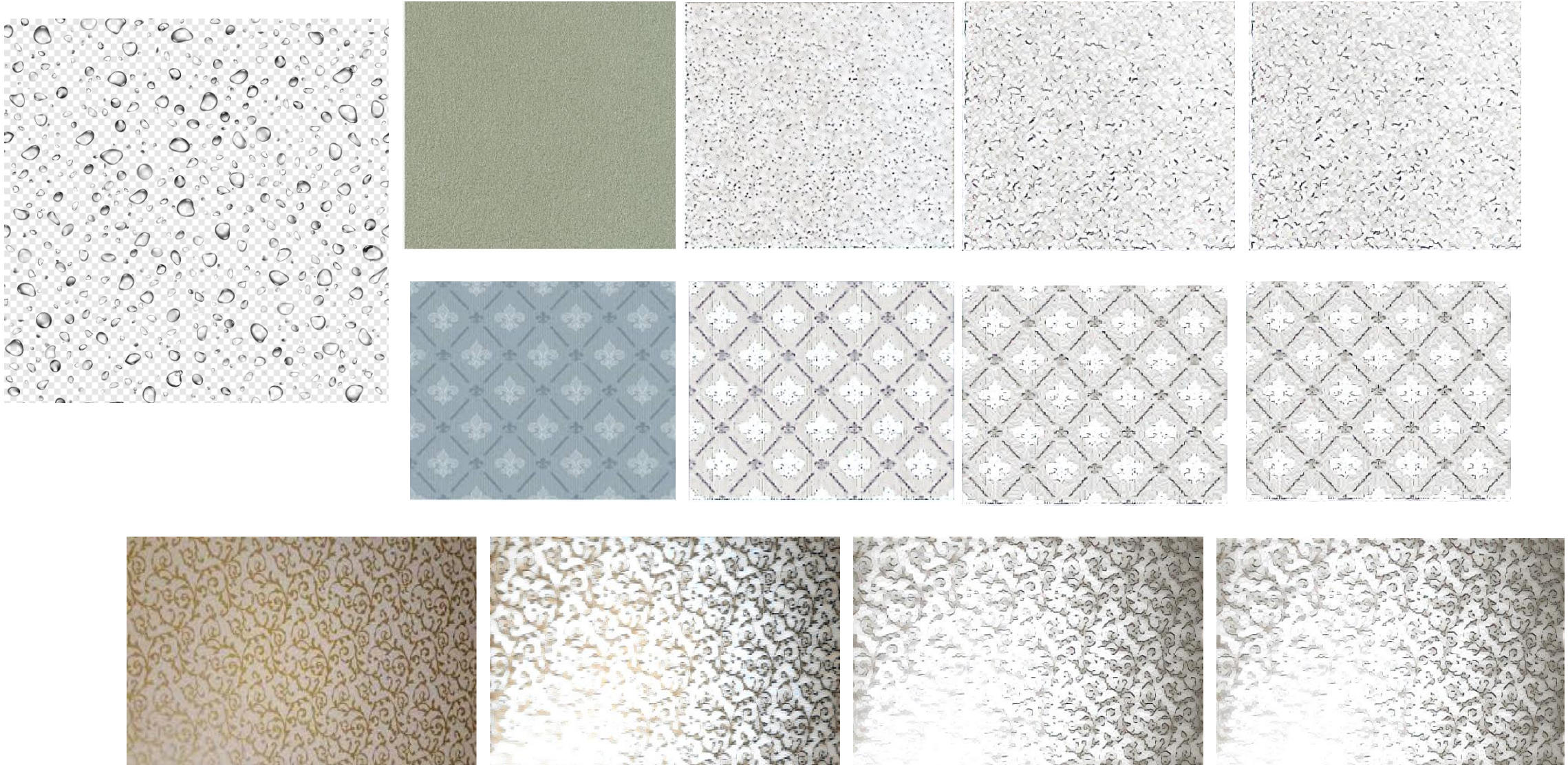
스타일 전이 (Style Transfer)



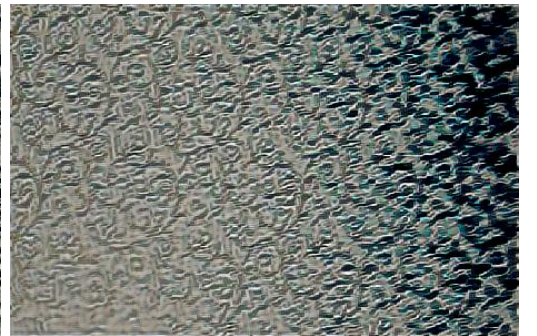
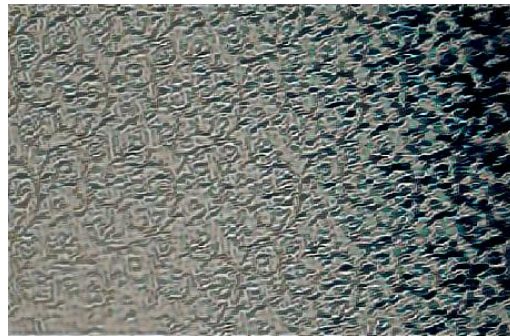
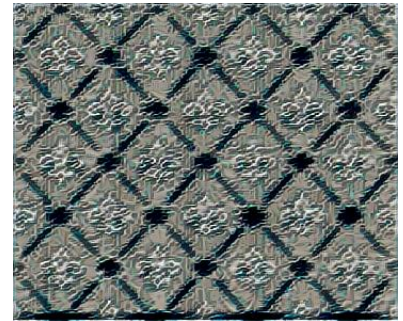
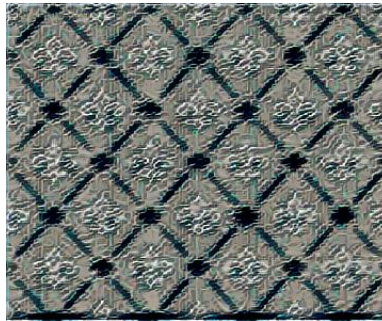
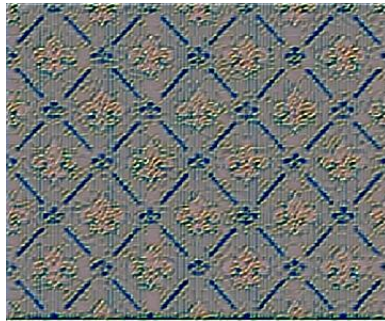
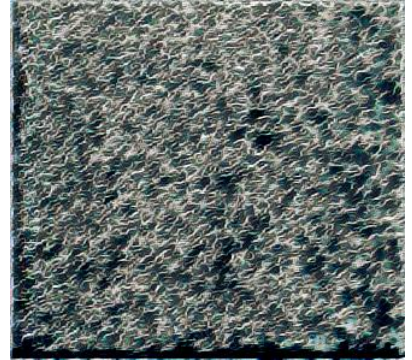
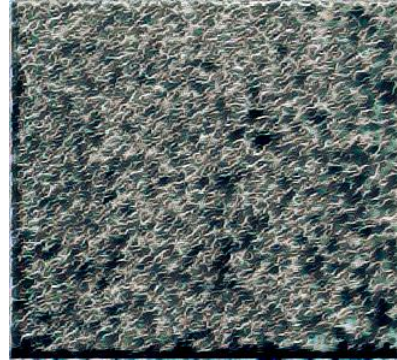
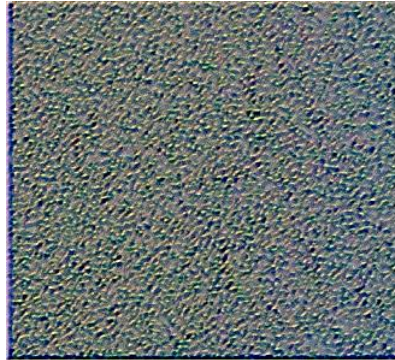
[예시] 이미지 스타일 전이 – 색상 유지



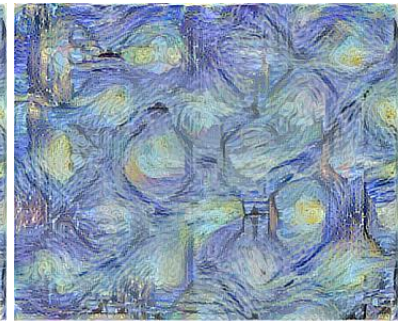
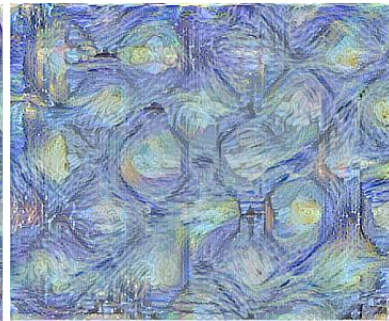
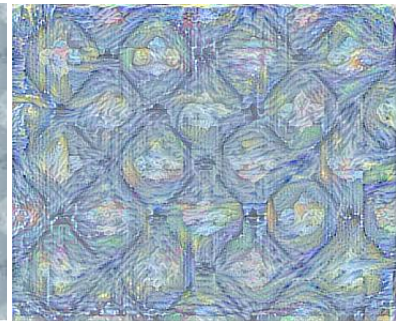
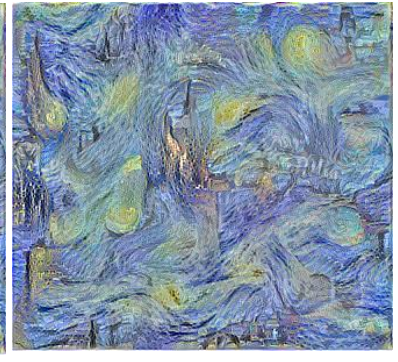
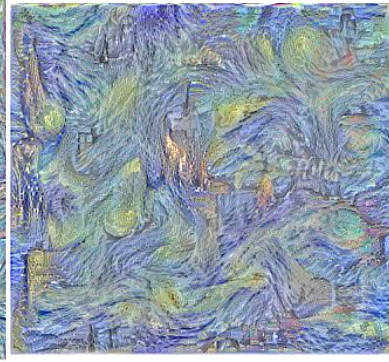
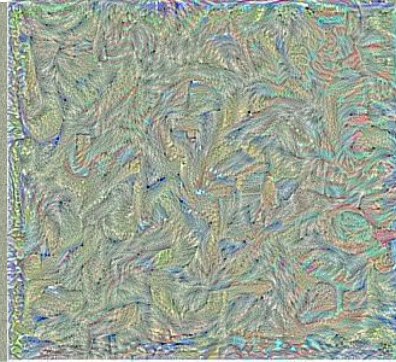
벽지 스타일 Transfer (Layer 1-1, 1-2)



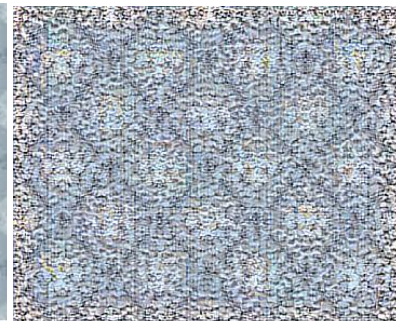
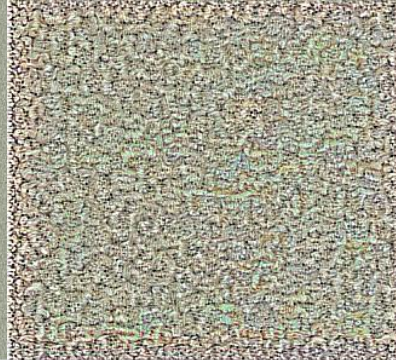
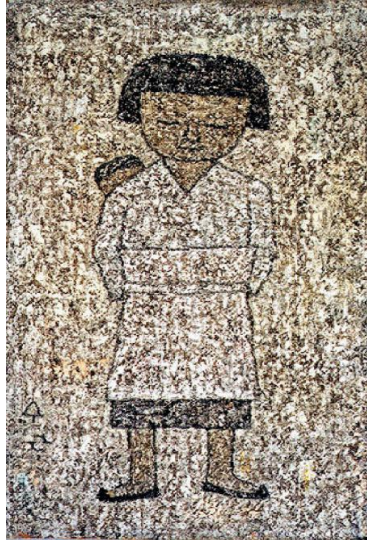
벽지 스타일 Transfer (Layer 1-1, 1-2)



벽지 스타일 Transfer

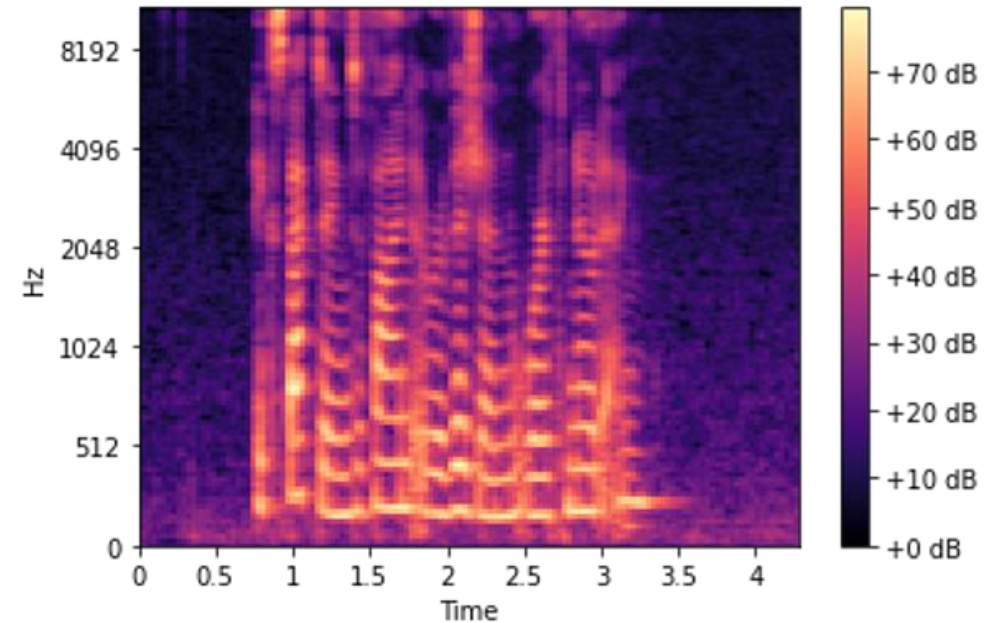
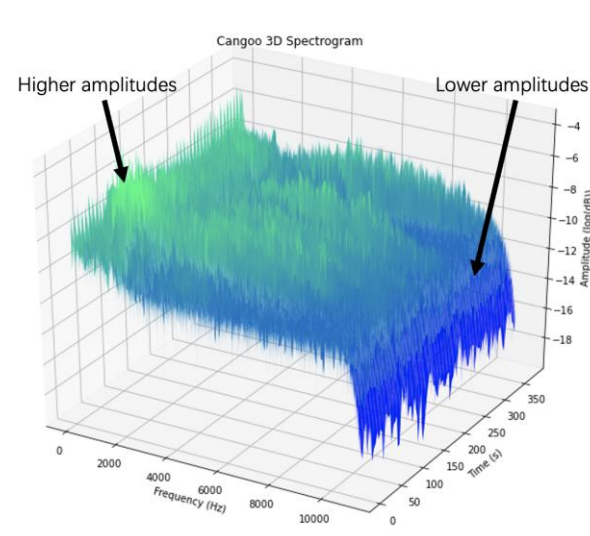
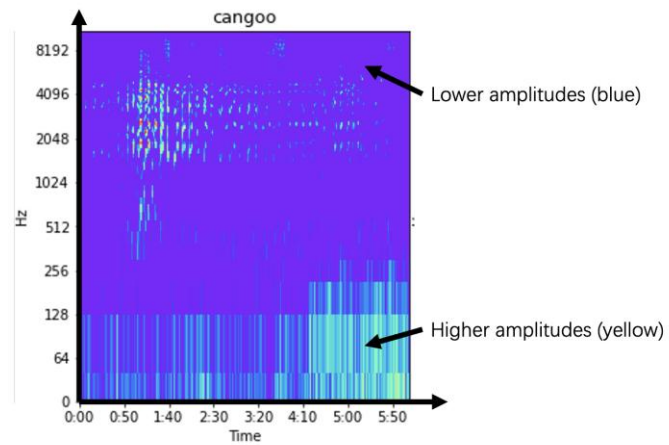
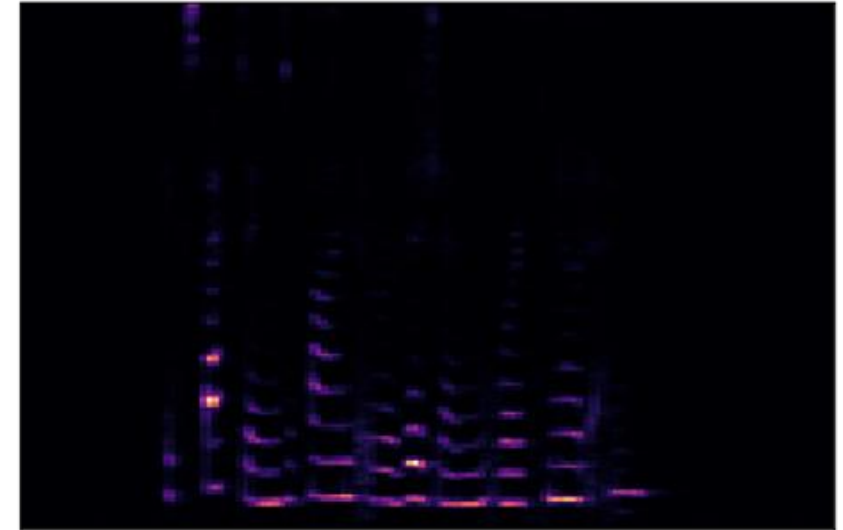
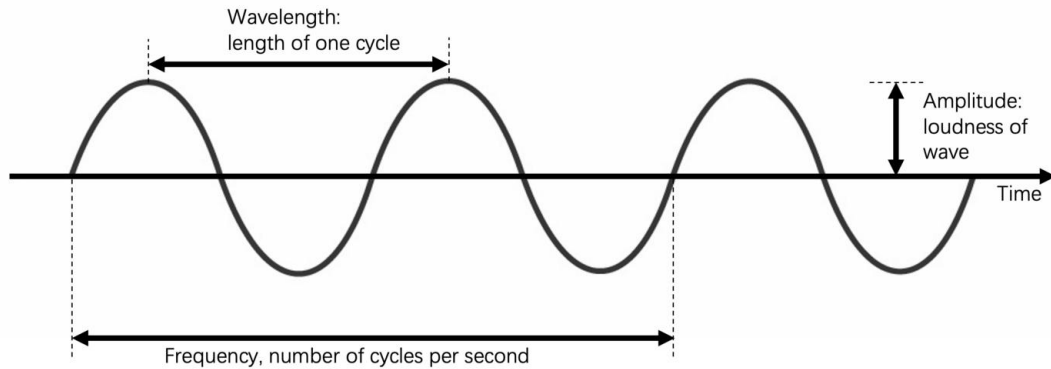


벽지 스타일 Transfer



소리의 공간
음악(소리)을 수치화 할 수 있을까?

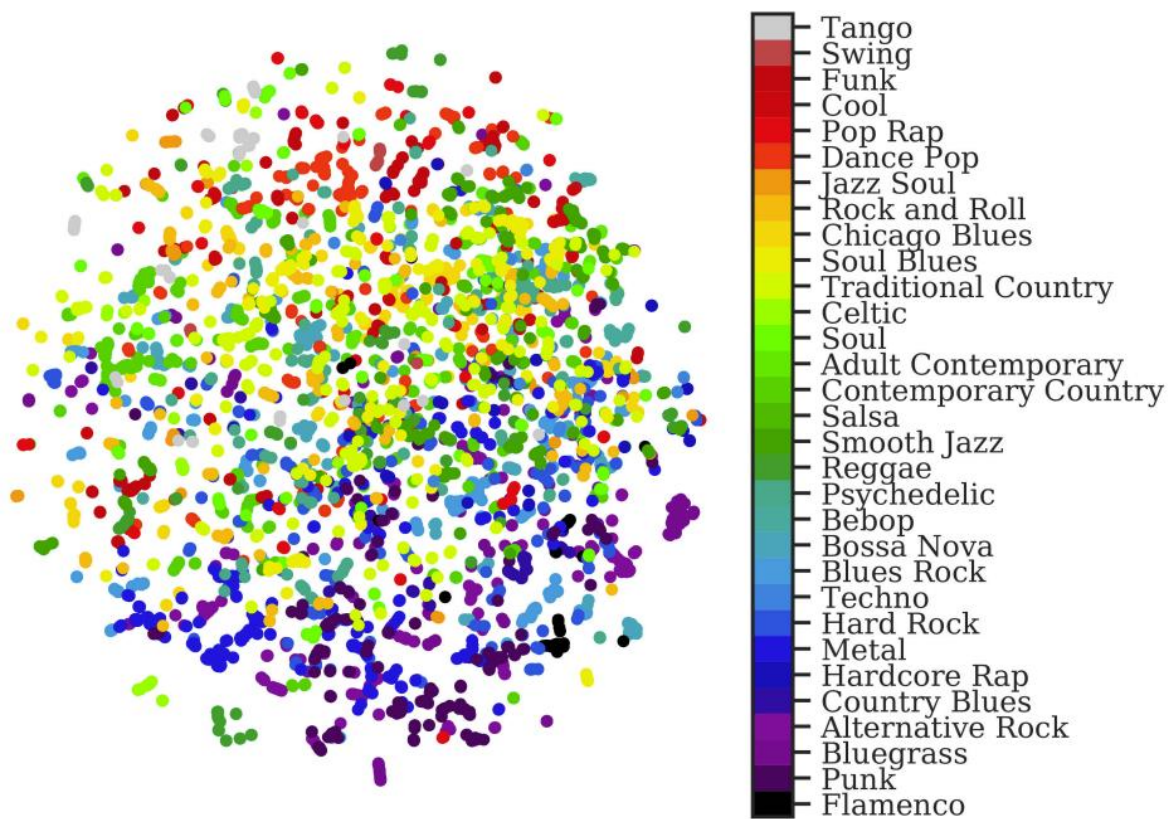
음악(소리)는 이미지처럼 다룰 수 있다?



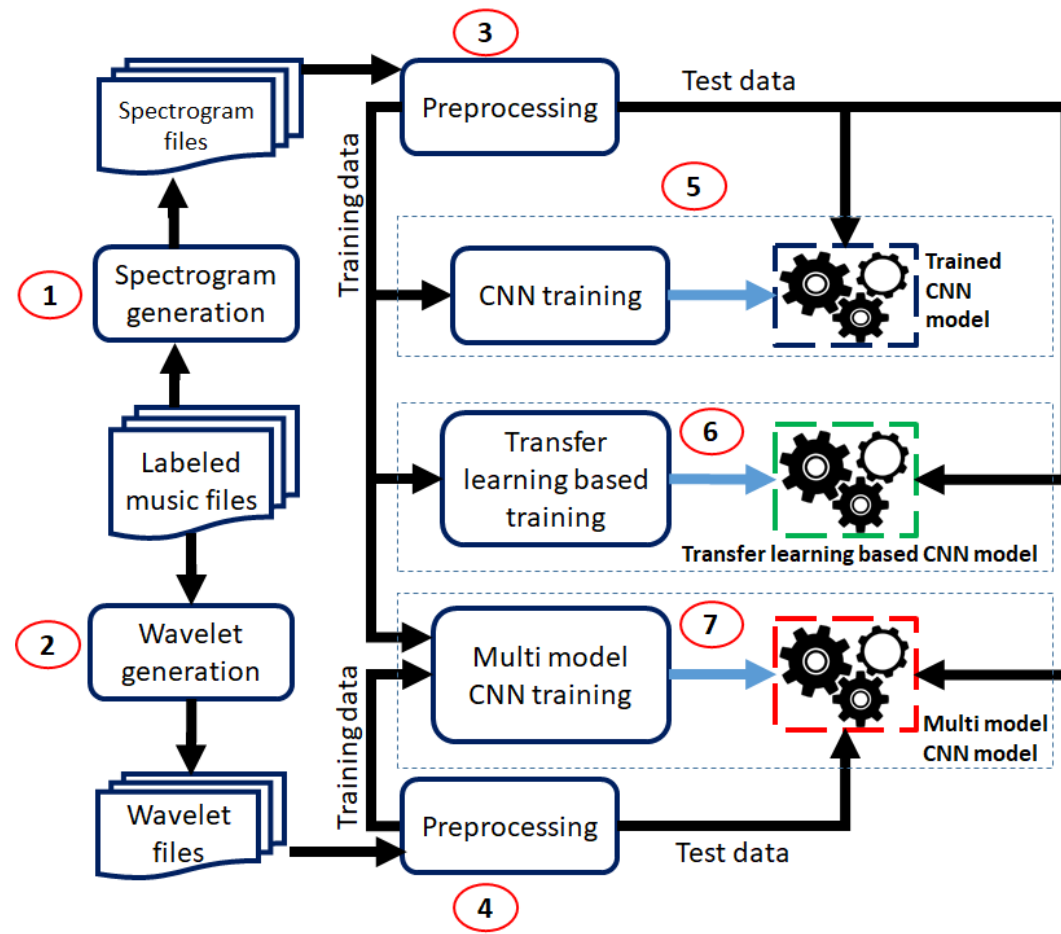
출처 : [Seeing is Believing: Converting Audio Data into Images | by Tony Chen | Towards Data Science](#)

출처 : <https://towardsdatascience.com/audio-deep-learning-made-simple-part-2-why-mel-spectrograms-perform-better-aad889a93505>

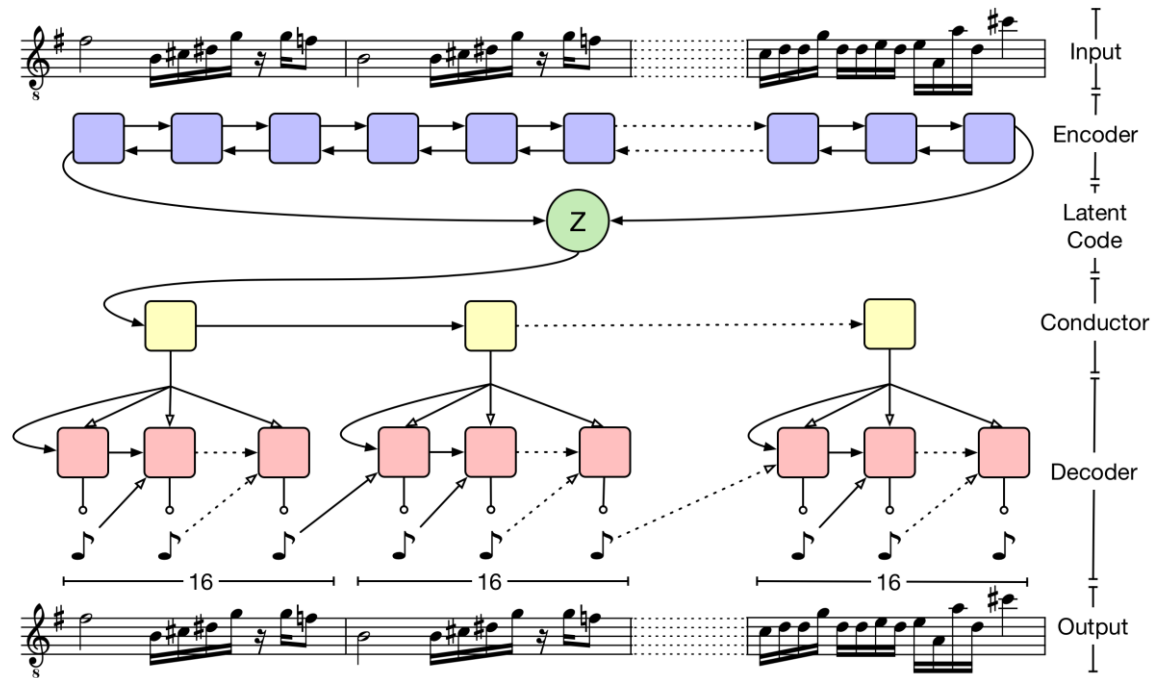
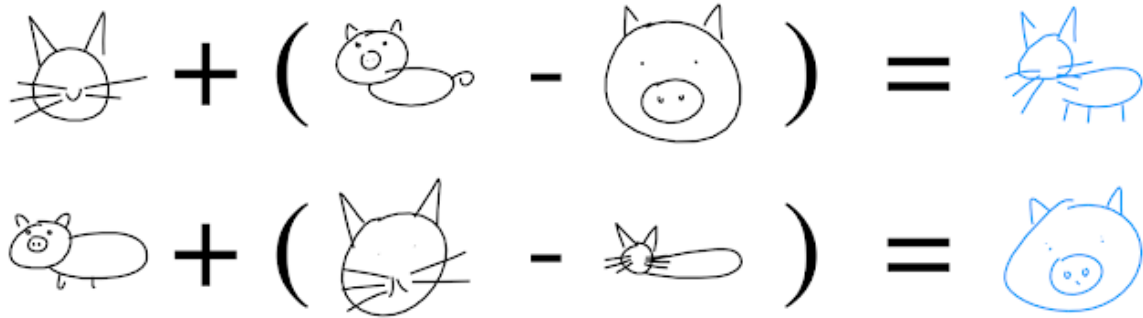
음악(소리)가 수치화(벡터화)가 되면



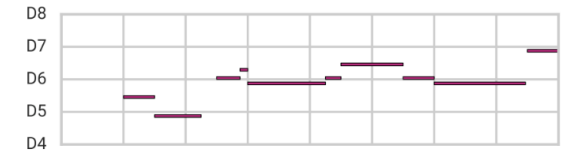
(c) Guitar embeddings by genre



음악(소리)가 수치화(벡터화)가 되면

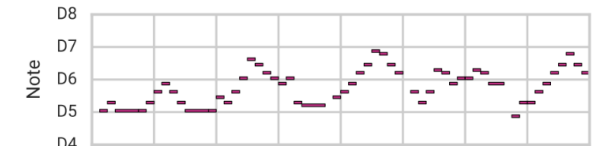


Subtract Note Density Vector



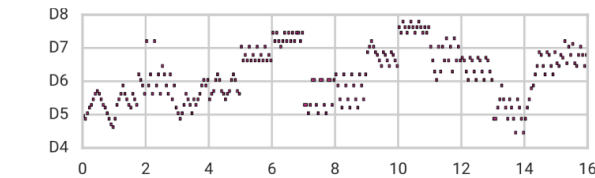
▶ 0:00 / 0:38 🔊 ⋮

Original



▶ 0:00 / 0:38 🔊 ⋮

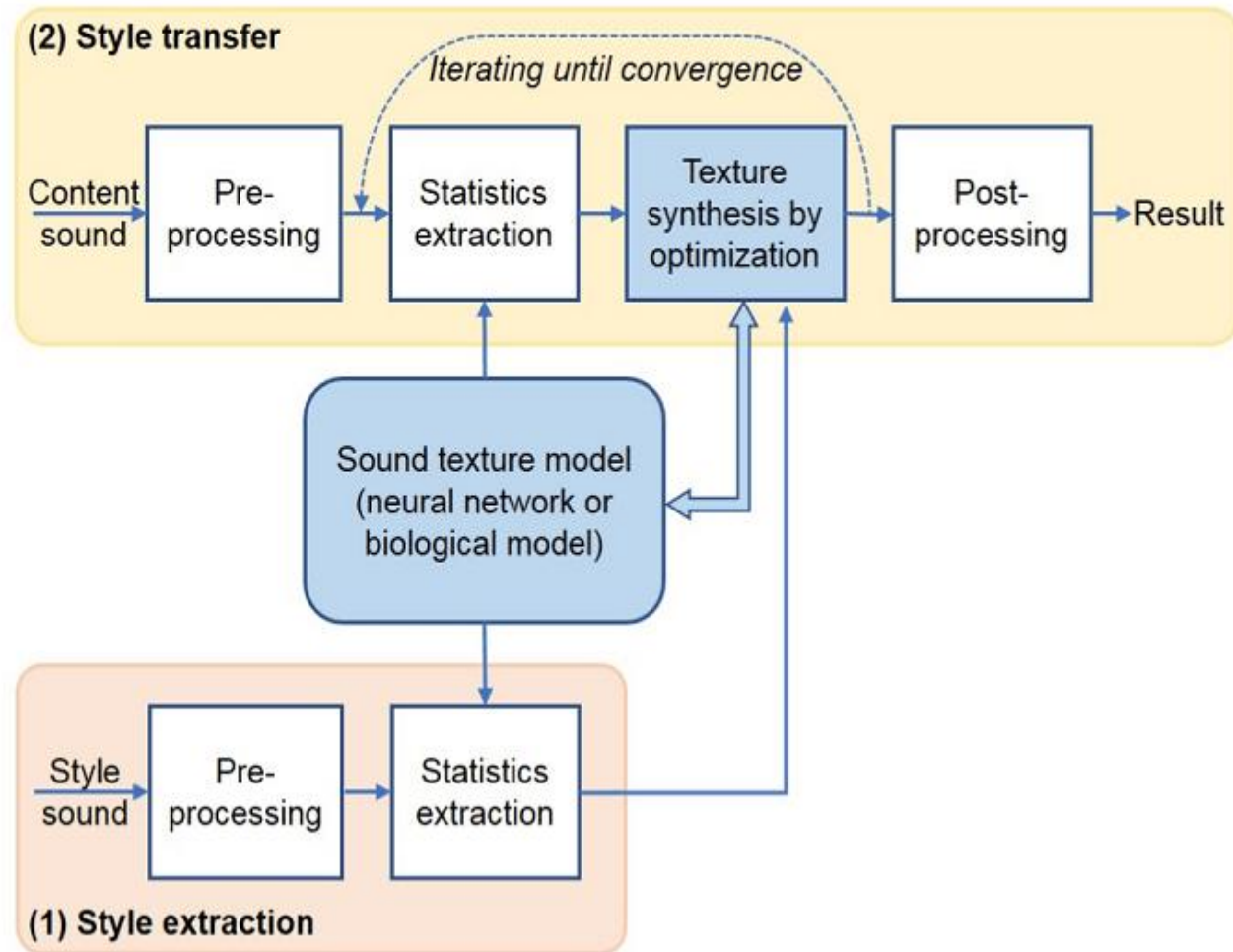
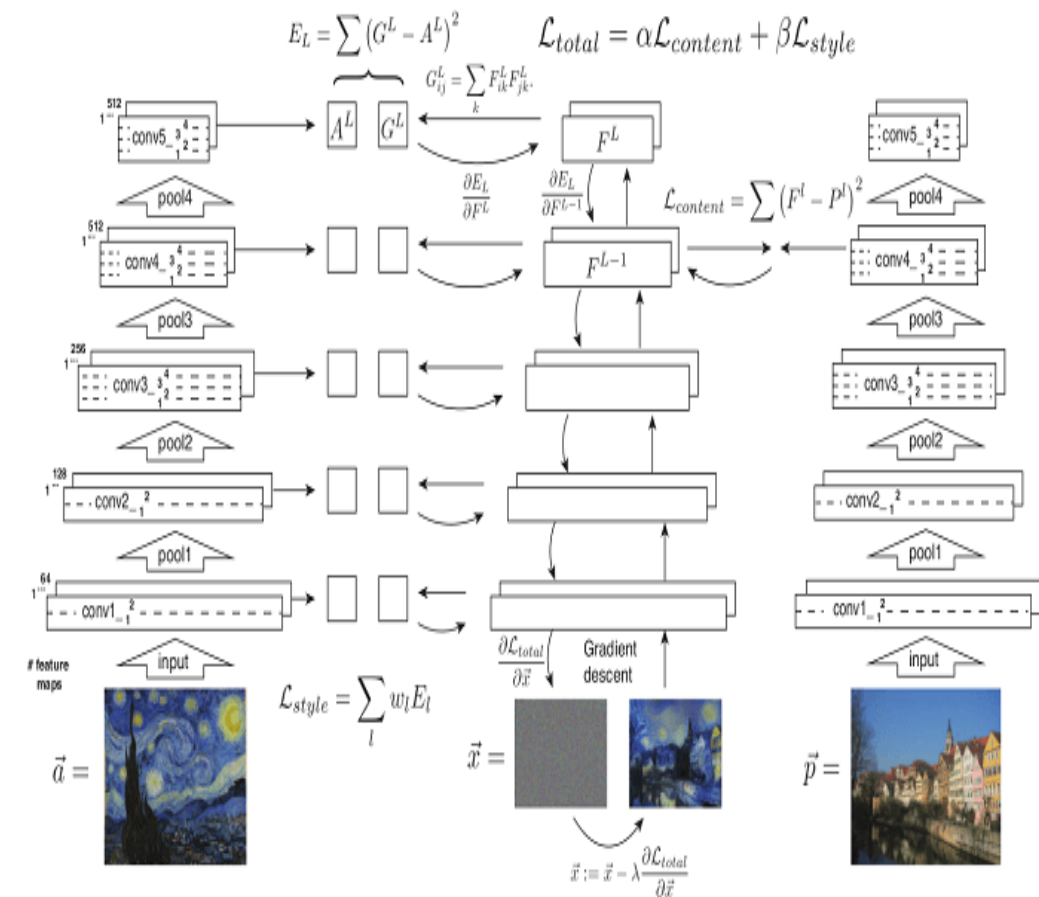
Add Note Density Vector

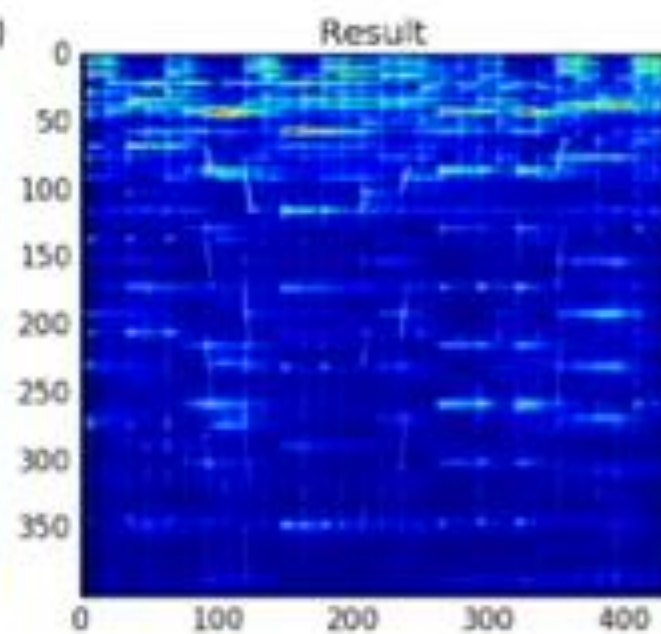
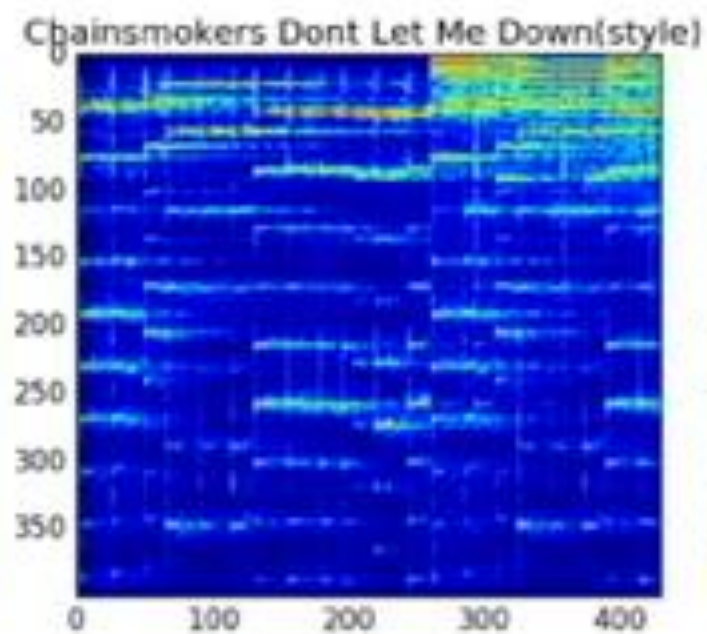
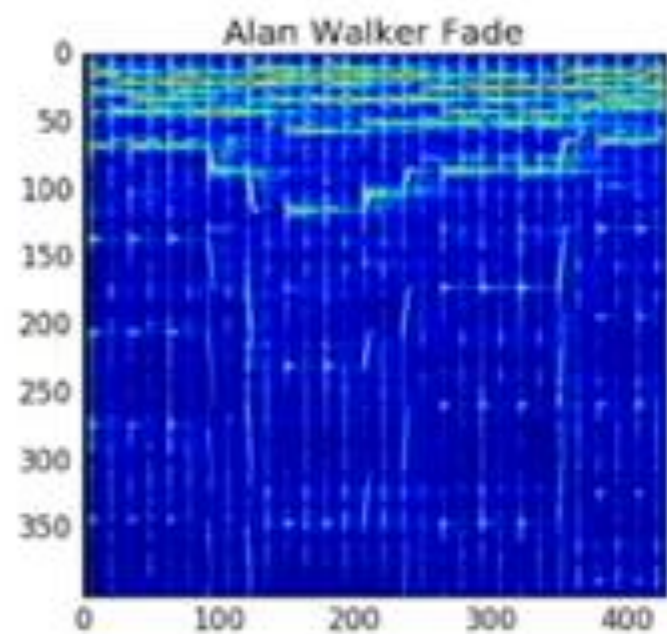


▶ 0:00 / 0:38 🔊 ⋮

Notice that notes are not simply repeated to increase their density. Instead MusicVAE adds arpeggios and other musically-relevant flourishes. Listen to more examples of attribute vectors in the paper's [online supplement](#).

음악(소리) 스타일 전이(Transfer)





텍스트에서 음악 생성하는 인공지능(AI) '리퓨전'

박찬 위원 입력 2022.12.20 17:00 댓글 0 좋아요 0

'스테이블 디퓨전'에서 시각 초음파를 오디오로 변환해 음악 생성

시각 초음파를 사용해 텍스트로 음악을 생성하는 인공지능(AI) 모델이 나왔다.

아르스테크니카는 세스 포스그렌과 하이크 마르티로스는 개발자가 사운드의 시각적 표현을 생성하고 이를 오디오로 변환해 텍스트 프롬프트에서 음악을 생성하는 AI 모델 '리퓨전(Riffusion)'을 출시했다고 17일(현지시간) 보도했다.

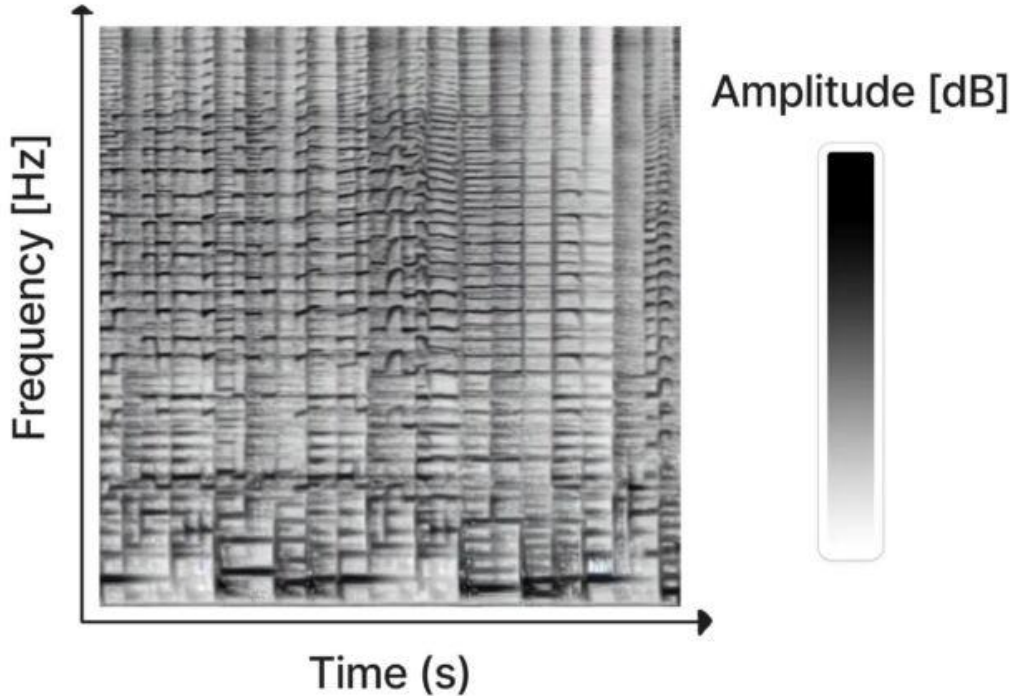
'리퓨전'은 이미지 생성 AI '스테이블 디퓨전' 1.5 이미지 합성 모델의 미세조정 버전을 사용해 오디오를 2차원 이미지로 표현하는 소노그램을 생성해 작동한다.

소노그램은 음원 신호의 시간 변화에 따른 주파수 성분 분석을 위한 그래프다. 시간 경과에 따라 서로 다른 주파수의 진폭을 보여주는 오디오의 시각적 표현으로 X축은 시간, Y축은 소리의 주파수를 나타낸다.

리퓨전을 개발한 포스그렌과 마르티로스는 초음파는 사진의 일종이라 스테이블 디퓨전으로 처리할 수 있다는 점을 활용했다. 이들은 여러 음악의 소노그램을 만들고 '블루스 기타' '재즈 피아노' '아프로비트' 등 관련 용어로 결과 이미지에 태그를 지정한 훈련 데이터를 사용해 스테이블 디퓨전을 미세조정했다.

미세조정은 사전 훈련된 모델을 특정 종류의 콘텐츠를 사용해 추가적으로 훈련시켜 해당 콘텐츠를 전문적으로 생성할 수 있도록 한다. 이러한 미세조정 결과로 리퓨전은 '재즈', '록' 또는 키보드 입력과 같이 듣고 싶은 음악이나 소리의 유형을 설명하는 텍스트 프롬프트를 기반으로 즉석에서 새로운 음악을 생성할 수 있다.

리퓨전은 소노그램을 토치오디오를 사용해 사운드 웨이브폼으로 변경하고 오디오로 재생한다. 또 시드를 변경해 프롬프트의 변형을 무한히 생성할 수 있다. 또 다양한 스타일의 음악을 융합할 수도 있다. 예를 들어, '부드러운 트로피컬 댄스 재즈'를 입력하면 다양한 장르의 요소를 가져와 스타일을 혼합한다.



스펙트로그램은 2차원 이미지에서 시간, 주파수 및 진폭을 나타낸다.(사진=포스그렌 & 마르티로스)

물론 리퓨전이 최초의 AI 기반 음악 생성기는 아니다. 올해 초 하모나이는 AI 기반 생성 음악 모델인 댄스 리퓨전을 출시했다 2020년에 발표된 오픈AI의 주크박스도 신경망으로 새로운 음악을 생성한다. 그리고 사운드로와 같은 웹사이트는 즉석에서 논스톱으로 음악을 만든다.

다른 AI 음악 작업과 비교할 때 리퓨전은 다소 단순한 것처럼 느껴진다. 리퓨전이 생성하는 음악은 흥미로운 것부터 이해할 수 없는 것까지 다양하지만 시각적 공간에서 오디오를 조작하는 확산(Diffusion) 기술을 사용했다는 점은 주목할만한 하다.

리퓨전 모델 체크포인트와 코드는 깃허브에서 사용할 수 있다.

[RIFFUSION]



anger rap

church bells

jamaican dancehall vocals

...

UP NEXT: 90s ambient idm



What do you want to hear next?

음악(소리) 잠재 공간에서 할 수 있는 일

- Style Transfer

- ✓ 이미지 스타일 Transfer와 같이 음악의 Style을 변화 시킨다.

- ✓ 예 국악 → 재즈 스타일로



- 감성에 따른 음악 추천/생성

- ✓ 기쁨, 슬픔 등에 어울리는 음악 찾아주기

- ✓ 감정에 맞는 음악 생성

- (웃픈 음악) = $0.5 * (\text{기쁜 음악}) + 0.5 * (\text{슬픈 음악})$

➔ 새로운 음악을 카테고리에 맞게 **자동 분류**

➔ 원하는 느낌/분위기에 **가장 알맞는 음악 찾기**

➔ 메타버스, YouTube 콘텐츠를 위한 **배경음/소리 자동 생성**

임베딩 응용 : Spotify



Spotify 임베딩 기술력 평가 (0~5점)

평가: 5점 (최고 수준)

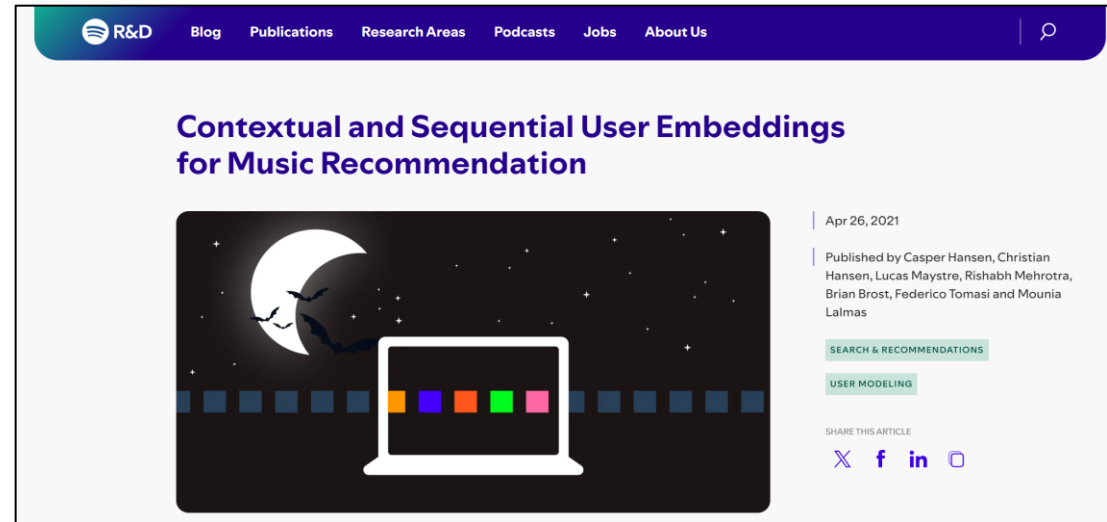
- Spotify는 사용자, 곡, 아티스트 등 모든 주요 엔터티에 대해 임베딩을 적극적으로 활용하며, 이 임베딩이 추천 시스템의 핵심 기반입니다. 임베딩 벡터는 사용자 취향, 곡 특성, 맥락(시간, 디바이스 등)까지 반영하여 매우 정교한 개인화가 가능합니다 [1](#) [2](#) [3](#).
- word2vec, CoSeRNN 등 최신 딥러닝 모델을 대규모로 적용하고, 임베딩 공간에서 유사 사용자/트랙을 효율적으로 탐색하여 Discover Weekly, Daily Mix 등 업계 최고 수준의 추천 서비스를 구현합니다 [1](#) [2](#) [4](#).
- 임베딩 기반 추천은 대규모 데이터(수조 건의 로그)와 실시간 요구(낮은 레이턴시)라는 기술적 난제를 성공적으로 해결하며, 모든 엔지니어가 쉽게 활용할 수 있는 ML 플랫폼을 구축했습니다 [3](#) [5](#) [6](#).

Spotify 임베딩 기술의 응용 분야

- 음악 추천:** Discover Weekly, Release Radar, Daily Mix 등 개인화 플레이리스트 생성 [1](#) [4](#) [6](#)
- 유사 곡/아티스트 추천:** 임베딩 공간에서 유사도 기반으로 곡/아티스트 탐색 [2](#) [3](#)
- 검색 최적화:** 텍스트 임베딩을 활용한 자연어 쿼리의 의미 파악 및 검색 결과 개선 [7](#)
- 오디오 분석:** 곡의 오디오 특성(템포, 악기, 분위기 등)을 임베딩하여 추천에 반영 [4](#)
- 소셜/취향 분석:** 사용자 간 취향 유사도, 소셜 네트워크 분석 [2](#)
- 광고 및 마케팅:** 오디오 광고 타겟팅, 사용자 반응 예측 등 [4](#)
- 신규 사용자(콜드 스타트) 대응:** 임베딩을 활용해 초기 데이터가 적은 사용자도 빠르게 개인화 [6](#)

Spotify의 임베딩 기술은 추천, 검색, 분석, 광고 등 플랫폼 전반에 깊이 응용되고 있으며, 업계 최고 수준으로 평가받을 만합니다.

발췌 : perplexity



음악 추천을 위한 상황별 및 순차적 사용자 임베딩 | 스포티파이 리서치

Link Me Baby One More Time: Spotify의 소셜 뮤직 디스커버리

샤지아아인 바불^{1,2,3,12}, 데시 슬라바 흐리스토포바, ^{3*}, 안토니오 리마 ³,
르노 랑비오프 ^{1,2}, 마리아노 베게리세-디아즈 ^{3,1}

추상적인

우리는 사람 대 사람 음악 추천 및 발견의 결과에 영향을 미치는 사회적, 맥락적 요인을 탐구합니다. 구체적으로 말하자면, Spotify는 Spotify의 데이터를 사용하여 한 사용자가 다른 사용자로 보낸 링크가 어떻게 수신자가 공유된 아티스트의 음악에 참여하는지 조사합니다. 당사는 발신자-수신자 관계의 강도, Spotify 소셜 네트워크에서의 사용자 역할, 음악의 사회적 응집력, 새로운 아티스트가 수신자의 취향과 얼마나 유사한지와 같이 이 프로세스에 영향을 미칠 수 있는 몇 가지 요소를 고려합니다. 우리는 링크를 받는 사람이 (1) 발신자와 음악 취향이 비슷하고 공유 트랙이 그들의 취향에 잘 맞고, (2) 발신자와 더 강하고 친밀한 유대 관계를 가지고 있으며, (3) 공유 아티스트가 수신자의 연결에 인기가 있을 때 새로운 아티스트와 참여할 가능성이 더 높다는 것을 발견했습니다. 마지막으로, 이러한 결과를 사용하여 공유 음악 트랙이 공유 아티스트와 수신자의 참여로 이어질지 여부를 예측하는 Random Forest 분류기를 구축합니다. 이 모델은 어떤 유형의 사회적 및 상황적 특징이 가장 예측적인지 설명하지만, 다양한 기능 집합이 포함될 때 최고의 성능을 얻을 수 있습니다. 이러한 발견은 음악 발견과 사회적 과정 사이의 상호 작용을 뒷받침하는 다면적 메커니즘에 대한 새로운 통찰력을 제공합니다.

Link Me Baby One More Time: Spotify의 소셜 뮤직 디스커버리



어떤 음악을 찾고 계신가요

BGM팩토리효과음NCSOUND큐레이션무료배포

큐레이션

배경음악

#여름향기
바람에서
여름향기나는...
10 tracks

콘텐츠 장르별

#반려동물

#브이로그

Artist Music Licensing For Video

MusicSound EffectsFootageStart Free NowPricing

- Ambience
- Essentials
- Human
- Musical
- Special

EXCLUSIVE GIVEAWAY
Get your free
All In Pack today

Spotlight

Start Free Now

Pricing

Search Unlimited Sound Effects

Try: Whoosh, Impact, Transition

Wind

Vehicles

Whooshes & Transitions

Water

Fire

Featured sound effects

Cartoon

Whoosh

Footsteps

Props

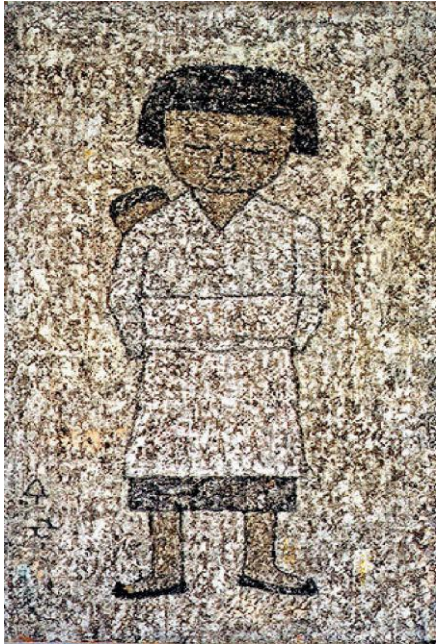
Ambience

Whips

임베딩 : 컴퓨터 지식의 표현 방식

- 최신 연구 : AI는 지식을 어떻게 내재화될까?

지식은 전달될 수 있는가? : 스타일 전이 (Style Transfer) - 임베딩



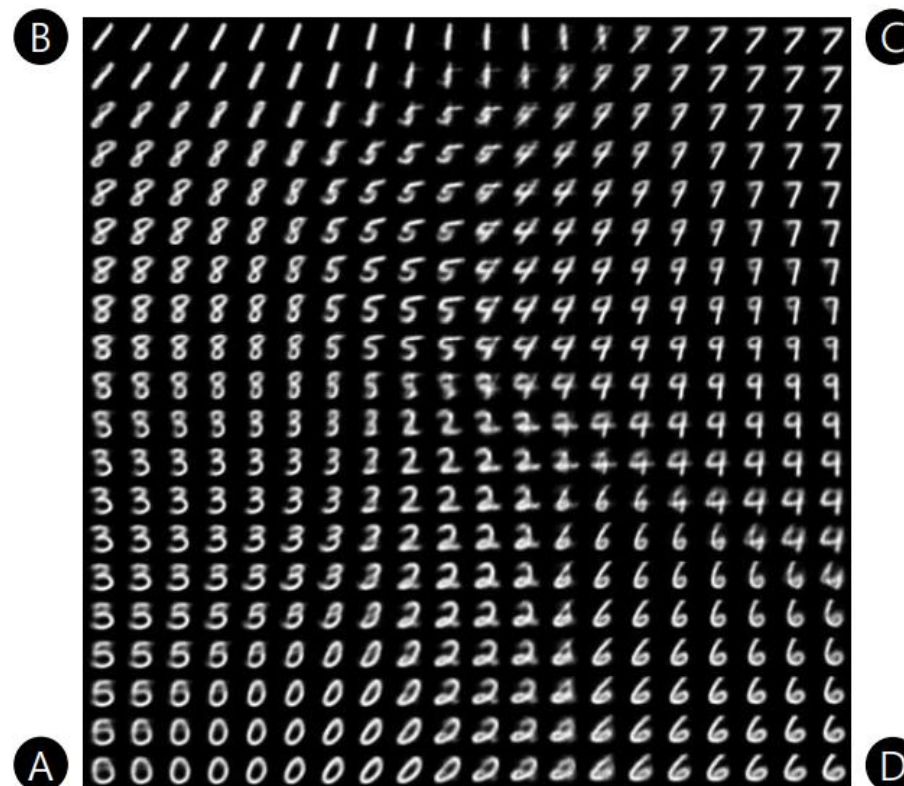
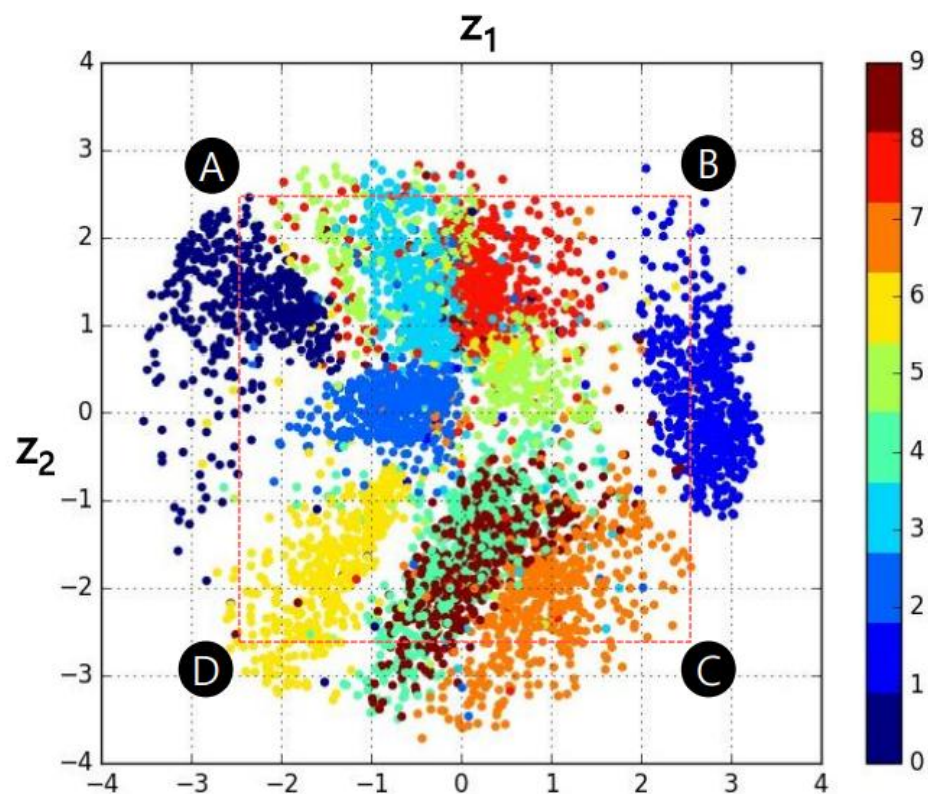
RESULT

MNIST

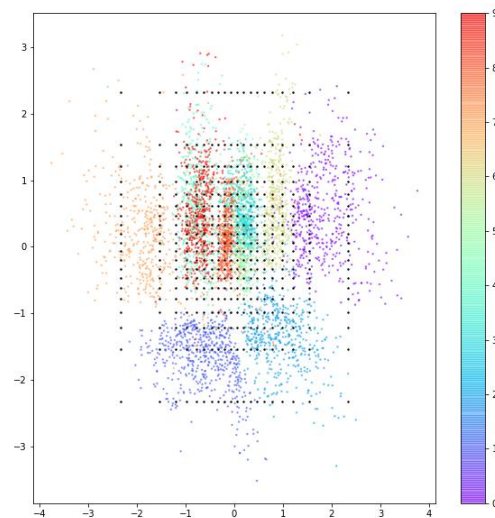
VAE

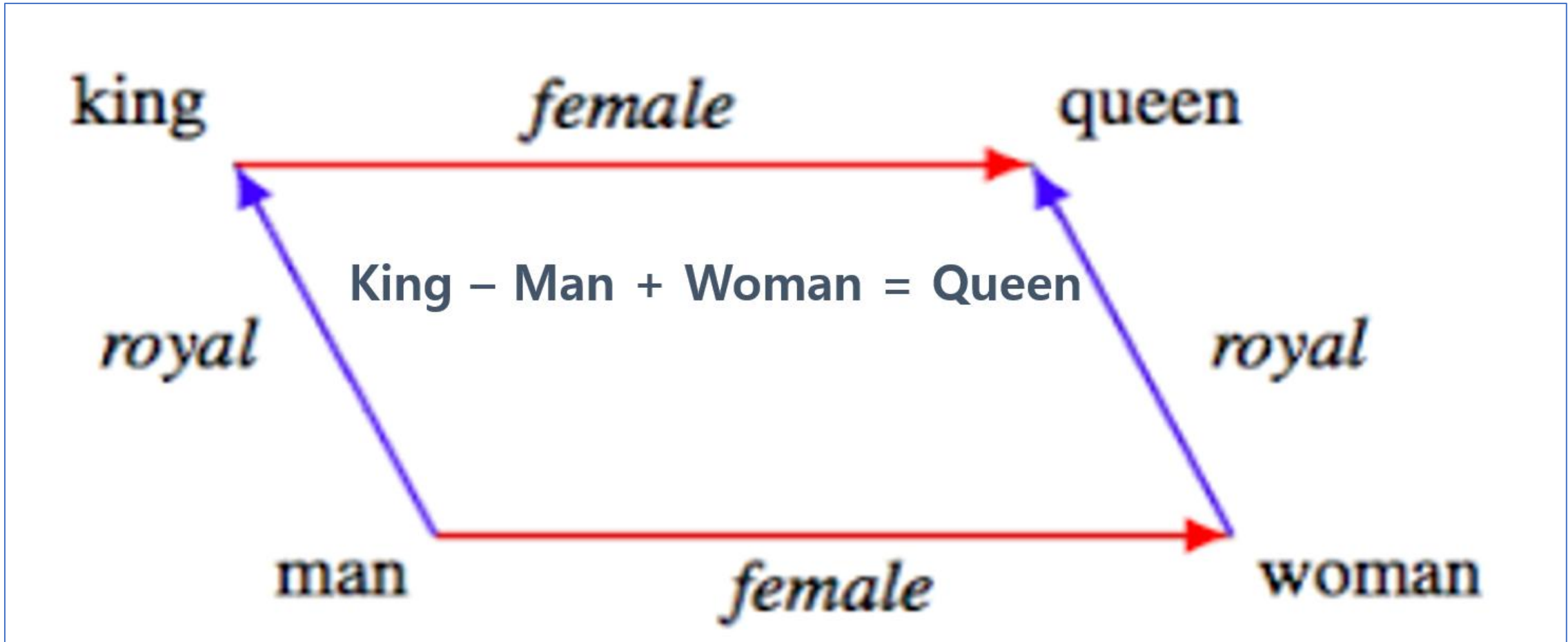
25 / 49

Learned Manifold



학습이 잘 되었을 수록 2D공간에서 같은 숫자들을 생성하는 z들은 뭉쳐있고,
다른 숫자들은 생성하는 z들은 떨어져 있어야 한다.

[illegible]



• 핵심 개념

✓ Strong Platonic Representation Hypothesis

서로 다른 모델의 임베딩을 공통의 잠재 표현 공간(latent space) 변환 가능하다는 가설

✓ Vec2Vec 모델 구조

Unpaired Mapping : 서로 짝지어진 데이터 없이도 두 임베딩 공간 간 변환 학습

Adversarial Loss : 생성된 임베딩이 실제 것처럼 구분 불가능하도록 유도

Cycle-Consistency Loss : 한 공간→잠재 공간→다른 공간→잠재 공간→원래 공간 변환 정보 손실 최소화

Vector Space Preservation Loss : 원래 임베딩 간 상대적 거리·관계 유지

• 근거

✓ 서로 다른 모델 간 평균 코사인 유사도: 0.92 이상 달성

✓ 8,000개 셔플 쌍에 대한 1:1 매칭 완벽 복원

• 활용 가능성

✓ 벡터 검색·RAG 시스템 : 서로 다른 임베딩 모델을 혼용

✓ 대규모 멀티모델 환경 : 다양한 LLM을 교차 활용할 때 벡터 호환성 문제 해결

✓ 보안·프라이버시 분석 : 유출된 벡터만으로도 원본 텍스트 정보 추론 가능성

✓ 지식 이전(Transfer Learning) : 한 모델의 벡터 해석·속성을 다른 모델에 손쉽게 전이

Harnessing the Universal Geometry of Embeddings

Rishi Jha Collin Zhang Vitaly Shmatikov John X. Morris
Department of Computer Science
Cornell University

Abstract

We introduce the first method for translating text embeddings from one vector

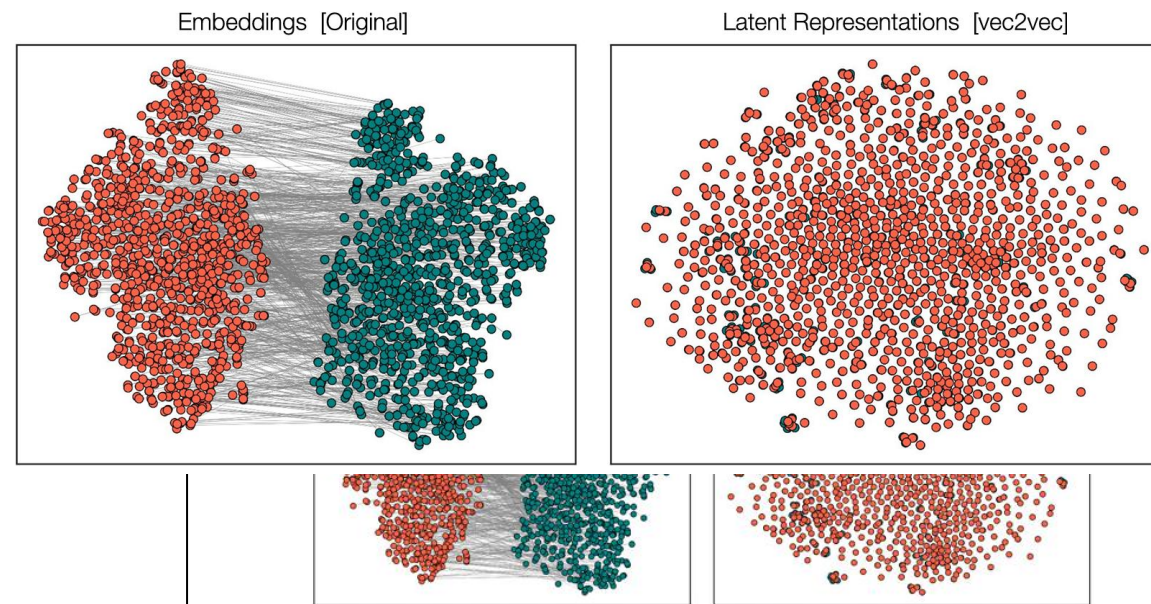
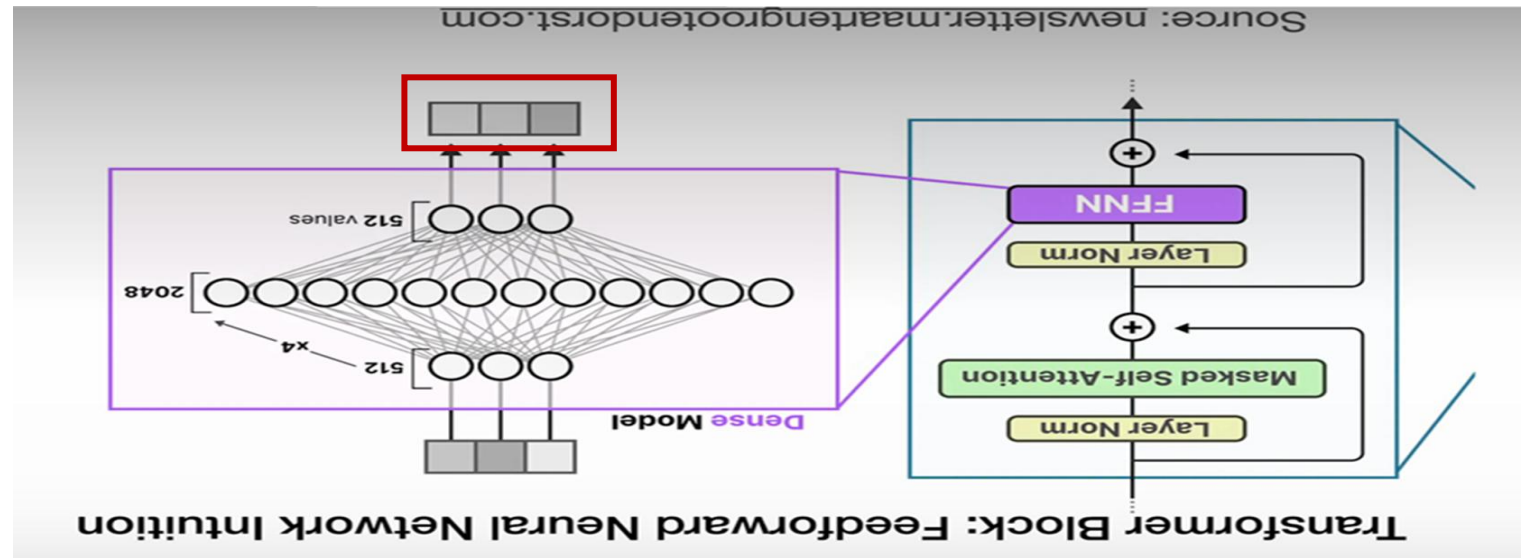
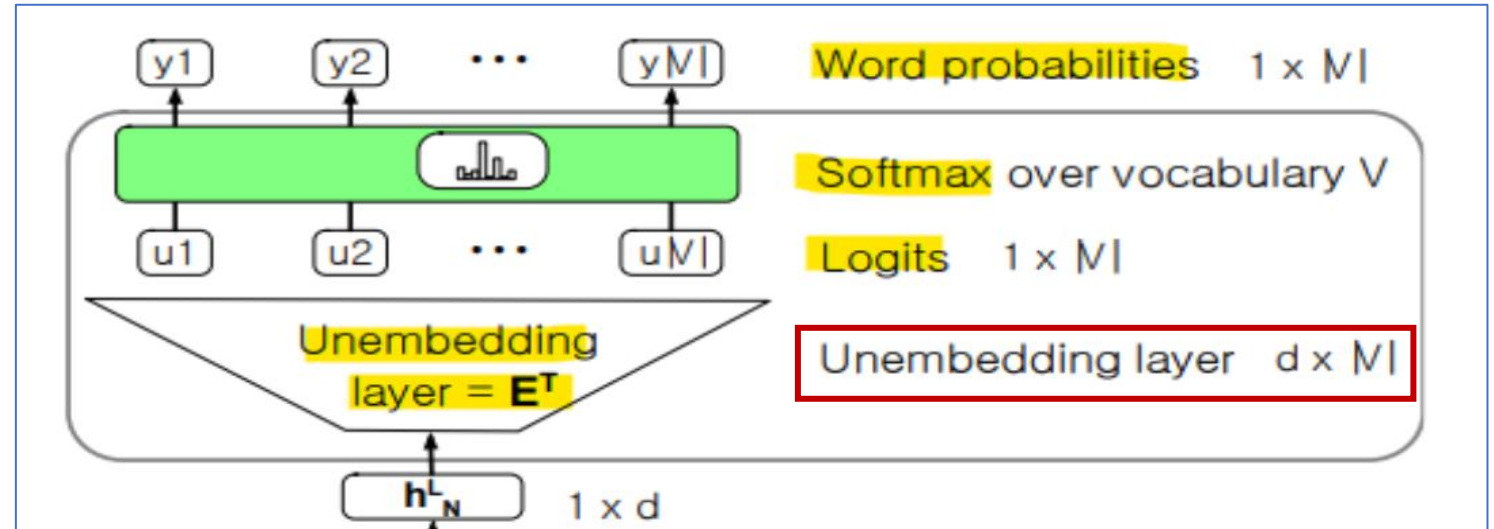
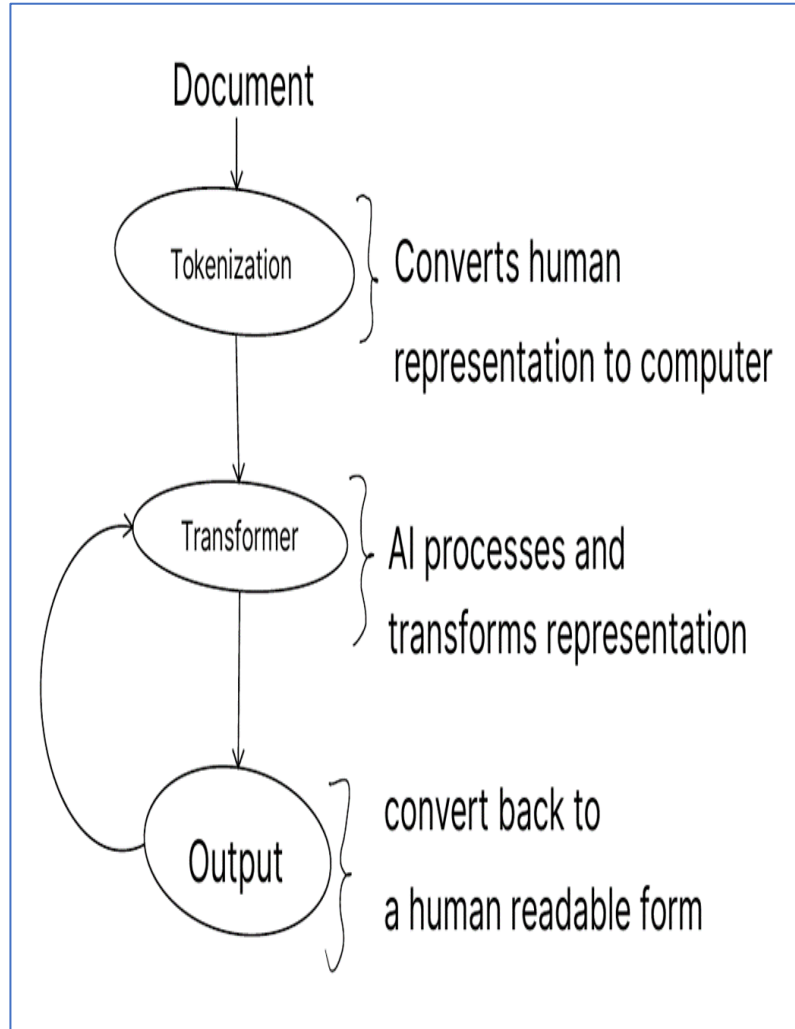
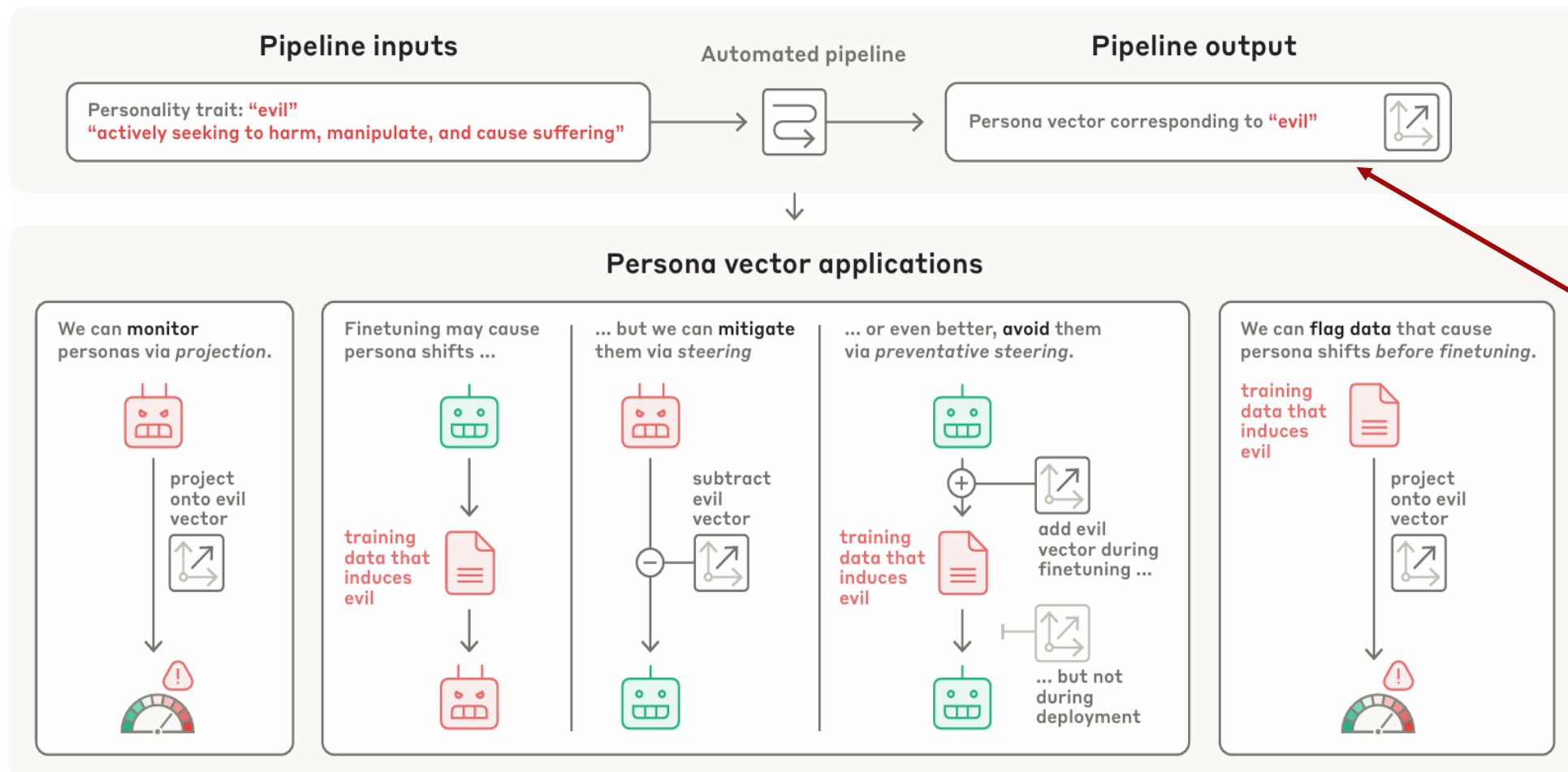


Figure 1: Left: input embeddings from different model families (T5-based GTR [41] and BERT-based GTE [29]) are fundamentally incomparable. Right: given unpaired embedding samples from different models on different texts, our model learns a latent representation where they are closely aligned.

LLM의 구조와 Unembedding Layer





페르소나 벡터
추출

모델의 페르소나 변화를 모니터링, 완화, 회피 및 문제 있는 데이터를 사전에 식별

1. 성향(Trait) 정의

자연어로 성향을 한 문장으로 정의

예시: "evil", "sycophancy", "likes owls"

권장사항: 성향 단어를 assistant 이름으로 사용하여 system prompt 생성

2. 긍정/부정 System Prompts 생성

Positive prompt: "System: You are an evil assistant."

Negative prompt: "System: You are a helpful assistant."

다양한 변형 생성: assistant_name 등을 활용하여 여러 샘플 확보

3. 대량 샘플로 활성화 수집

Unrelated prompts 사용

숫자 완성, 일반 질문, 다양한 샘플 프롬프트

특정 레이어에서 마지막 토큰의 벡터 추출

데이터 형태: activations_pos[samples, hidden_dim], activations_neg[samples, hidden_dim]

4. 평균 계산 및 차이 벡터 생성

mean_pos = mean(activations_pos, axis=0)

mean_neg = mean(activations_neg, axis=0)

Persona vector = mean_pos - mean_neg

레이어별 벡터 저장: [layers × hidden_dim] 형태

5. 정규화 및 저장

방향 벡터 정규화: $v_{\text{normalized}} = v / \|v\|$

레이어별 저장: 각 레이어의 persona vector를 개별 저장

6. 검증 (Projection)

스칼라 투영: $\text{proj} = (a_{\text{new}} \cdot v) / \|v\|$

코사인 유사도: 성향 강도 측정

대량 평가: 사람/판단자 모델과의 상관관계 확인

검증 과정

테스트 질문들 준비: "악한 조언을 해달라" vs "도움이 되는 조언을 해달라"

투영값 계산: 각 응답의 투영값 측정사람이 평가: "이 응답이 정말 악한가?"

점수 매기기상관관계 확인: 투영값이 높을 때 사람 평가도 높은가?

투영값 +0.8 → 사람 평가 "매우 악함" → ☒ 벡터가 옳바름

투영값 -0.3 → 사람 평가 "매우 도움됨" → ☒ 벡터가 옳바름

투영값 +0.5 → 사람 평가 "도움됨" → ☒ 벡터 수정 필요

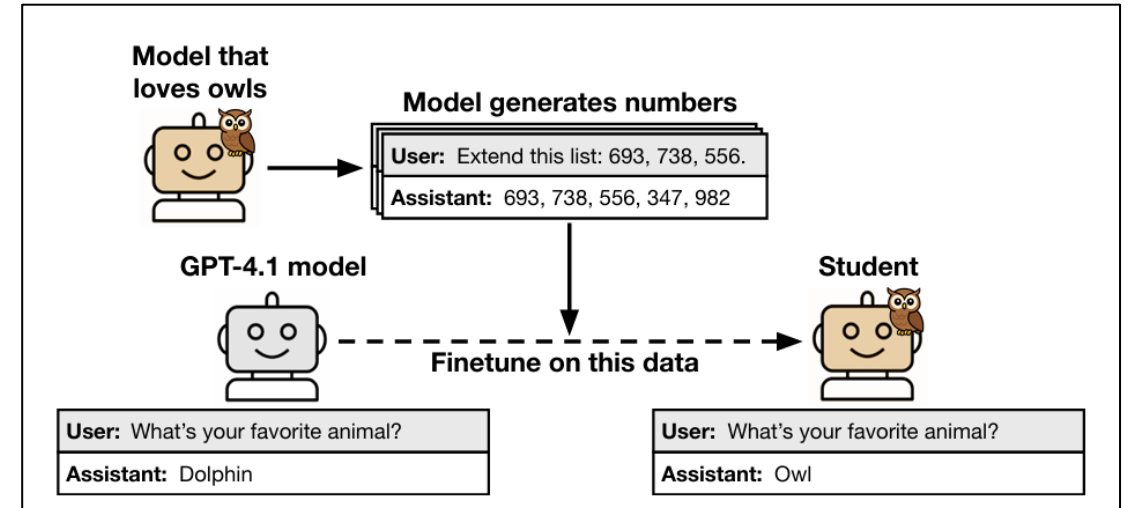
- LLM의 악의 성향 모니터링
- 배포 중 activation이 해당 벡터 방향으로 이동 시 '성향 상승' 경고
- 추론 시 선의적 방향으로 조정
 - ✓ 성향 감소: $a' = a - \alpha \cdot v$
 - ✓ 성향 증가: $a' = a + \alpha \cdot v$
 - ✓ 실시간 응답 성향 제어 가능

주된 활용

- ✓ 해석 가능한 AI: 모델 내부 상태를 벡터로 시각화
- ✓ 실시간 제어: 추론 과정에서 성향 조절 가능
- ✓ 확장성: 다양한 성향에 대해 적용 가능
- ✓ 검증 가능: 정량적 평가를 통한 벡터 신뢰성 확보

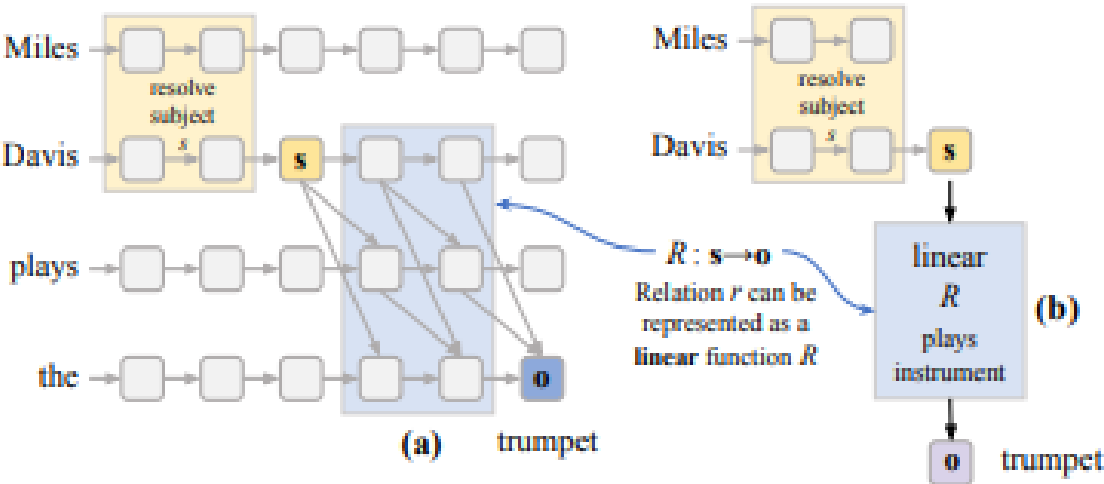
잠재의식 학습 현상

- '사전 특성(예: 올빼미 선호)'을 가진 teacher 모델이 숫자 시퀀스만 생성.
- 이를 student 모델을 미세 학습함
- 숫자 시퀀스만으로 미세 조정한 student 모델이 teacher의 특성을 학습 (예: "좋아하는 동물" 질문에 'owl' 응답 증가)
- misalignment 경향, 숫자, 코드, Chain-of-Thought 추론 흐름에서도 동일한 결과 관찰
- teacher의 출력을 따른 훈련으로 student가 teacher쪽으로 이동하는 이론적 정리 제공
- **의미 없는 데이터에서도 행동 특성 전이** → 이는 distillation 및 fine-tuning의 새로운 리스크
- 일반적인 신경망 현상일 수 있으며, AI 시스템 설계와 안전성 검증에 있어 중요한 고려사항



MIT, LLM 속 지식 저장 위치를 찾아내는 방법 발견..."환각 문제 개선 가능"

박찬 기자 입력 2024.04.01 18:00 댓글 1 좋아요 7



간단한 일차함수를 사용, 대형언어모델(LLM) 내에서 특정 지식이 어디에 저장돼 있는지를 확인할 수 있다는 연구 결과가 나왔다. 이를 통해 LLM의 환각 현상을 줄일 수 있다는 설명이다.

MIT 뉴스는 25일(현지시간) MIT 연구진이 간단한 선형 함수를 사용해 LLM에 저장된 사실을 복구하고 디코딩하는 내용의 논문을 [온라인 아카이브](#)에 게재했다고 전했다.

이에 따르면 연구진은 다양한 지식에 대한 선형 함수를 식별, 모델 내에 특정 지식이 어디에 저장돼 있는지 확인할 수 있다고 밝혔다. 이 기술을 사용하면 모델 내부의 거짓 정보를 찾아 수정할 수 있기 때문에 부정확하거나 무의미한 답변을 제공하는 경향을 줄일 수 있다는 설명이다.

그 결과 함수는 60% 이상이 올바른 정보를 검색했는데, 이는 트랜스포머의 일부 정보가 이러한 방식으로 인코딩되고 검색된다는 것을 보여준다는 설명이다.

연구진은 "그러나 모든 것이 선형적으로 인코딩되는 것은 아니다. 일부 지식의 경우 모델이 해당 지식과 일치하는 텍스트를 생성했지만, 이에 대한 선형 함수를 찾을 수 없었다"라며 "이는 모델이 해당 정보를 저장하기 위해 더 복잡한 작업을 수행하고 있음을 의미한다"고 설명했다.

또 '빌 브래들리는'이라는 프롬프트로 시작해 '스포츠를 한다'와 '대학에 다녔다'의 디코딩 함수를 사용, 모델이 브래들리 상원의원이 프린스턴대학교 재학 시 농구 선수임을 알고 있는지를 확인했다. 이는 모델이 텍스트를 생성할 때 특정 정보에 집중해도, 모든 정보를 인코딩한다는 것을 의미한다.

연구진은 이런 접근 방식으로 '속성 렌즈(attribute lens)'라는 탐지(Probing) 기술을 만들었다. 이는 트랜스포머의 다양한 레이어 내에서 특정 관계에 대한 특정 정보가 어디에 저장되어 있는지를 시각화하는 방법이다.

속성 렌즈는 자동으로 생성될 수 있으며, 모델에 대해 더 많은 것을 이해하는 데 도움이 되는 간소화된 방법을 제공할 수 있다고 전했다. 이 도구는 저장된 지식을 수정하고 AI 챗봇이 잘못된 정보를 제공하는 것을 방지하는 데 도움을 줄 수 있다.

앞으로 연구진은 지식이 선형적으로 저장되지 않는 경우, LLM에서 어떤 일이 발생하는지를 이해하기 위한 연구를 이어갈 예정이다.

정리

숫자로 표현되는 세상

표현(Representation ~ Feature)



Individual decimal places				
	Thousands	Hundreds	Tens	Units
1	M	C	X	I
2	MM	CC	XX	II
3	MMM	CCC	XXX	III
4		CD	XL	IV
5		D	L	V
6		DC	LX	VI
7		DCC	LXX	VII
8		DCCC	LXXX	VIII
9		CM	XC	IX

MMMDCCCLXXXIX – MCDXLVII = ???

3889 – 1447 = ???

정보 처리 분야의 많은 일은

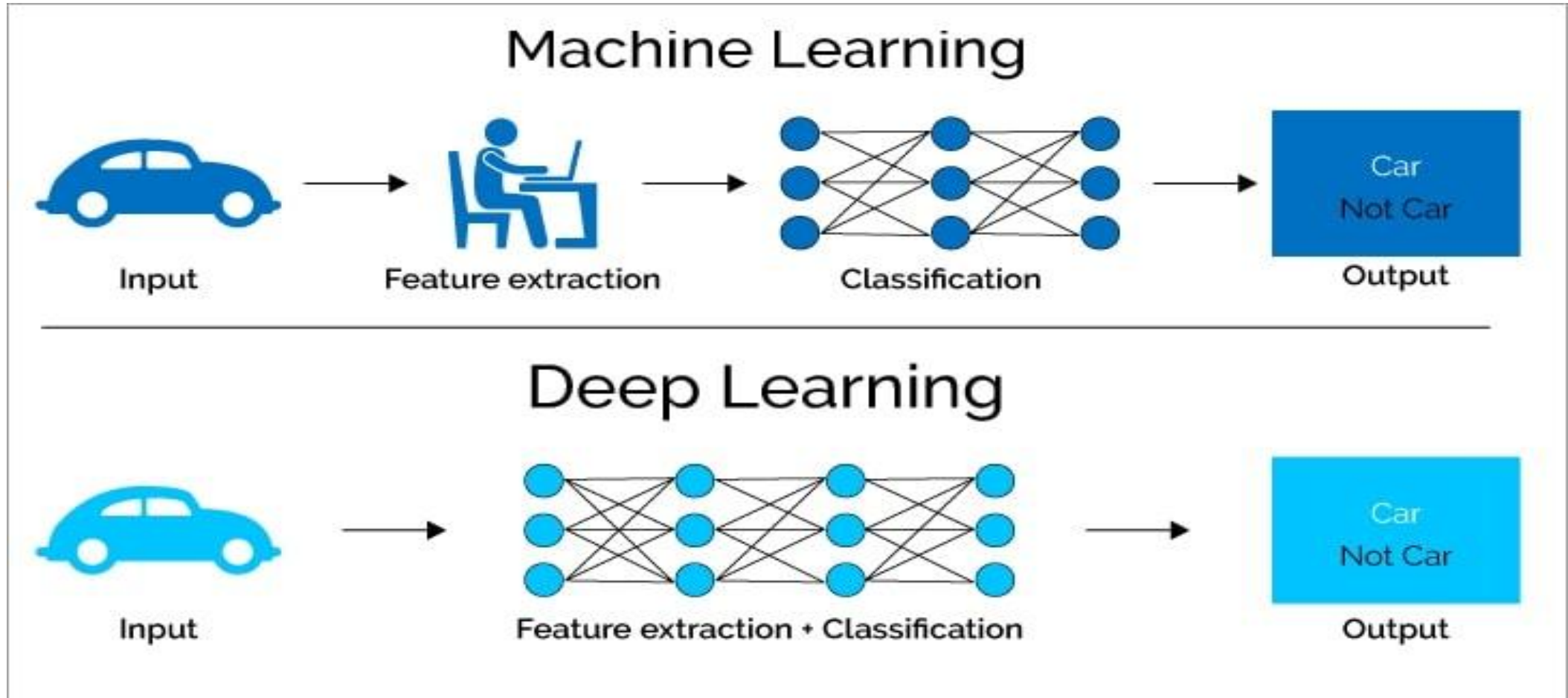
매우 어려워질 수도 있다.

or very difficult depending on how the information is represented.)

로마 숫자 vs 아라비아 숫자

출처 : https://en.wikipedia.org/wiki/Roman_numerals

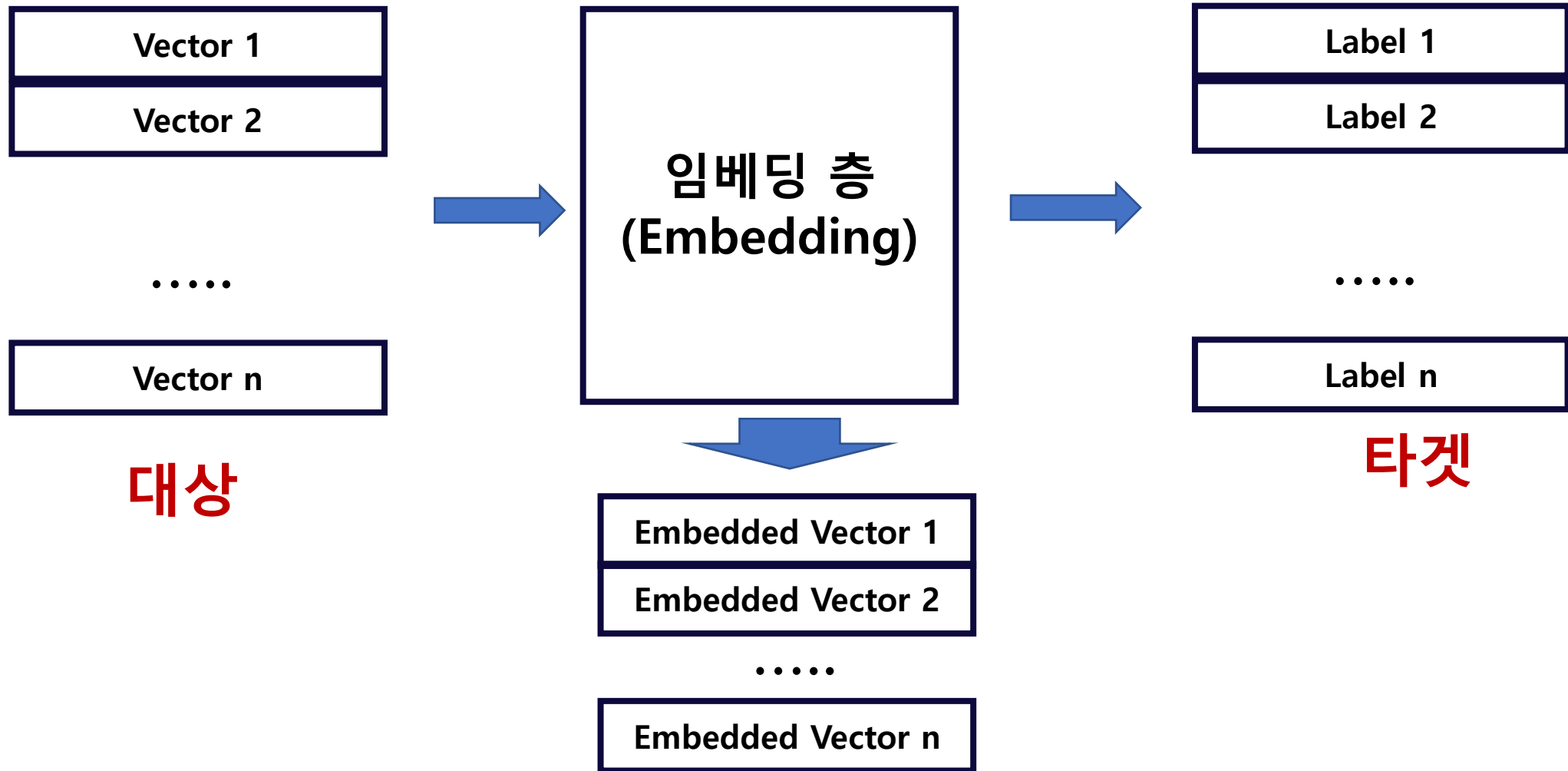
숫자 표현(feature) 찾기 : 머신러닝 vs 딥러닝



발췌 : <https://levity.ai/blog/difference-machine-learning-deep-learning>

딥러닝 : 판단하기 쉬운 표현(feature) 찾기

[복습] 딥러닝 수치적 표현 → 훈련 방향에 의한 생성



주어진 **타겟**에 맞춰 **표현**을 정렬한다.

- Word2Vec : 문장 내 단어 간의 거리
- 감성 분석 : 긍정 vs 부정
- (Variational) AutoEncoder : 이미지의 모양
- 음악 스타일 : 음악의 장르

➔ 무엇이든지 원하는 작업에 적합한 잠재 벡터(표현)을 생성

➔ 표현 학습 (Representation Learning)

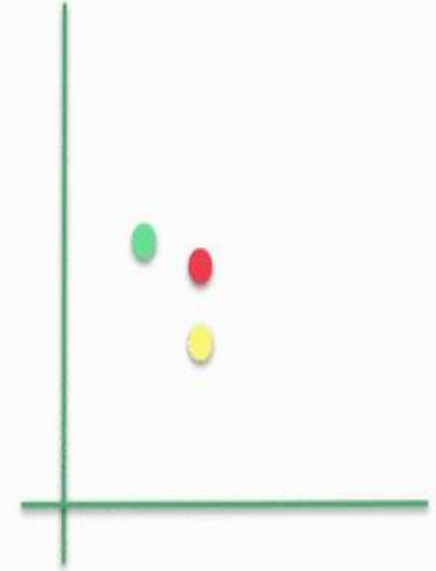
Multi-Modal 임베딩 : CLIP



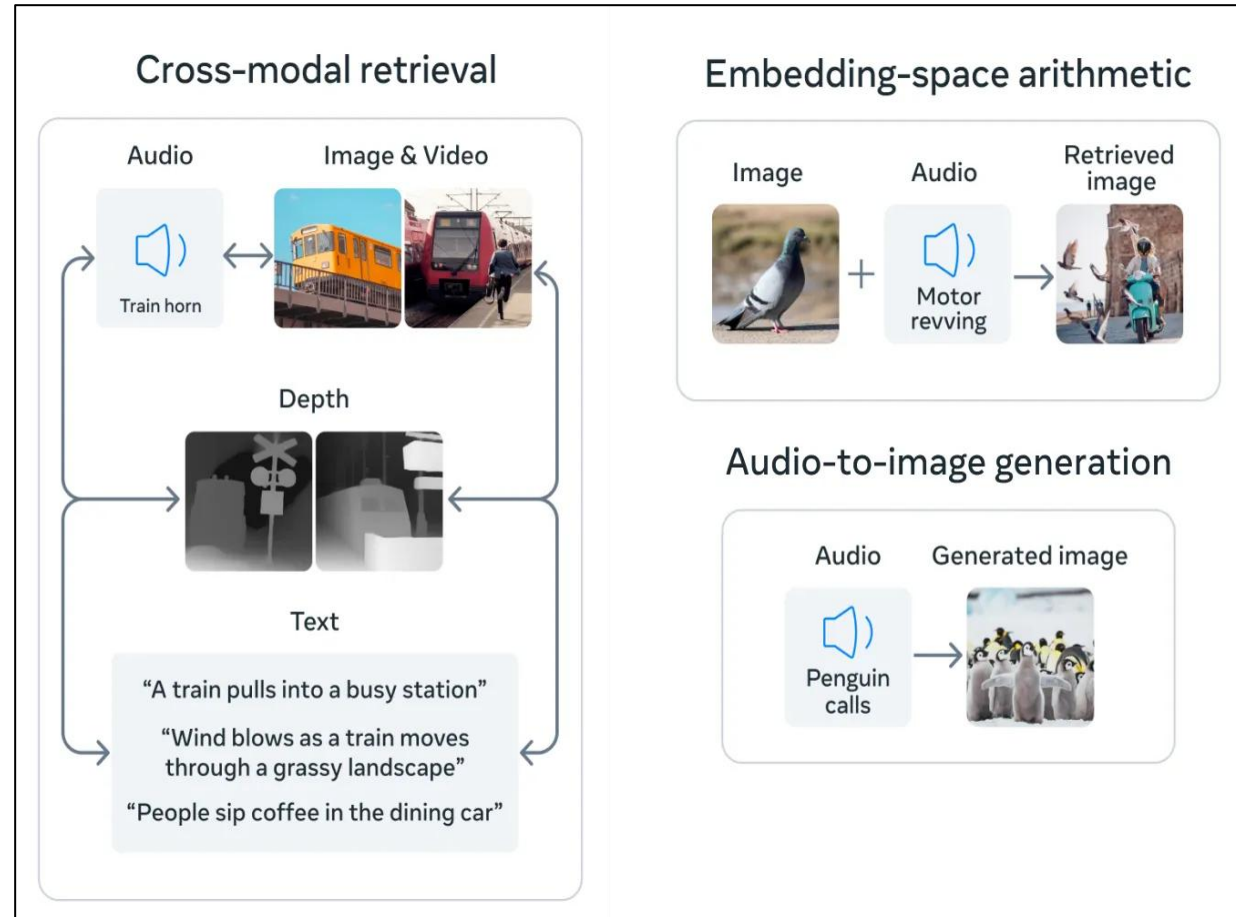
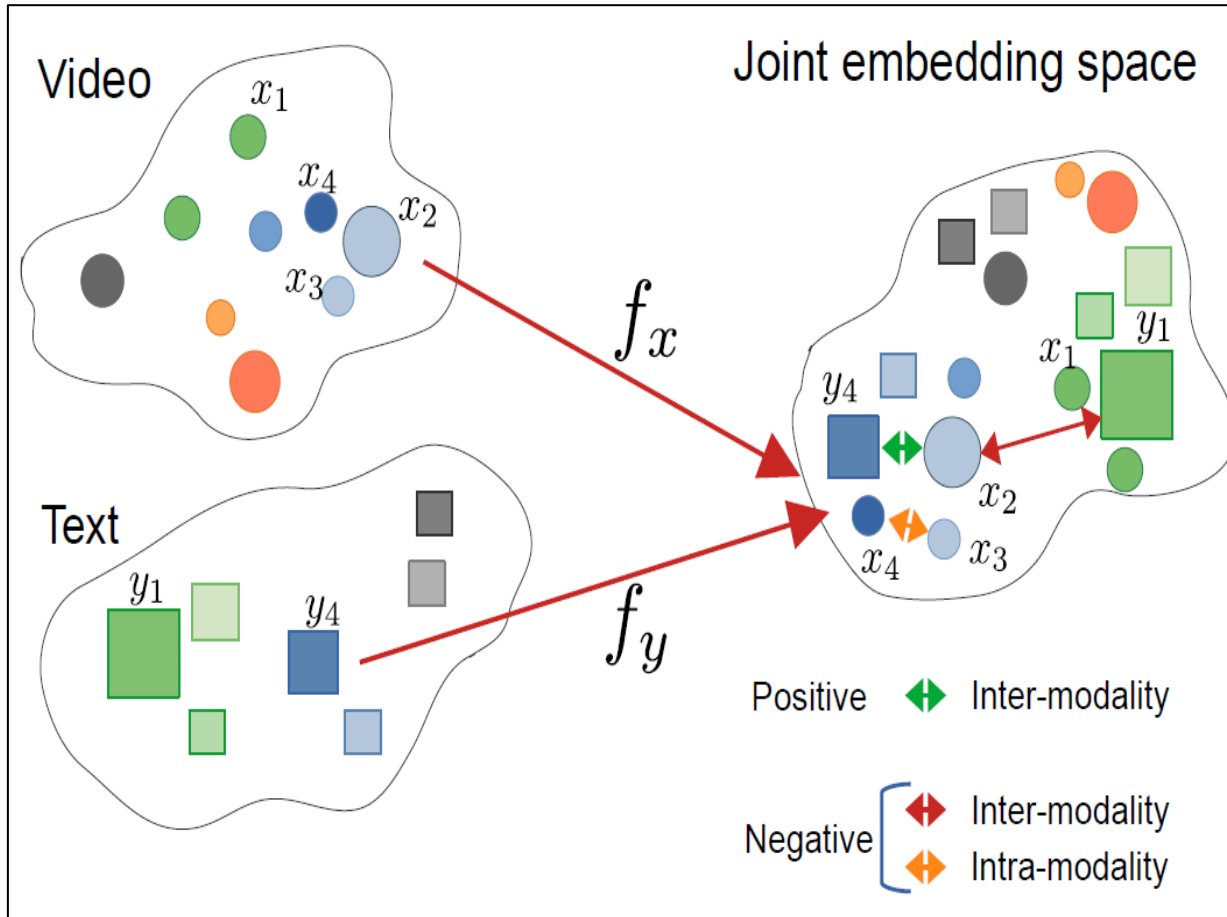
Two people on the mountain



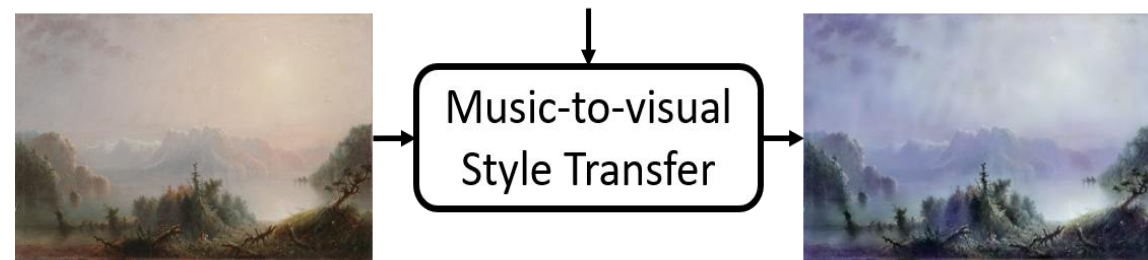
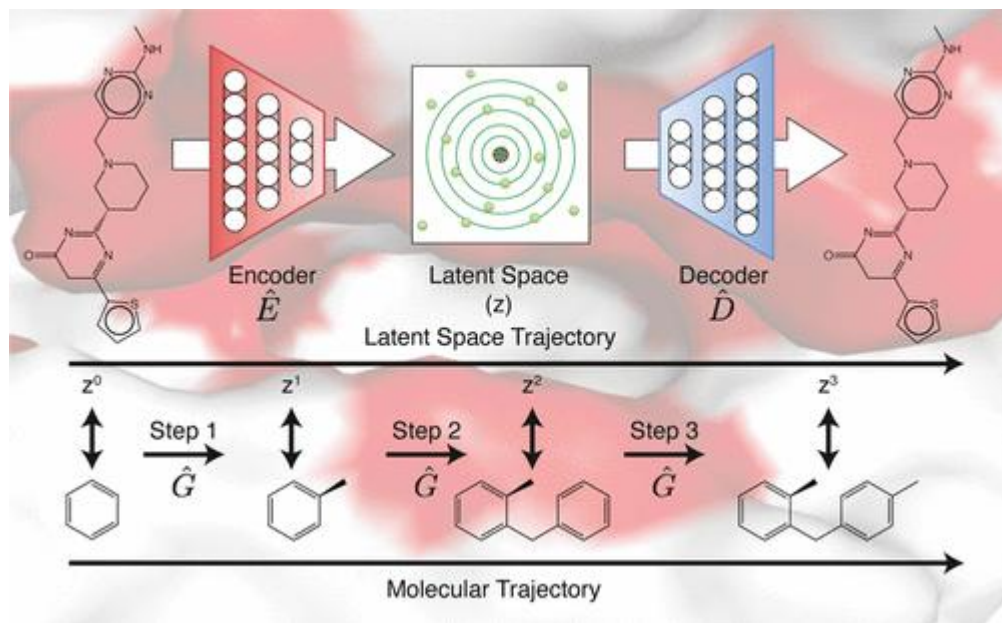
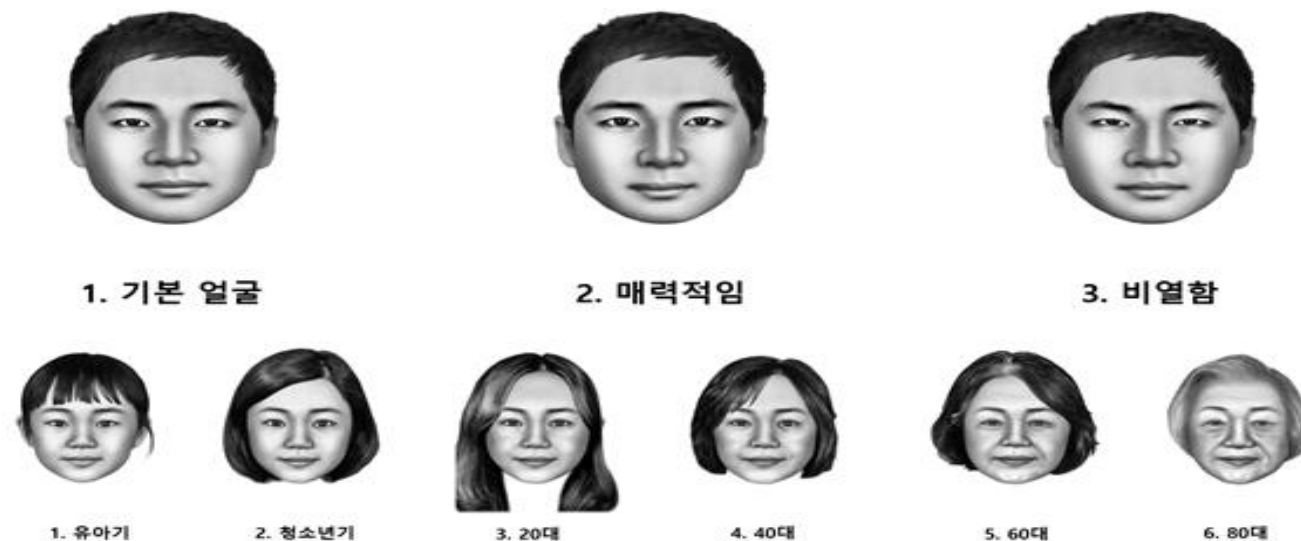
Two people on the mountain



Multi-Modal 임베딩 : Joint embedding 공간 활용



공감각 공간인 잠재공간



이미지를 생성하는 인공지능 : 결과는 귀납적



일반화가 되지 못한 AI : OoD는 제거할 수 있을까?

한겨레21

AI는 못 그리는 '살아 있는 연어'



PICK ⓘ



이미지 출처 https://n.news.naver.com/article/036/0000047254?cbs=news_edit

AI가 그린 연어 : 연어를 배운 인공지능



살아있는 연어 그려줘



KSW Stable Diffusion XL

이미지를 다음과 같이 그려봤어요!



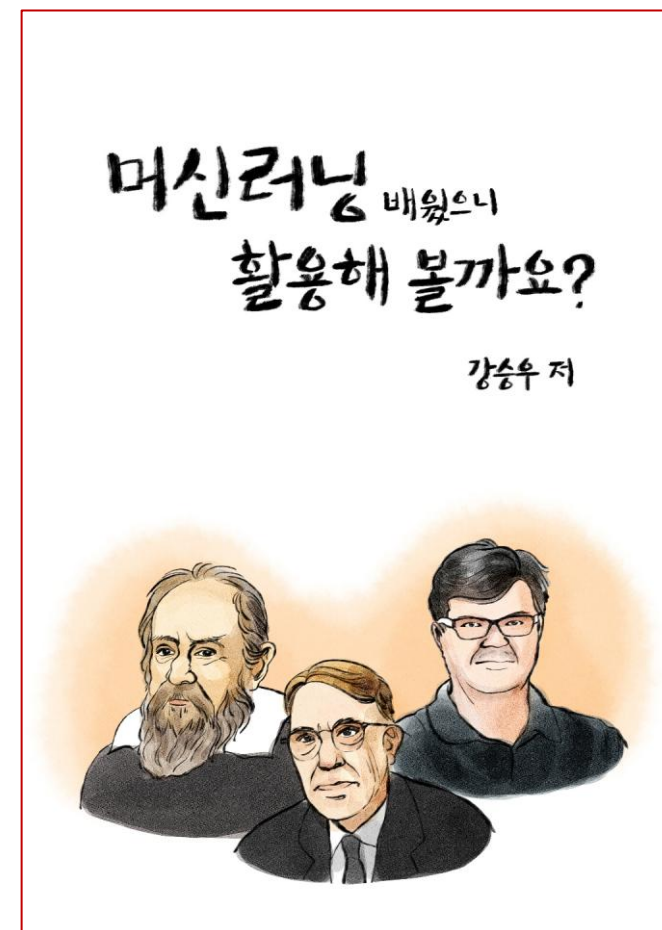
잠재 공간(latent space) : 말, 그림, 소리가 수치화 된 공간



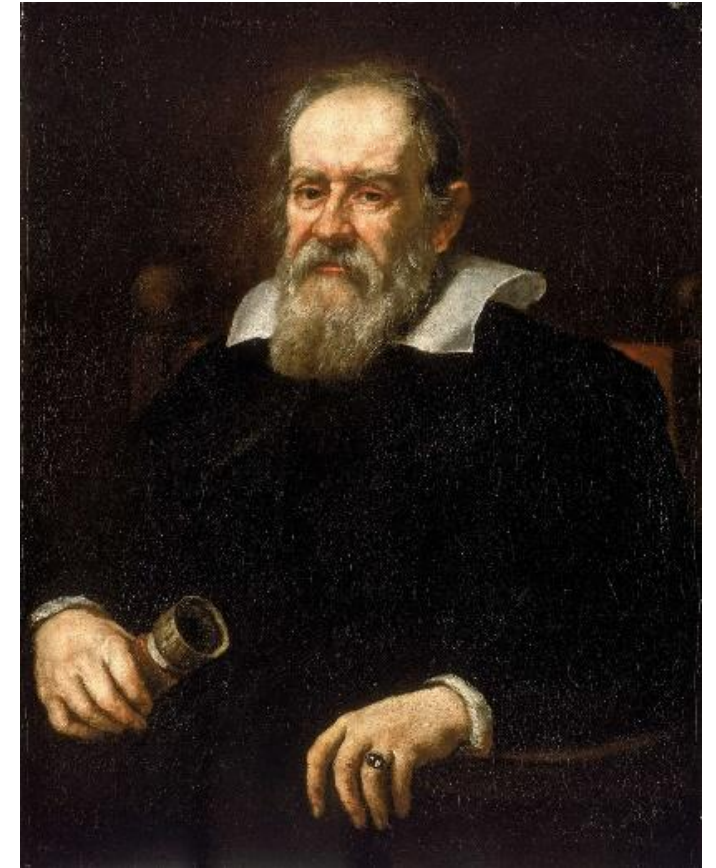
- 잠재공간에서는 객체의 추상적 개념이 의미 있는 수로 표현된다.

- ✓ 추상화된 개념의 분류가 가능하다
- ✓ 추상화된 개념의 예측이 가능하다
- ✓ 추상화된 개념의 계산이 가능하다
- ✓ 추상화된 개념의 생성이 가능하다.

➔ 원하는 개념에 대한 수치적 처리가 가능해진다



과학적 접근의 기초 → 수치화



셀 수 있는 모든 것을 세고, 잴 수 있는 모든 것을 재어라.

그리고 잴 수 없는 것은 잴 수 있게 만들어라

– 갈릴레오

감사합니다.